Data Analytics (PETR 6397)

Regression

Dr. Ahmad Sakhaee-Pour

Petroleum Engineering Department

University of Houston

Spring 2025

Discussed topics

Regression

Linear regression

Cost functions

Gradient Descent

Batch Gradient Descent

Stochastic Gradient Descent

Mini-batch Gradient Descent

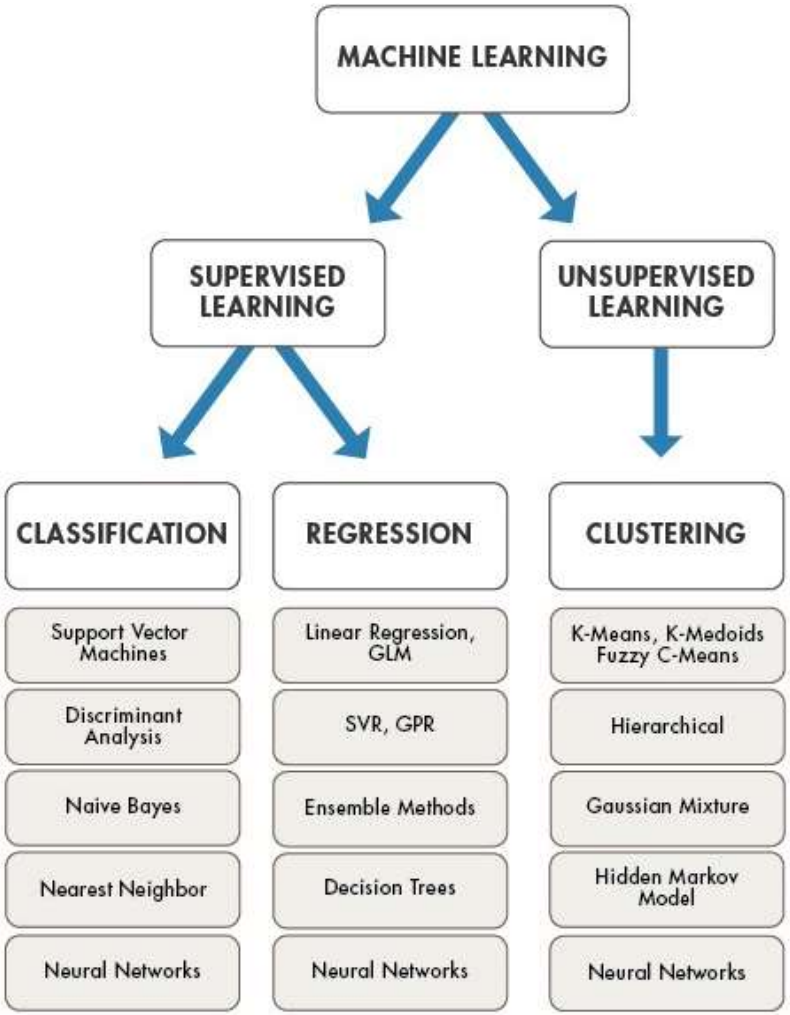
Regularized: Ridge regression, Lasso regression, Elastic Net regression

K-nearest Neighbors regressor

K-fold cross-validation

Logistic regression

Regression


MACHINE LEARNING
SUPERVISED LEARNING
UNSUPERVISED LEARNING
CLASSIFICATION
REGRESSION
CLUSTERING
Support Vector Machines
Discriminant Analysis
Naive Bayes
Nearest Neighbor
Neural Networks
Linear Regression, GLM
SVR, GPR
Ensemble Methods
Decision Trees
Neural Networks
K-Means, K-Medoids Fuzzy C-Means
Hierarchical
Gaussian Mixture
Hidden Markov Model
Neural Networks

Regression analysis

- Regression is a statistical process to estimate the relationships among variables
  - Dependent variables
  - Independent variables
- There are various types of regression algorithms, and each is suitable for different purposes:
  - Linear regression
  - Ridge regression
  - Lasso regression
  - ….

Linear regression

- Best straight line with the lowest error

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \cdots + \theta_n x_n$$

$\hat{y}$: predicted value

n: number of features (input variables)

x_i: feature value

θ_i: model parameter (tuning parameter)

θ_0: bias

$\theta_1, \theta_2, \ldots, \theta_n$: weights

Linear regression (vector)

$$\hat{y} = h_{\theta}(\mathbf{x}) = \boldsymbol{\theta}.\mathbf{x}$$

h_{θ}: hypothesis function (definition)

$$\boldsymbol{\theta} = [\theta_0, \theta_1, \theta_2, \ldots, \theta_n]$$

$$\mathbf{x} = [x_0, x_1, x_2, \ldots, x_n]$$

Q: what is the value of x_0?

Cost function (or loss function)

- Cost function (or evaluation metrics) quantitively determines the performance of the model
- **1. Mean absolute error (MAE):**
  - Average of the absolute value of the difference between actual and predicted values

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n} |\widehat{y}_i - y_i|$$

n: number of samples

y_i: actual value

$\hat{y}_i$: predicted value

Cost function (continued)

- **Mean squared error (MSE):**

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

- **Root mean squared error (RMSE):**
  - RMSE is basically the square root of MSE

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2}$$

- It is easier to minimize MSE than RMSE
- Minimizing MSE or RMSE leads to similar tuning parameters
- MSE is used more often

Minimizing the cost function leads to an optimal scenario

- There is an analytical solution for minimizing the MSE

$$\mathrm{MSE}(\theta) = \frac{1}{n}\sum(\hat{y}_i - y_i)^2$$

$$\mathrm{MSE}(\theta) = \frac{1}{n}(\mathbf{\theta x} - \boldsymbol{y})^T(\mathbf{\theta x} - \boldsymbol{y})$$

$$\mathrm{MSE}(\theta) = \frac{1}{n}\left(\mathbf{\theta^T x^T} - \boldsymbol{y^T}\right)(\mathbf{\theta x} - \boldsymbol{y})$$

$$\mathrm{MSE}(\theta) = \frac{1}{n}\left(\mathbf{\theta^T x^T \theta x} - \mathbf{\theta^T x^T}\boldsymbol{y} - \boldsymbol{y^T}\mathbf{\theta x} + \boldsymbol{y^T y}\right)$$

$$\mathrm{MSE}(\theta) = \frac{1}{n}\left(\mathbf{\theta^T x^T \theta x} - \mathbf{2\theta^T x^T}\boldsymbol{y} + \boldsymbol{y^T y}\right)$$

$$\frac{\partial}{\partial\theta}\mathrm{MSE}(\theta) = \frac{1}{n}\left(\mathbf{x^T \theta x} + \mathbf{x^T \theta x} - \mathbf{2x^T}\boldsymbol{y} + \mathbf{0}\right) = \frac{2}{n}\mathbf{x^T}(\mathbf{\theta x} - \boldsymbol{y}) = 0$$

Normal equation minimizes the error, but it is not always applicable

$$\widehat{\boldsymbol{\theta}} = \boldsymbol{\theta}\ (\mathbf{minimum\ error}) = \left(\mathbf{x}^{\mathbf{T}}\mathbf{x}\right)^{-\mathbf{1}}\mathbf{x}^{\mathbf{T}}\boldsymbol{y}$$

$\boldsymbol{y}$ Includes target values

Reasons for inapplicability:

1. $\mathbf{x}^{\mathbf{T}}\mathbf{x}$ may not be invertible
2. Matrix inversion is computationally expensive

Singular value decomposition addresses the shortcomings to some extent

Singular value decomposition (SVD) solution for minimizing MSE

Decomposition:

$$\mathbf{x} = \mathbf{V}\Sigma\mathbf{U}^{\mathbf{T}}$$

$\mathbf{V}$: orthogonal matrix

Σ: diagonal matrix

$\mathbf{U}^{\mathbf{T}}$: orthogonal matrix

$$\widehat{\boldsymbol{\theta}} = \boldsymbol{\theta}\ (\mathbf{minimum\ error}) = \mathbf{V}(\Sigma^{\mathbf{T}}\Sigma)^{-\mathbf{1}}\Sigma^{\mathbf{T}}\mathbf{U}^{\mathbf{T}}\mathbf{y}$$

Computational cost prevents us from using the Normal equation when there are many input variables

- If there are *n* variables:
  - Normal equation: ~ O(n^2.4) to O(n^3)
  - Singular value decomposition: O(n^2)

- SVD is better than the Normal equation, but its cost is still prohibitive
- n:10,000
  - Normal equation: 3.98e-9 to 1e12
  - SVD: 1e8

Gradient Descent

- It initializes the tuning parameters of the regression model randomly
- Then, it finds the gradient of the cost function with respect to the variables
- It descends (goes downward) to lower the cost
- It iterates this updating to minimize the cost

$$\boldsymbol{\theta} \coloneqq \boldsymbol{\theta} - \eta \boldsymbol{\nabla} \mathrm{MSE}(\boldsymbol{\theta}) = \boldsymbol{\theta} - \eta \boldsymbol{\nabla} \mathbf{J}(\boldsymbol{\theta})$$

$\boldsymbol{\theta}$ includes the tuning parameters (vector)

η Is the learning rate (scalar)

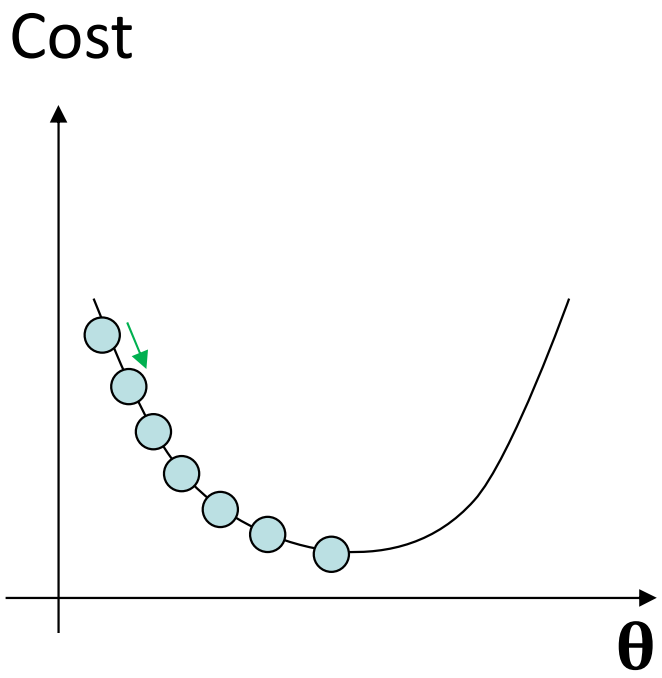
Expansion of the gradient term

- $\nabla \mathbf{J}(\boldsymbol{\theta}) = \nabla \mathrm{MSE}(\boldsymbol{\theta}) = \begin{Bmatrix} \frac{\partial}{\partial \theta_0} \mathrm{MSE}(\theta_0, \theta_1, \theta_2, \dots, \theta_n) \\ \frac{\partial}{\partial \theta_1} \mathrm{MSE}(\theta_0, \theta_1, \theta_2, \dots, \theta_n) \\ \frac{\partial}{\partial \theta_2} \mathrm{MSE}(\theta_0, \theta_1, \theta_2, \dots, \theta_n) \\ \vdots \\ \vdots \\ \frac{\partial}{\partial \theta_n} \mathrm{MSE}(\theta_0, \theta_1, \theta_2, \dots, \theta_n) \end{Bmatrix}$
- Gradient shows upward direction, so we use its negative
- The learning rate controls pace of going downward

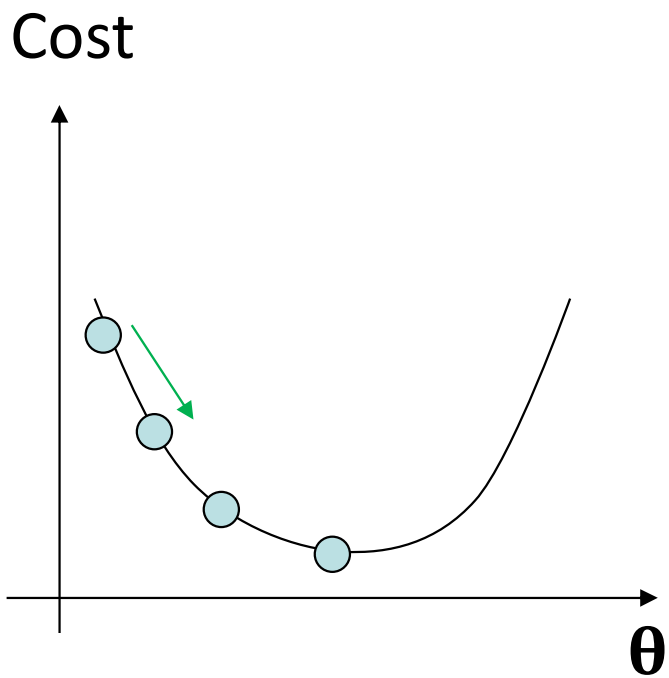
$$\boldsymbol{\theta} \coloneqq \boldsymbol{\theta} - \eta \nabla \mathrm{MSE}(\boldsymbol{\theta}) = \boldsymbol{\theta} - \eta \nabla \mathbf{J}(\boldsymbol{\theta})$$

Effects of the learning rate on the Gradient Descent

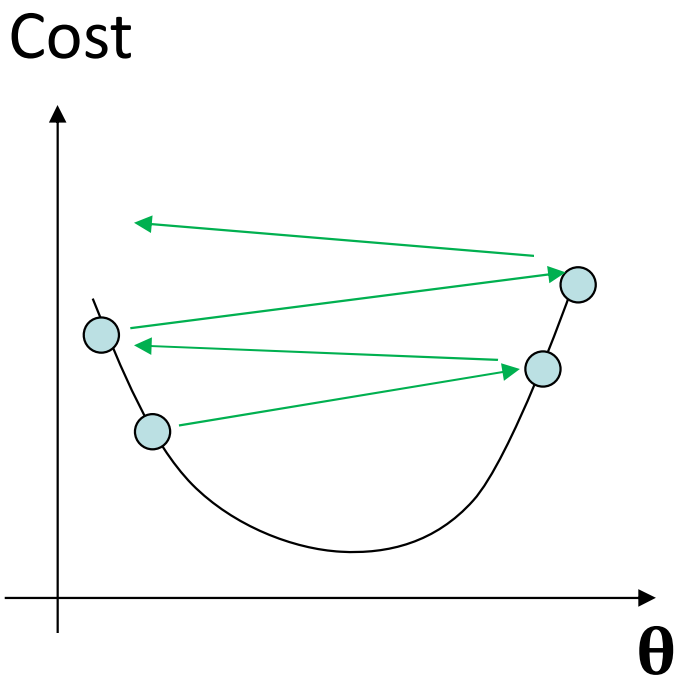
- Step size depends on the learning rate



Small rate



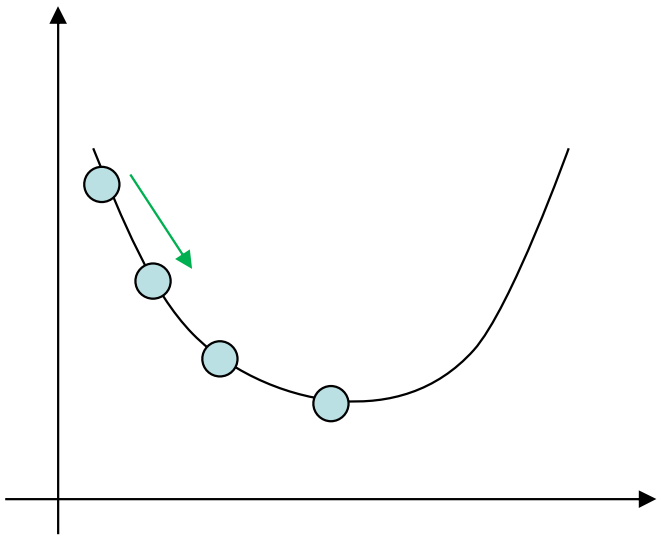
Medium rate



Large rate

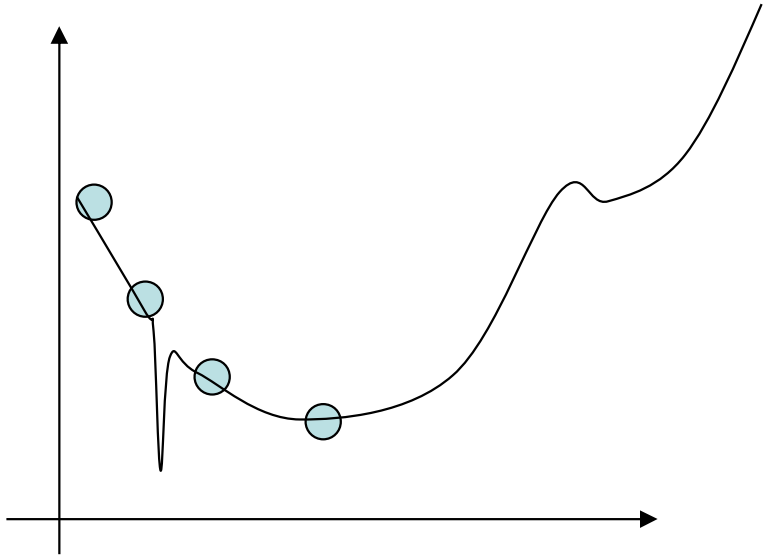
Cost function (MSE) of linear regression is convex, unlike many other regression models

Cost



Linear regression model

Cost



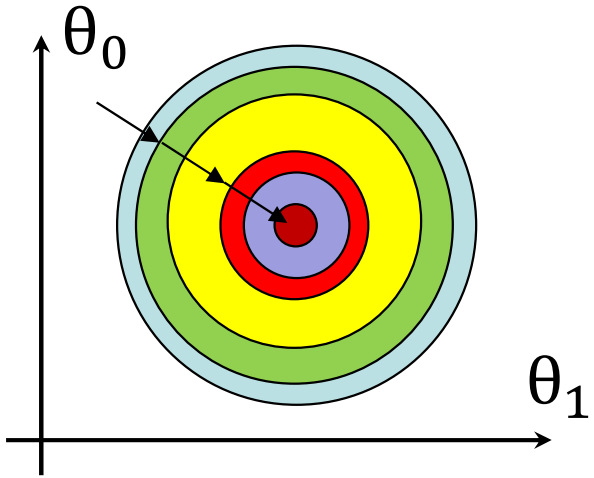
More complex model

Cost of Gradient Descent

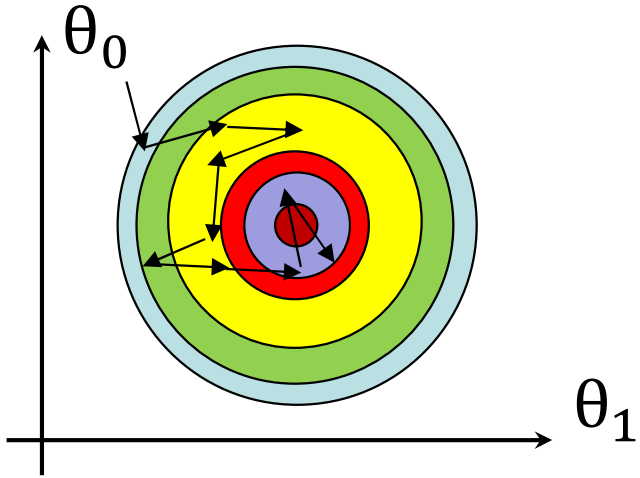
- If we have *n* variables and *m* tuning parameters in the model, its computation cost is ~ O(*m.n*) for each iteration
- If you have 10,000 observations (*n*) and plan to find the best line with 10 variables (m): 10^4.10=10^5
- Gradient Descent is faster than the Normal equation but still slow because it uses all the training data with *n* variables
- Solution: choose a random set (obviously, this is smaller than *n*).

Stochastic Gradient Descent

- It uses a random set of observations (training set) to minimize the cost function at every step
- It behaves stochastically (less regular) compared to the Gradient Descent
- Cost function is shown using contours



Gradient Descent



Stochastic Gradient Descent

Main features of Stochastic Gradient Descent

- Its results are good (not necessarily optimal)
- It is likely to skip the local minimum in its search to minimize the cost function
- Note: always shuffle the data, especially when you use Stochastic Gradient Descent because your solution is based on stochasticity (randomness)

Batch versus mini-batch

- Batch: the entire set of data
- Mini-batch: a small set of data (for instance, 100 samples from 10000 observations)
- At each iteration:
  - Stochastic Gradient Descent uses one random sample
  - Batch Gradient Descent uses the entire data at each to minimize the
  - Mini-batch Gradient Descent uses a random set

Epoch versus iteration

- Each iteration corresponds to a single update of the model weights (single update)

- Each epoch corresponds to going through the entire data once (full pass)

- Q1: Suppose we have 10,000 observations (data) and use a Mini-Batch Gradient Descent with a mini-batch size of 500. What is the relation between iteration and epochs?

- Q2: What if we use Stochastic Gradient Descent?

Polynomial regression

- We first define new features based on the original features
- We then apply linear regression using the new features
- Let us suppose we have one input variable (x) and like to apply polynomial regression of order 3:

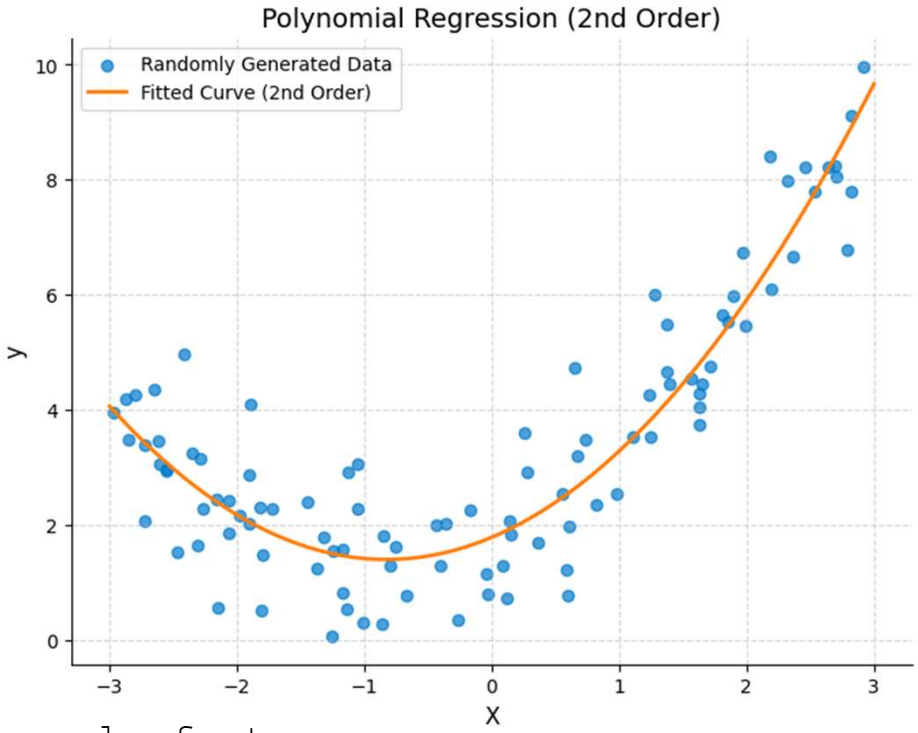
  $z_1 = x$

  $z_2 = x^2$

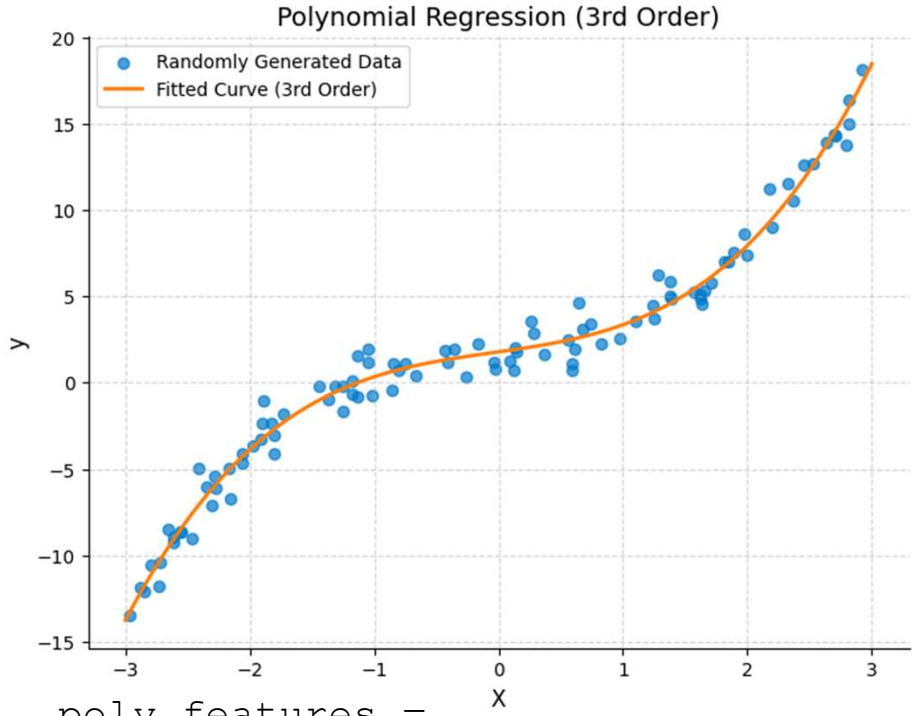
  $z_3 = x^3$

- New model: y= $\theta_0 + \theta_1 \times z_1 + \theta_2 \times z_2 + \theta_3 \times z_3$
- What if you have two variables (x_1, x_2) and like to apply the polynomial regression of order 2?

Examples of polynomial regression




```
poly_features =
PolynomialFeatures(degree=2,
include_bias=False)
X_poly = poly_features.fit_transform(X)
```




```
poly_features =
PolynomialFeatures(degree=3,
include_bias=False)
X_poly = poly_features.fit_transform(X)
```

Transform input features for polynomial regression

Linear regression based on other parameters

- Is it possible to use other variables in linear regression instead of polynomials?
- How about the following?

  z_1 =sin(x_1)

  z_2 =cos(x_2)

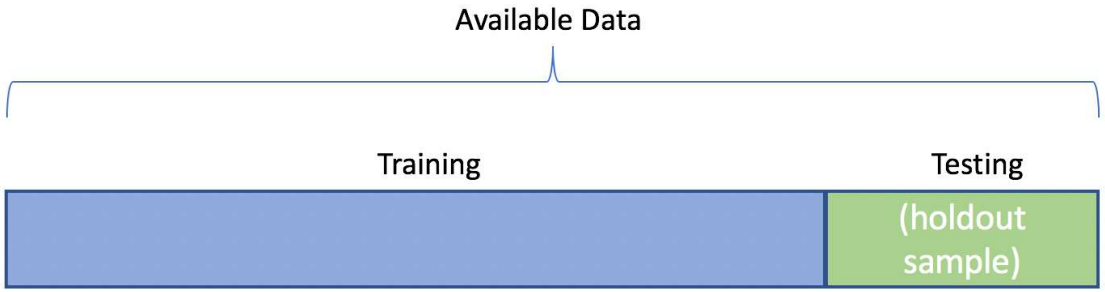
  z_3 =sin(x_1)cos(x_2)
- How does linear regression find the coefficients?

Data division for training and testing the model

- We usually divide the data (100%) into training (~80%) and test (~ 20%) sets
- We tune the parameters by minimizing the loss using the training data
- We then check the model performance using the test data
- We may divide the data into training, validation, and test data
- Validation is used to check the model performance during the training

Training-testing divide

	Predictor 1	Predictor 2	Predictor 3	Predictor 4	Predictor 5	Target
Training	Observation1					
	.					
	.					
	.					
Testing	.					
	.					
	Observation1000					



Original data

Gamma Ray	Resistivity	Poisson ratio	Density	Velocity	Rock Type
73.215	0.25	0.4126	137.81	7256.3675	Sandstone
69.152	0.2	0.4118	138.06	7243.2276	Shale
65.965	0.21	0.4109	138.06	7243.2276	Siltstone
68.215	0.27	0.4104	137.31	7282.7908	Siltstone
70.84	0.36	0.4096	136.31	7336.2189	Sandstone
62.262	0.52	0.4039	134.31	7445.462	Shale
61.637	0.7	0.3927	127.78	7825.9509	Shale

Example of training-test data

Predictors (X1,X2,….Xn)	Y
train_X	train_Y
test_X	test_Y

Overfitting versus underfitting

- Underfitting is a scenario where the model is not complex enough for your problem
- Overfitting happens when your model is too complex for your problem
- Main features
  - Underfitting: Cost function remains high for training and test data
  - Overfitting: The model performs well on the training data (low cost) but does not generalize when assessed on the test data (high cost)
  - There is a sweet spot that you should find by checking the cost functions (RMSE, MSE,..) for the training and test data

Ideal outcome

- Cost functions of training and test data are small (model is not underfitting)
- Cost functions of training and test data are close (model is not overfitting)
- The cost function of test data is slightly higher than the cost function of the training data because the model has seen the training data to tune its parameters

Bias versus variance trade-off

- What are the definitions of bias and variance?
- How do these terms relate to overfitting and underfitting?
- What are their governing equations?

Regularized linear models

- Regularization constrains the weights of the model to prevent overfitting
- Ridge regression:

$$J_{Ridge}(\theta) = \mathrm{MSE}(\theta) + \frac{\alpha}{m}\sum \theta_i^2$$

- Lasso regression:

$$J_{Lasso}(\theta) = \mathrm{MSE}(\theta) + 2\alpha \sum |\theta_i|$$

- Elastic Net regression:

$$J_{Elastic\ Net}(\theta) = \mathrm{MSE}(\theta) + r\left(2\alpha \sum |\theta_i|\right) + (1-r)(\frac{\alpha}{m}\sum \theta_i^2)$$

- Note: θ_0 is not regularized ($i > 0$ in all equations)

Ridge regression uses L2 norm squared

- The model minimizes not only the MSE but also the weights

$$J_{Ridge}(\theta) = \mathrm{MSE}(\theta) + \frac{\alpha}{m}\sum \theta_i^2$$

- α is a hyperparameter
- α=0 simplifies the model to linear regression
- Large α makes weights close to zero
- Increasing α flattens the output of the model (less sensitive to the input variables)

Least Absolute Shrinkage and Section Operator (Lasso) regression

- This model is similar to the Ridge regression but uses the L1 norm in the regularization

$$J_{Lasso}(\theta) = \mathrm{MSE}(\theta) + 2\alpha \sum |\theta_i|$$

- Lasso regression sets the less-important weights equal to zero (this is similar to feature selection)

Elastic Net Regression

- This regression is between the Ridge and Lasso regressions (check the solution where r=0 and r=1)

$$J_{Elastic\ Net}(\theta) = \mathrm{MSE}(\theta) + r\left(2\alpha\sum|\theta_i|\right) + (1-r)(\frac{\alpha}{m}\sum\theta_i^2)$$

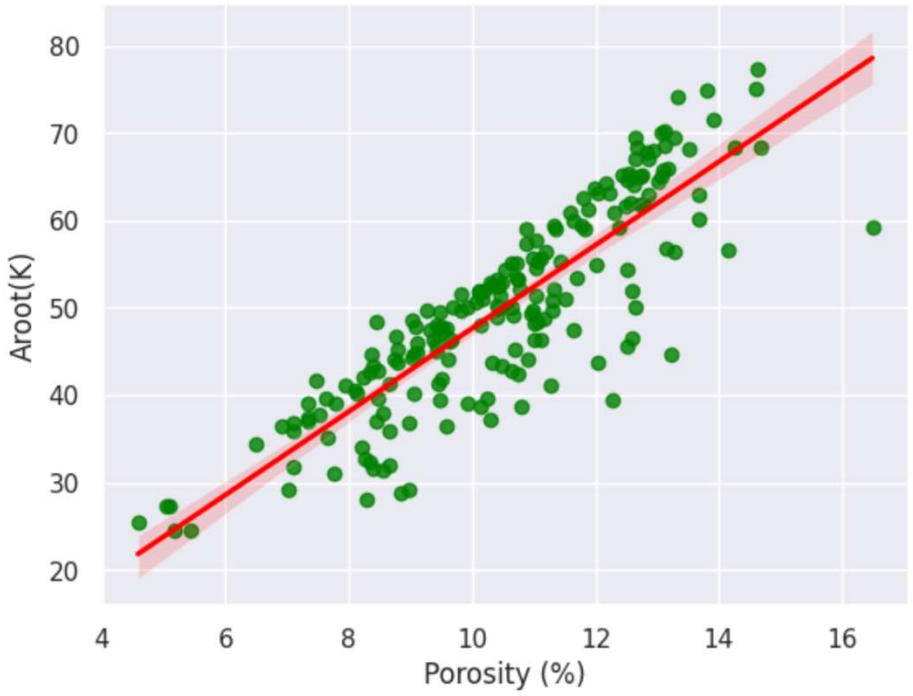
Which regression model works the best

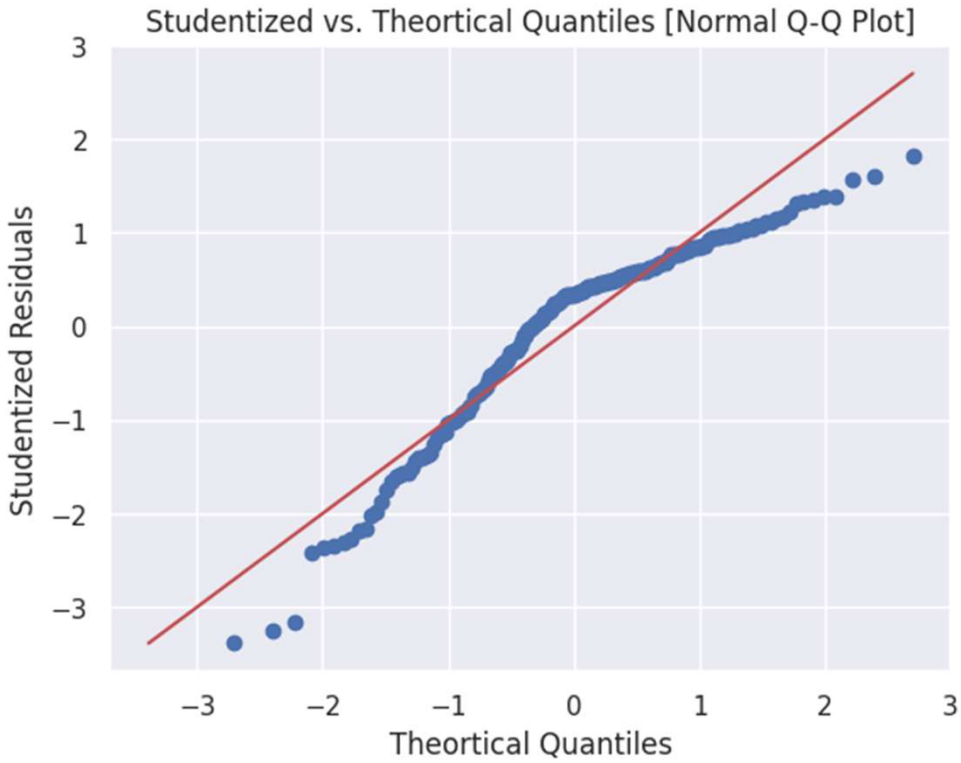
- Obviously, it depends!
- Ridge regression is usually better than the original regression because regularizations, in general, improve the performance
- Lasso and Elastic Net regressions are better than Ridge regression if you plan to check the effects of the number of tuning variables
- Lasso regression allows you to transition your model from Ridge to Lasso regression
- You should be able to compare them quickly using a few lines of codes

Code 1

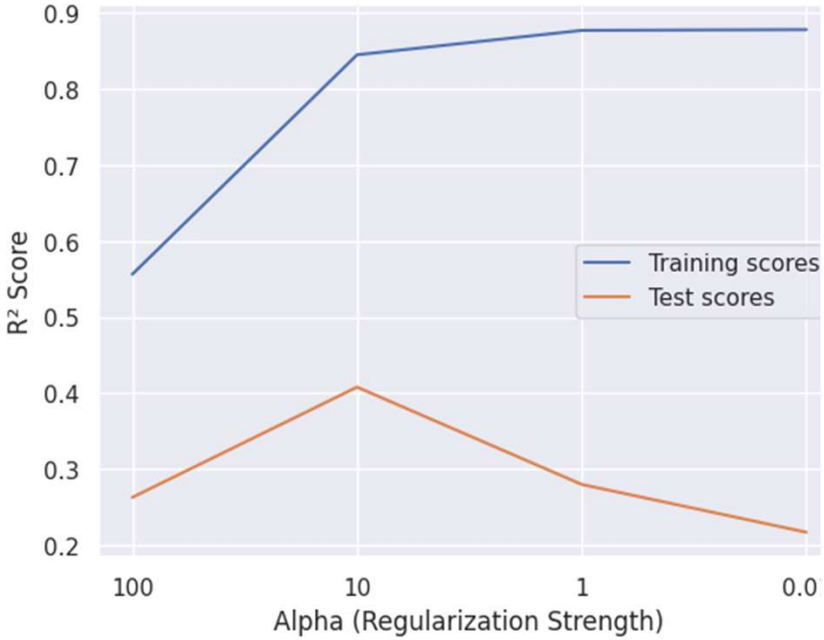
- Code_Linear_regression (actual code)
- data_3 (stored data)

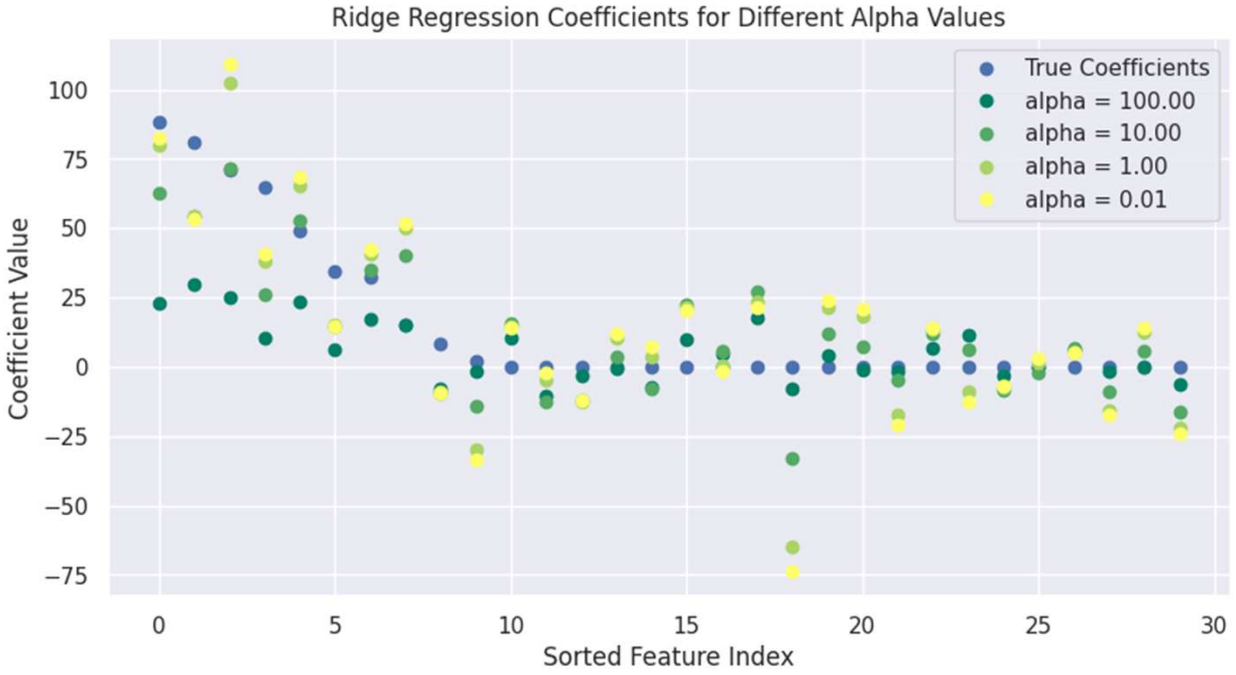
Code 1: Linear regression





Code 1: Ridge regression

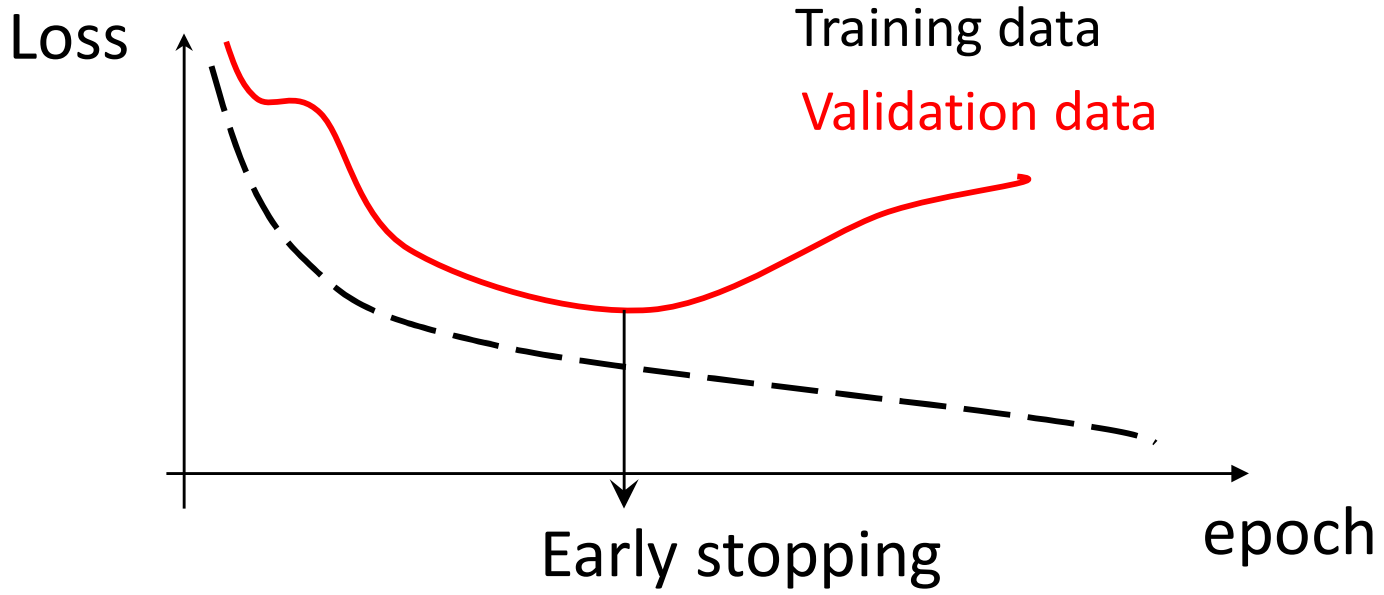




Code 1: Lasso Regression

Early stopping

- Early stopping helps us avoid overfitting
- The loss values of validation and training decrease initially as the model learns the underlying features
- We stop the training when the validation loss reaches a minimum (after this point, the model overfits the training but does not generalize)



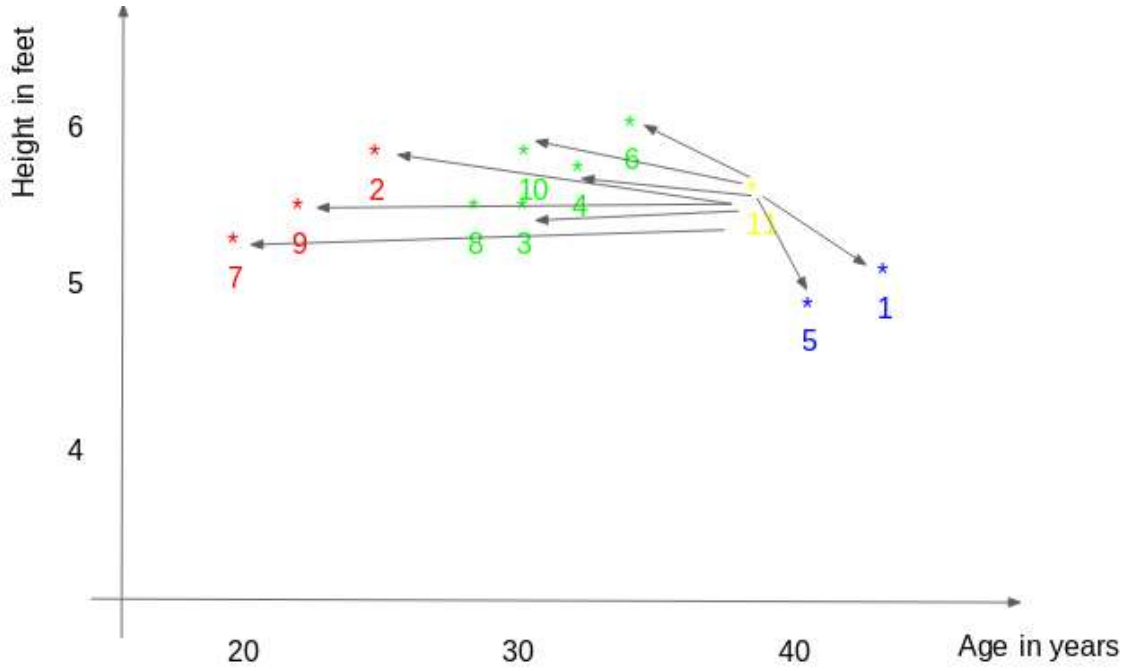
Another regression model

- *K*-nearest Neighbors Regressor

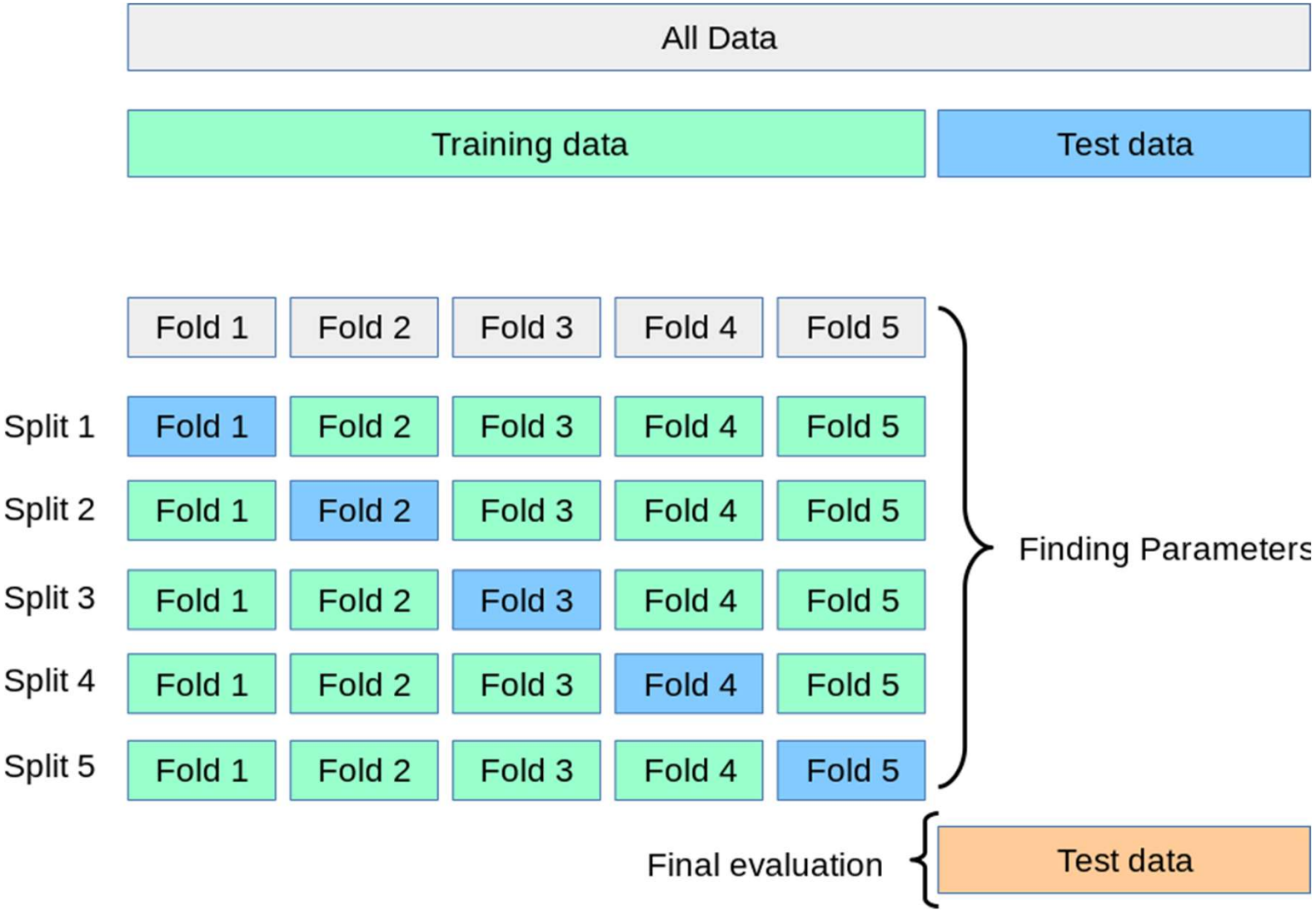
K-nearest Neighbors Regressor

- How does it work?

ID	Height	Age	Weight
1	5	45	77
2	5.11	26	47
3	5.6	30	55
4	5.9	34	59
5	4.8	40	72
6	5.8	36	60
7	5.3	19	40
8	5.8	28	60
9	5.5	23	45
10	5.6	32	58
11	5.5	38	?



K-fold Cross-validation



Some comments on KNN

- It is not popular, but it is a good baseline
- It does not require defining new features because it can handle oscillation and nonlinearity
- KNN is more powerful than Linear Regression for interpolation, but **it usually fails to extrapolate**

Code 2

Code- run the k-nearest regressor code

- A. Upload the Notebook (KNN_Regressor)
- B. Upload its data (KNN_reg_data)
- C. Run the code once
- Answer the following questions:
- 1. What is the effect of `random_state=55`?
- 2. What is the division percentage (training versus testing)? What is a more common division in data analytics?
- 3. Change the number of neighbors and discuss its effect on the model performance
- 4. What is the best number of neighbors?

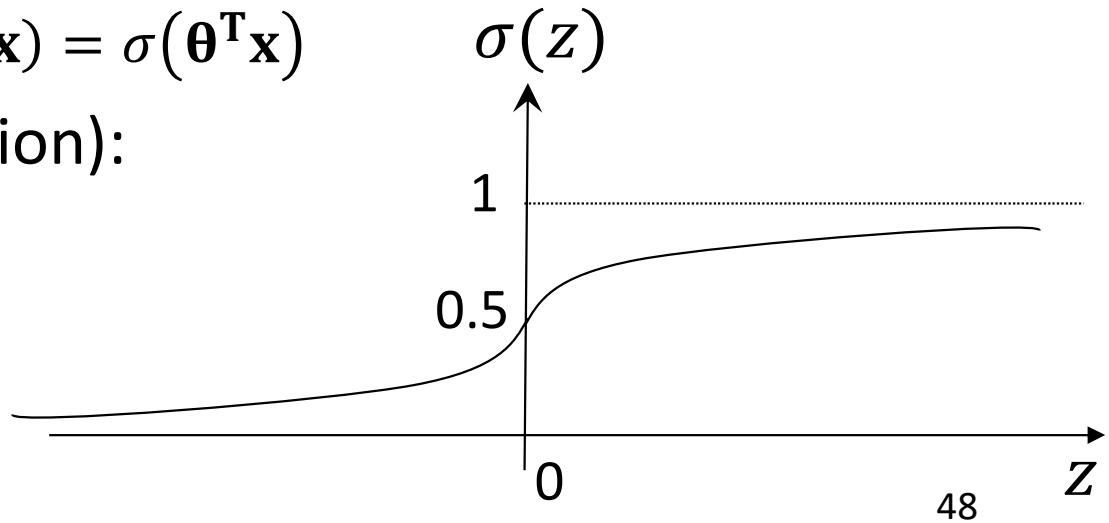
Logistic regression

- This regression determines the probability of a sample being part of a specific group:
  - What is the probability of a hydrocarbon reservoir being economically producible?
  - What is the probability of a patient having covid?
- It is a regression because it provides the probability and not just the class:

$$\hat{p} = h_\theta(\mathbf{x}) = \sigma(\boldsymbol{\theta}^{\mathbf{T}}\mathbf{x})$$

- Logistic function (definition):

$$\sigma(\mathrm{z}) = \frac{1}{1 + e^{-z}}$$



Derivative of the logistic function (sigmoid function)

Show: d/dz$(\sigma(\mathrm{z})) = \sigma(\mathrm{z})(1 - \sigma(\mathrm{z}))$

Cost function of Logistic regression and its partial derivative

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}[y^i \log(\hat{p}^i) + (1 - y^i)\log(1 - (\hat{p}^i))]$$

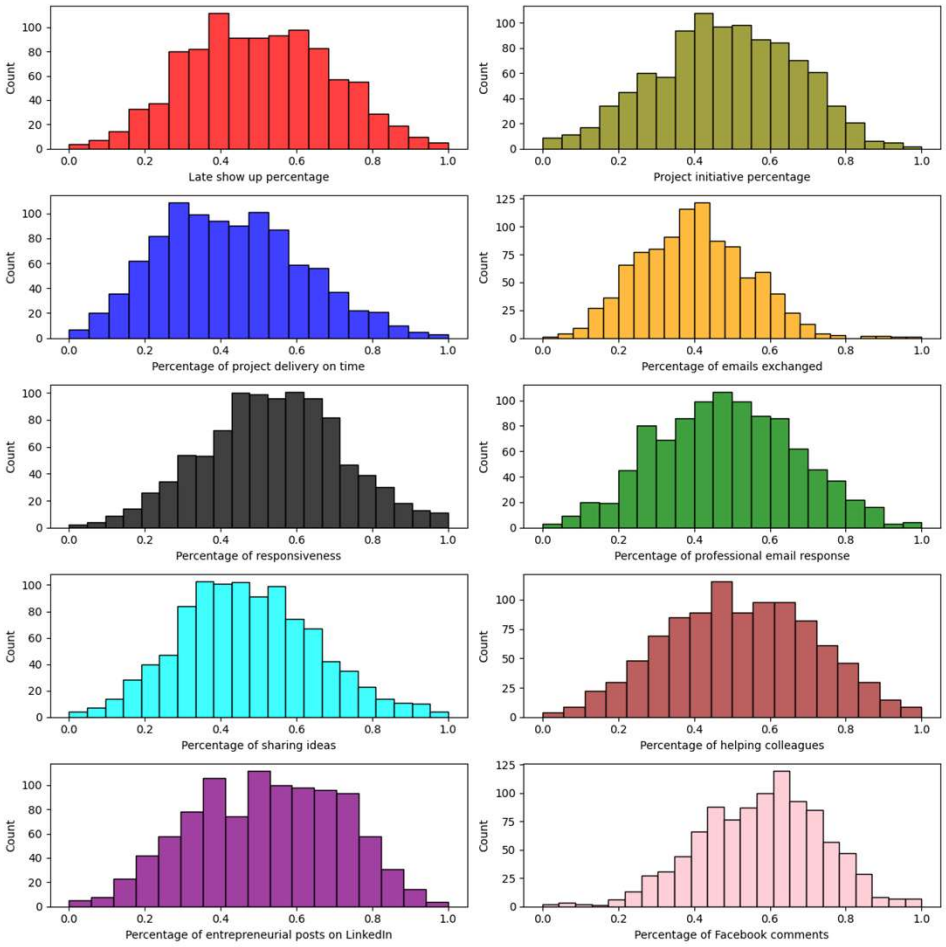
$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{1}{m}\sum_{i=1}^{m}(\sigma(\boldsymbol{\theta}^{\mathbf{T}} x^i) - y^i)\, x_j{}^i$$

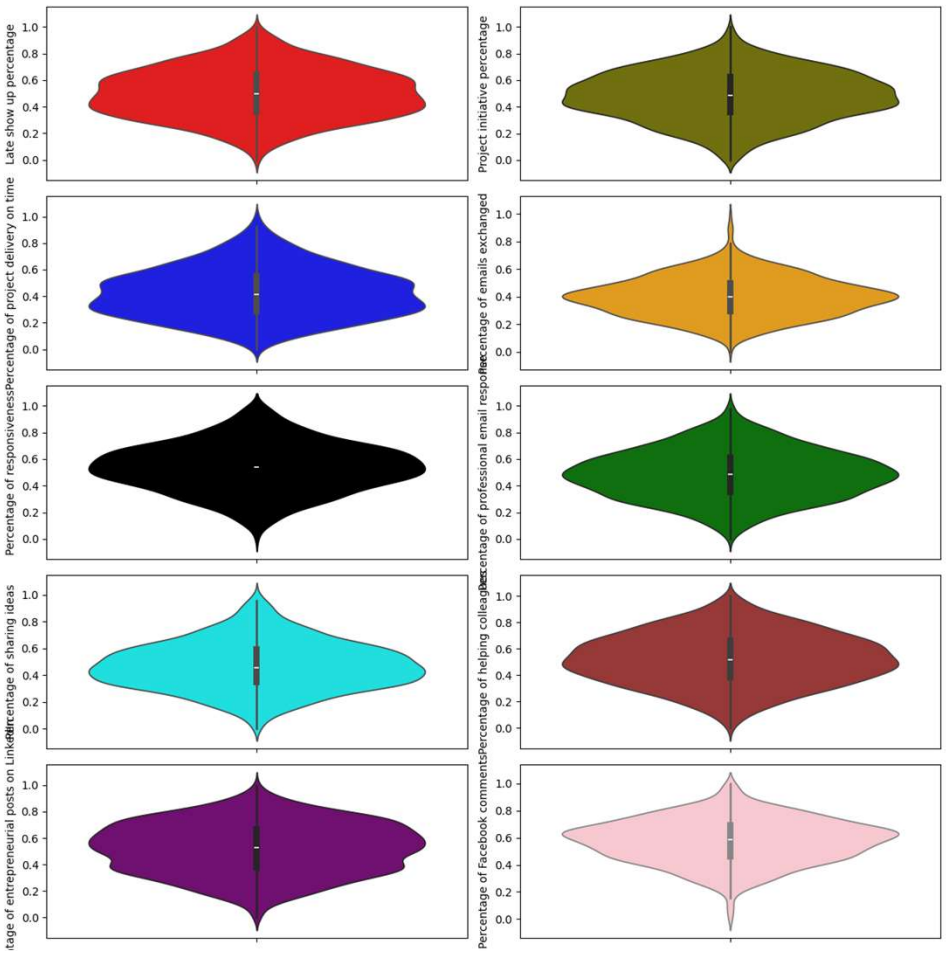
- We will discuss these in more detail later in the course

Code 3- upload its notebook (logistic regression) and dataset (HR_DataSet)

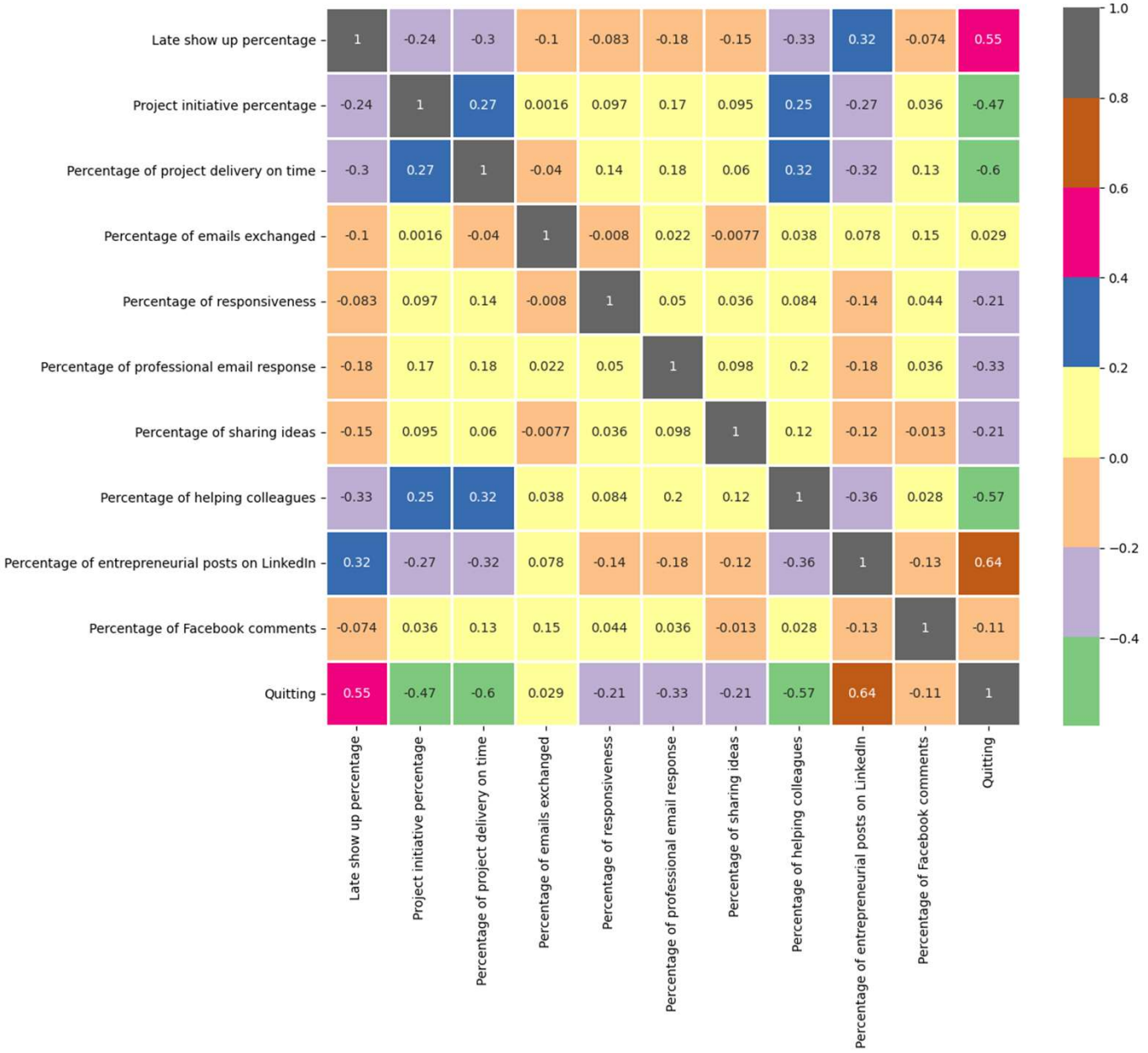
- **Data set:**
  - HR data set:
    - Late show-up percentage
    - Project initiative percentage
    - Percentage of project delivery on time
    - …
  - Target Feature
    - Quitting
- **Analytics Question:**
  - Using the HR data set, Can we create a classification logistic regression model to predict whether an employee is quitting or not?

Results from the second code

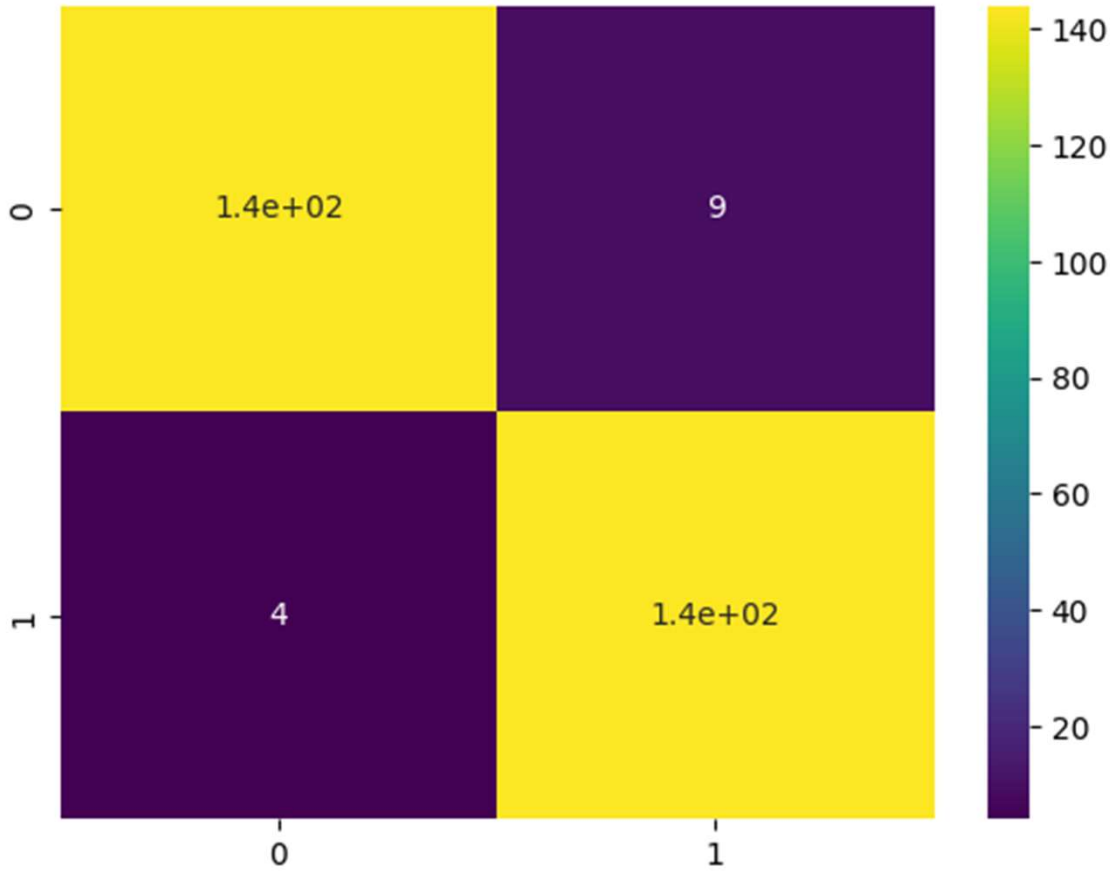




Logistic regression data – no obvious collinear relation



Confusion matrix



Data Analytics (PETR 6397)

Classification

Dr. Ahmad Sakhaee-Pour

Petroleum Engineering Department

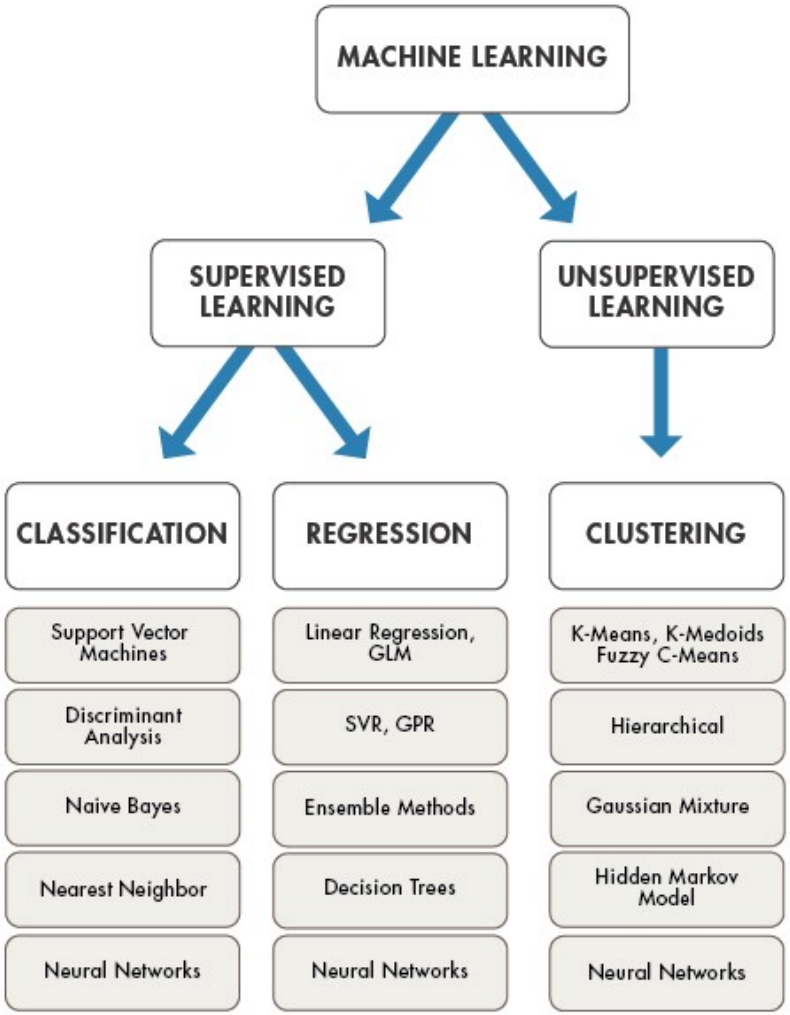
University of Houston

Spring 2025

Discussed topics

- Classification
- Commonly used datasets
- Stochastic Gradient Descent (SGD) classifier
- Classification Metrics
    - Precision and Trade
    - Confusion Matrix (CM)
    - Receiver Operating Characteristics (ROC)
- K-nearest neighbor (KNN)
- Multiclass classification
- Parametric methods (Logistic Regression)
- Random Forest

Reminder of various problems



Classification

- In classification, we predict a class label assigning points to groups
- In classification, the training inputs are similar to regression, but the outputs are discrete values, not continuous
- In classification, we know the correct outputs (which are unknown in clustering)

Classification types

Binary Classification

- Spam
- Not spam

Multiclass Classification

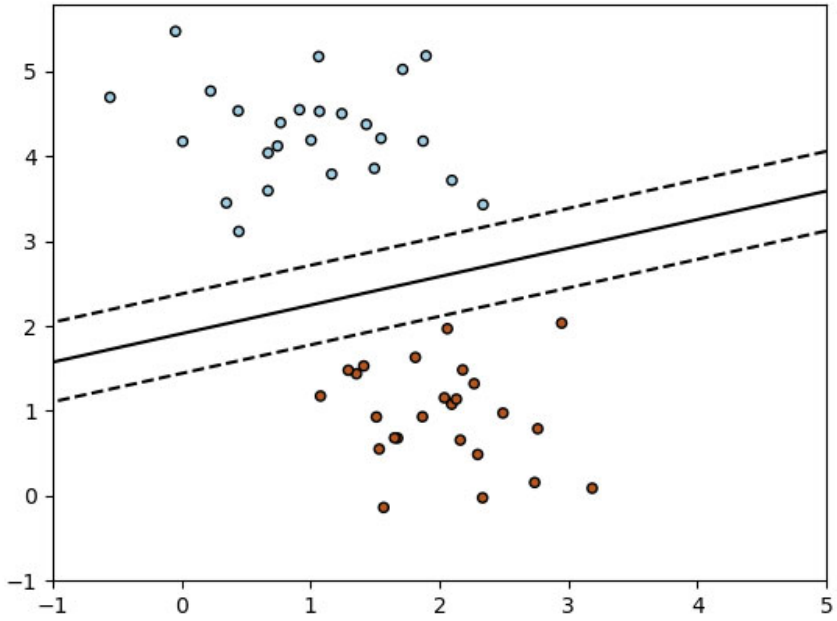
- Dog
- Cat
- Horse
- Fish
- Bird
- ...

Multi-label Classification

- Dog
- Cat
- Horse
- Fish
- Bird
- ...

Stochastic Gradient Descent (SGD) classifier

- It implements a stochastic gradient descent routine that supports different loss functions and penalties for classification

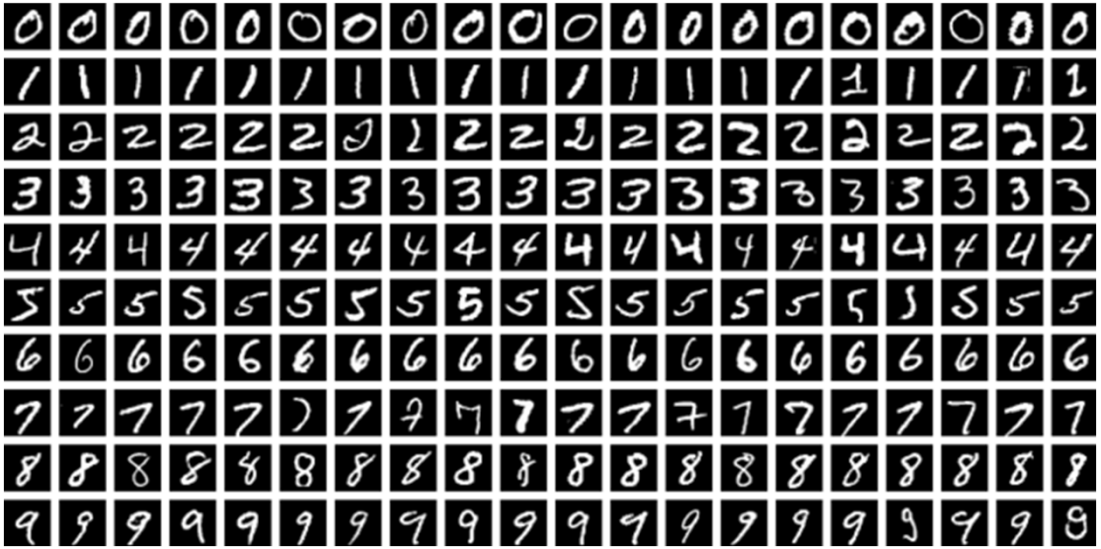


Commonly used datasets

- MNIST: Modified National Institute of Standards and Technology
- Fashion MNIST
- Canadian Institute for Advanced Research: CIFAR-10, CIFAR-100
- CelebFaces Attributes Dataset: CelebA
- …

Modified National Institute of Standards and Technology (MNIST)

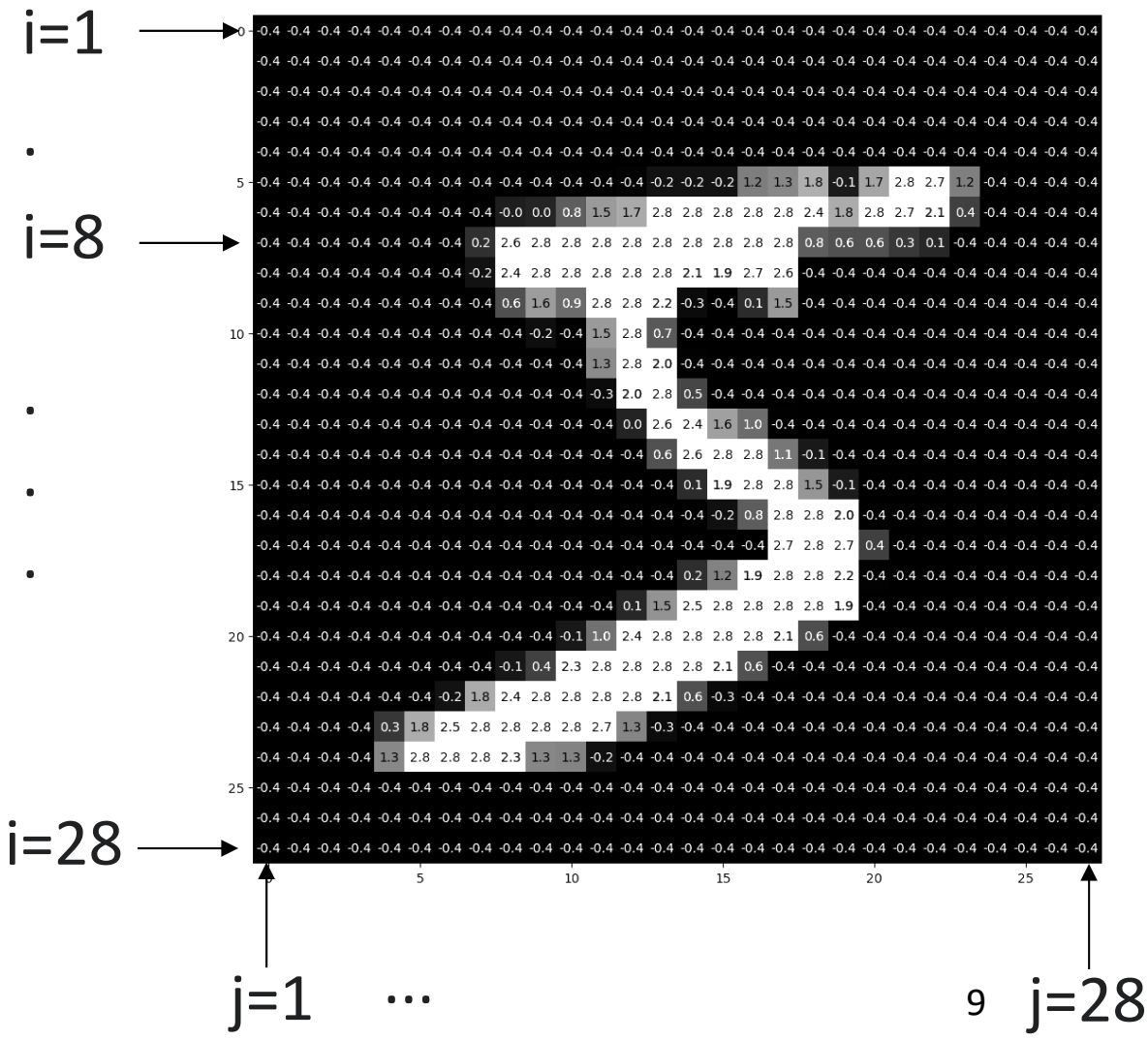
- A large database of handwritten digits from 0 to 9
- It is used for training various models
- The MNIST database has 60,000 training images and 10,000 testing images



Each image has 784 pixels

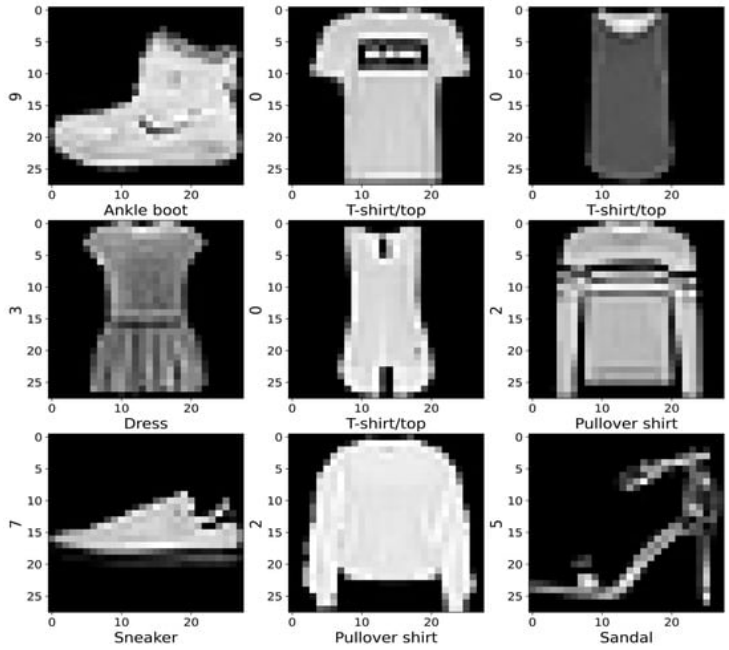
- Each image 28x28 pixels
- Each pixel has a single value that is smaller for the background (black region)
- Majority of pixels are background (black region)

Pixel values are scaled below



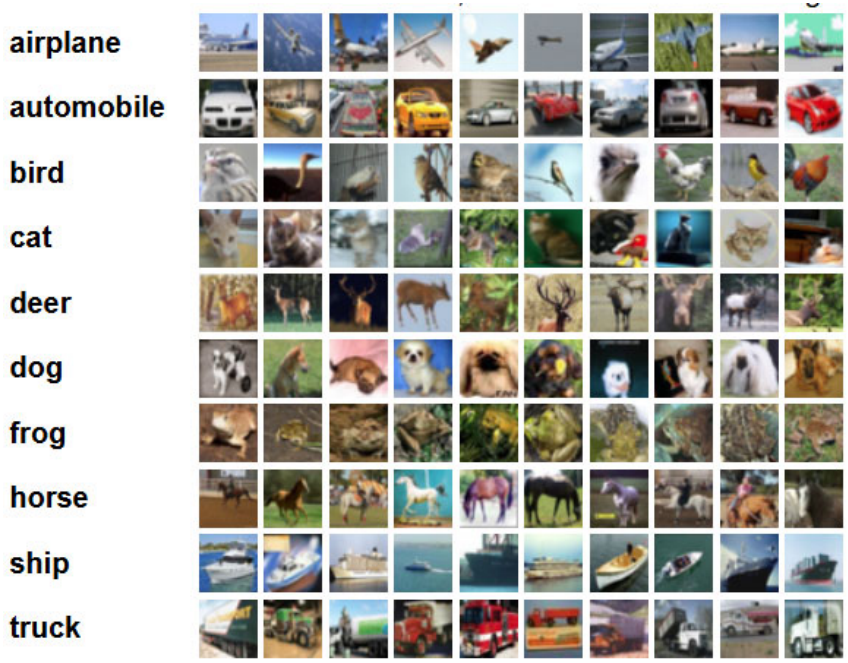
Fashion-MNIST

- It has 60,000 images (50,000 for training and 10,000 for testing)
- Each image has 28x28 pixels
- It has 10 classes



CIFAR-10

- 60000 color images in 10 classes
- Each image has 32x32 pixels
- The images are divided into 50000 training and 10000 test images



- Q: What does CIFAR-100 have?

Building a classifier

- Step 1: Split the data into training and testing
- Step 2: Select the function (classifier model)
- Step 3: Train the classifier on the training data
- Step 4a: Evaluate the classifier on the training data (memorization performance)
- Step 4b: Evaluate the trained classifier on the testing data (generalization performance)

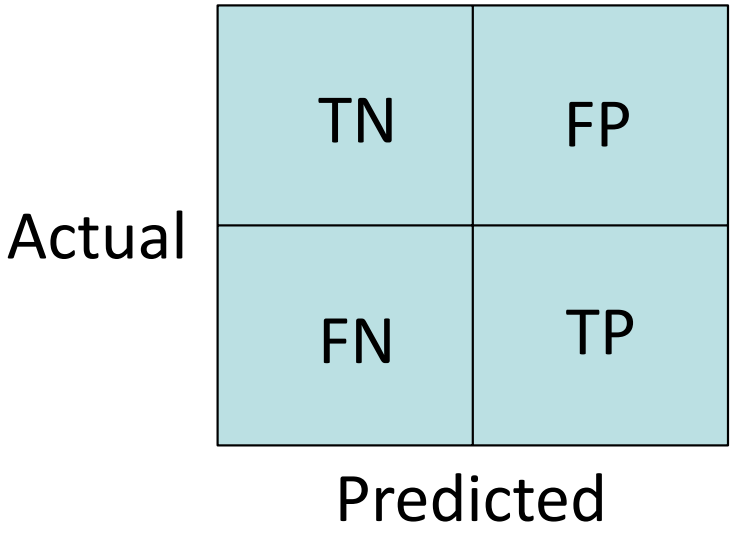
Accuracy

- Definition:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Number of all predictions}}$$

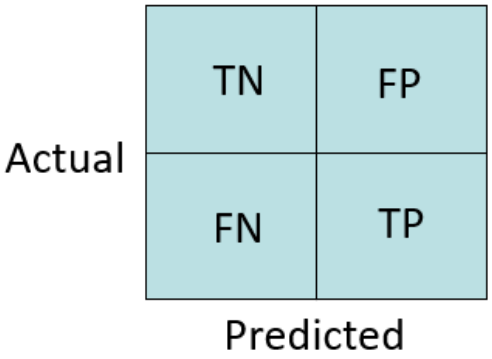
- Accuracy is the simplest and most basic measure

- It is usually not sufficient to assess the performance (see the example on the next slide)



Clarification of the accuracy

- Suppose we have a bunch of images from MNIST
- Our goal is to identify the digit 5
- Each row represents an actual class
- Each column is a predicted class



Non-5 (Class 0)

Digit 5 (Class 1)

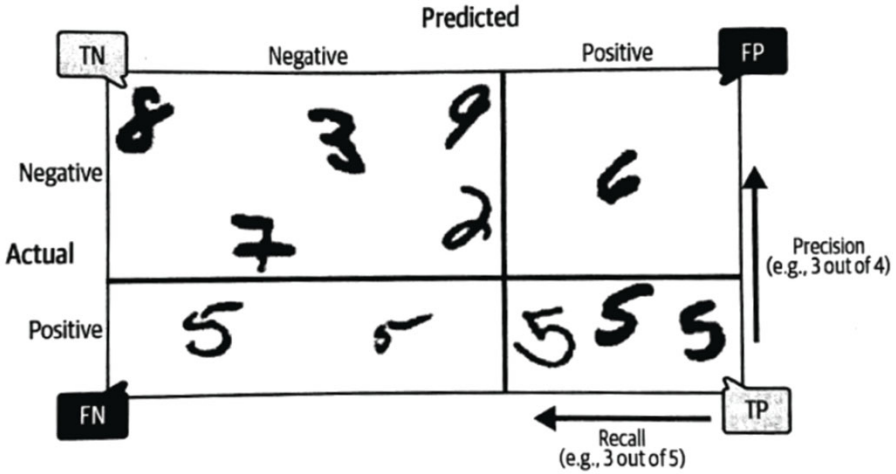


Fig. 3.3 (main reference)

Compare the accuracy measures of two models

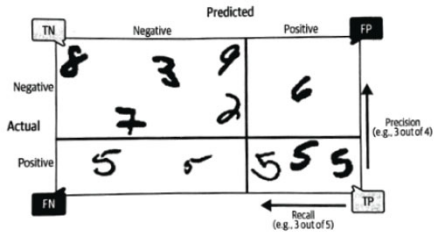
- Suppose you receive 1000 text messages, from which 10 are actual spam
- First model identifies all messages as non-spam
- Second model identifies the actual 10 spam (correct identification), but it also incorrectly identifies another 10 non-spam messages as spam
- Accuracy is usually not the preferred performance measure

Precision and Recall

$$\text{Precision} = \frac{\text{True positive}}{\text{Predicted results}} = \frac{\text{True positive}}{\text{True positive} + \text{ false positive}}$$

$$\text{Recall} = \frac{\text{True positive}}{\text{Actualresults}} = \frac{\text{True positive}}{\text{True positive} + \text{ false negative}}$$

- Other names for Recall are True Positive Rate or Sensitivity
- Two definitions for Confusion Matrix: both are acceptable if you are consistent



Actual		
	TP	FN
	FP	TN

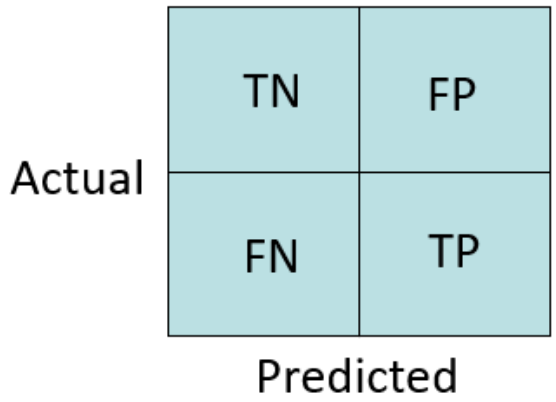
Predicted

Actual		
	TN	FP
	FN	TP

Predicted

Hypothetical perfect classifier

- It has only true positive and true negative values
- Its confusion matrix is diagonal: non-diagonal terms (FP and FN) are equal to zero



Effects of threshold on the Precision and Recall

- Suppose our goal is to find the digit five
- There are 12 numbers, from which six are 5
- Precision: How correct are the identified numbers (among themselves)?
- Recall: What fraction of the correct answers is collected (from the six samples)?

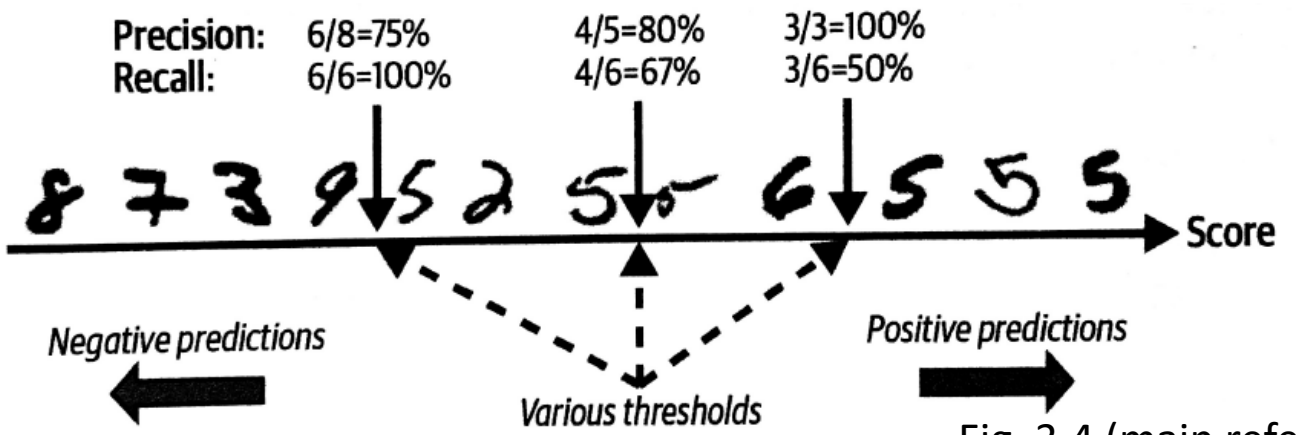


Fig. 3.4 (main reference)

F1 Score (unified measure)

- F1 accounts for the recall and precision

$$\mathrm{F}_1 = \frac{2}{1/\mathrm{precision} + 1/\mathrm{recall}} = 2\,\frac{\mathrm{precision} \times \mathrm{recall}}{\mathrm{precision} + \mathrm{recall}}$$

- F1 favors classifiers whose recall and precision have similar performance
- In practice, either precision or recall is more important

F_β Score

- This measure gives more weight to Precision or Recall (depending on β)

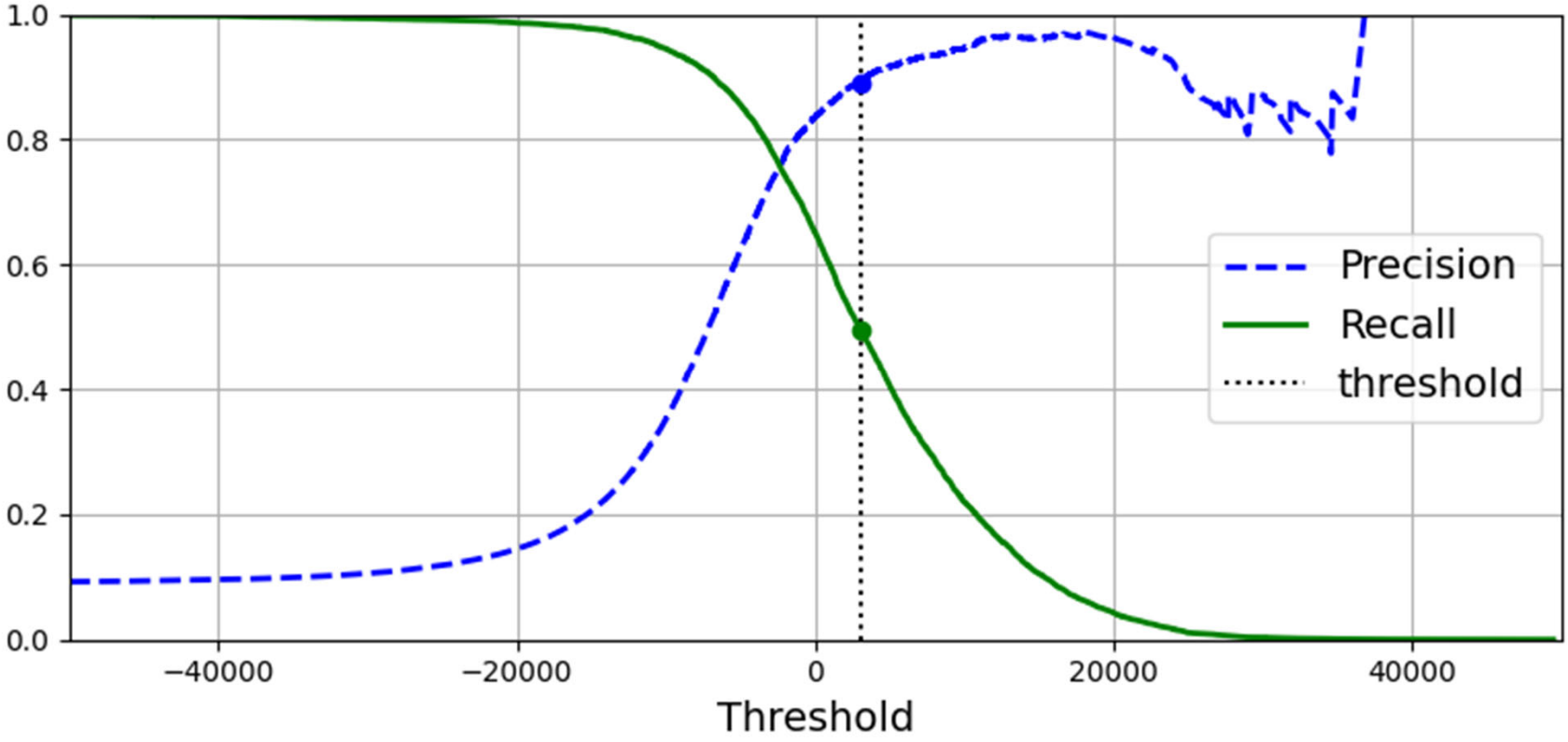
$$F_\beta = (1 + \beta^2) \frac{\text{precision} \times \text{recall}}{(\beta^2 \times \text{precision}) + \text{recall}}$$

- The parameter (precision or recall) with a lower weight increases more (becomes more important)

Precision and Recall trade-off

- Increasing precision reduces recall
- Increasing recall also reduces precision
- Run the classification code (classification_metrics) posted on CANVAS to determine where the importance of each measure
- This code is a modification of the code available from the main reference

Precision and Recall as a function of the threshold of a classifier

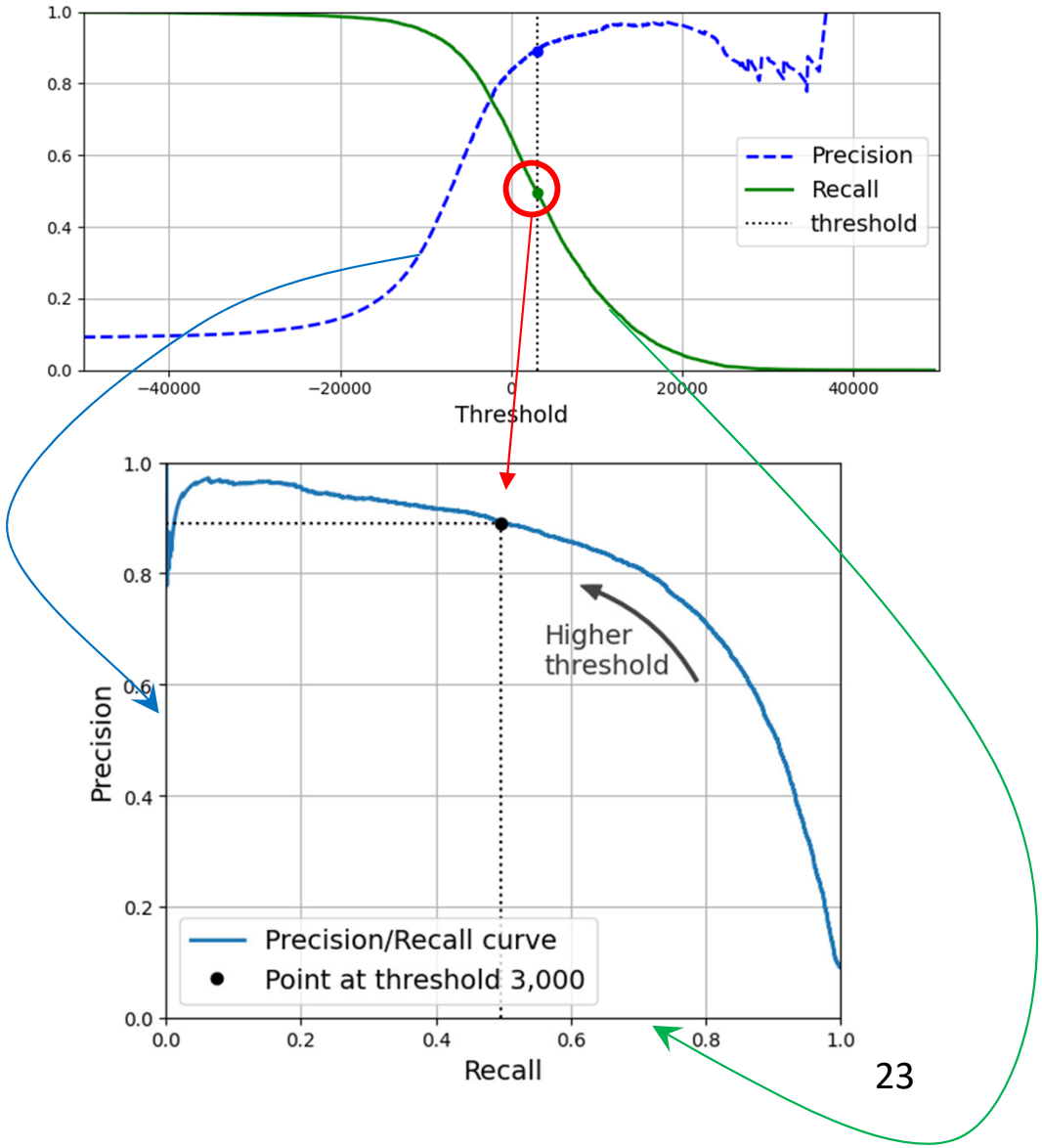


- Q1: Why is the Precision non-monotonic?
- Q2: Which threshold value would you choose? Why?

Precision vs Recall

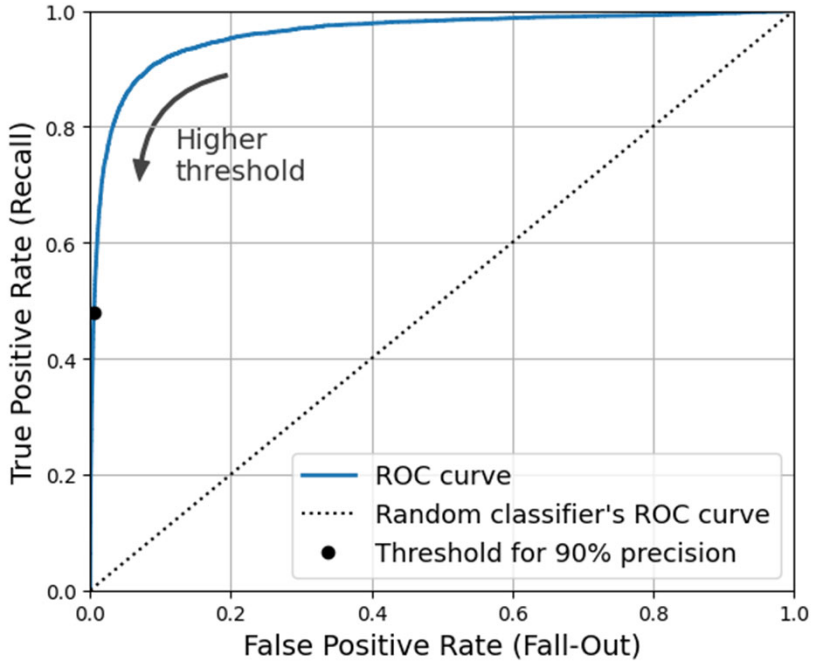
- This is just another presentation of the previous plot

- We can also plot the precision versus recall to assess the model further



Receiver operating characteristic (ROC)

- ROC provides another tool to analyze the model performance
- It shows true positive rate versus false positive rate



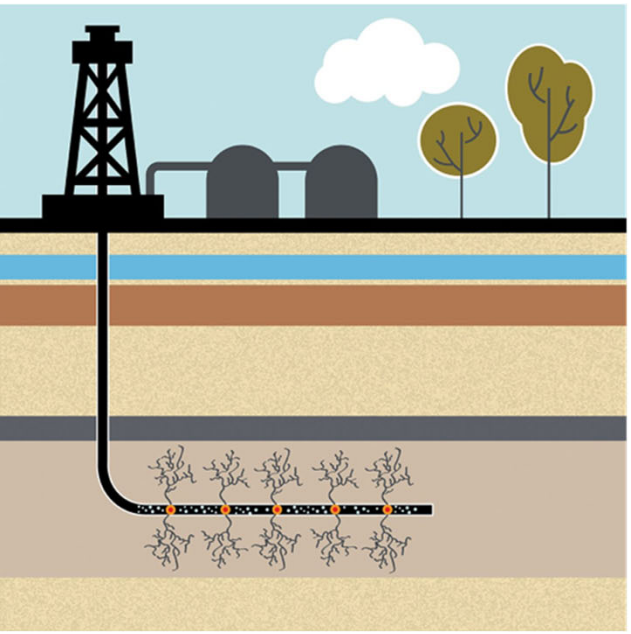
- Q: What was the other name for the true positive rate we defined earlier

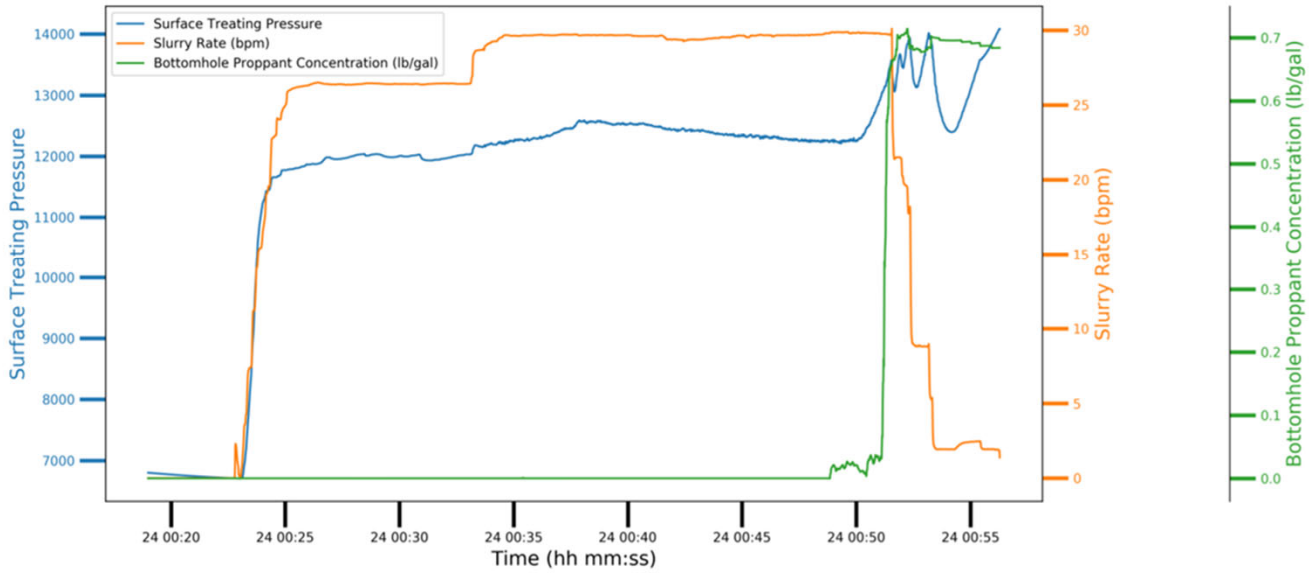
Which plot (PR vs. ROC) would better assess your classifier?

- In general, it would be good to create both to gain a better understanding of the model
- Use the Precision/Recall (PR) curve if the positive class is rare
- Use the PR curve if you care more about the false positive than the false negative
- Otherwise, use ROC

Screen Out Example

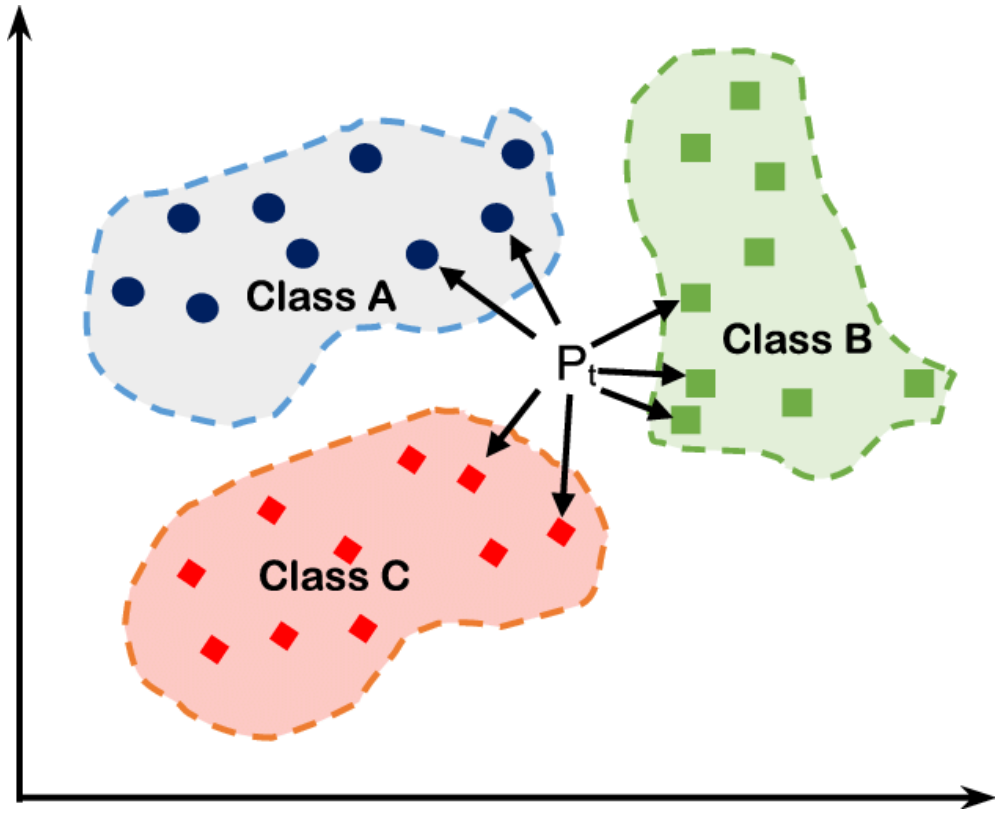
- Classify a screen-out stage ahead of time





high cost associated with false negative, and the goal is to maximize recall

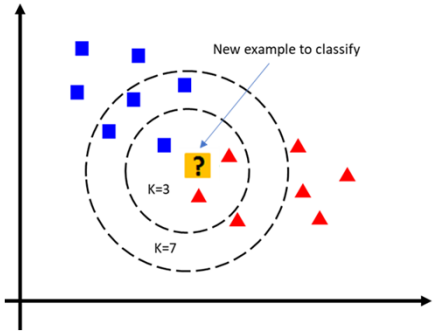
K Nearest Neighbors Classifier (KNN)



Main features of KNN

- It accounts for the closest neighbors' class
- It is non-probabilistic (it does not report the probability of a class-like Regression)
- It does not learn any parameter
- There is no linear combination of parameters
- It is non-linear

Voting system of KNN



- It memorizes (or stores) the training samples and their class
- Because it memorizes, KNN may struggle with unforeseen data
- It uses a majority vote. That is why we usually set K to an odd number
- It can form a complex non-linear boundary

Multiclass classification

- There are two general approaches

- First: Divide your *n* classes into *n* binary classes and apply methods used for binary classification if the classifier does not have a multiclass version (SGDClassifeir, SVC,…)

- Second: Use the multiclass versions of the commands if they are available (LigisticRegression, RandomForest,…)

Metrics for multiclass classification

- Macro averaging: simply average the F1 scores of different classes

$$\text{macro} = \frac{\text{F}_{\text{1class1}} + \text{F}_{\text{1class2}} + \text{F}_{\text{1class3}} + \cdots + \text{F}_{\text{1classN}}}{\text{N}}$$

- Weighting: you may want to weigh the scores based on the number of samples

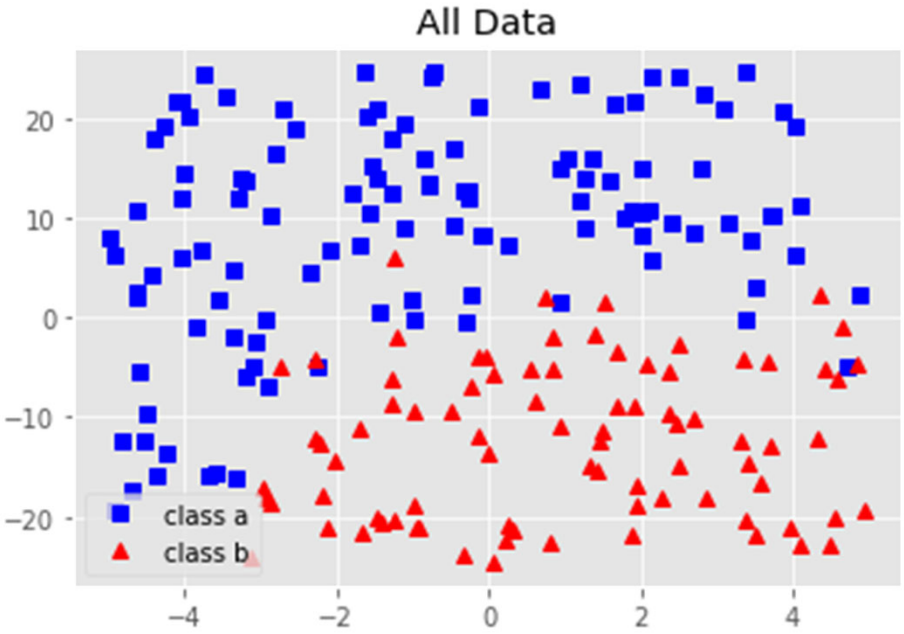
$$\text{weighted} = \text{W}_1 * \text{F}_{\text{1class1}} + \text{W}_2 * \text{F}_{\text{1class2}} + \cdots \text{W}_\text{N} * \text{F}_{\text{1classN}}$$

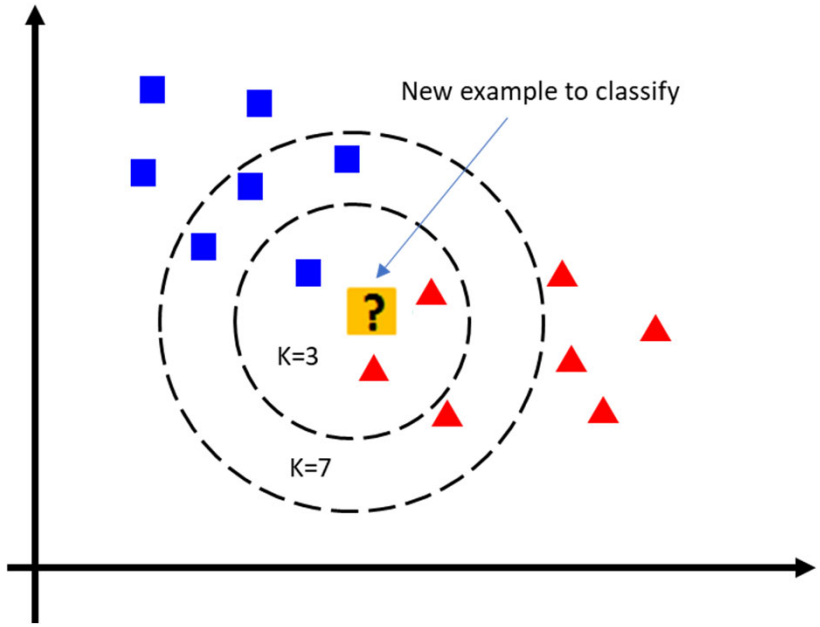
constrain: $\text{W}_1 + \text{W}_2 + \text{W}_3 + \cdots + \text{W}_\text{N} = 1$

KNN code

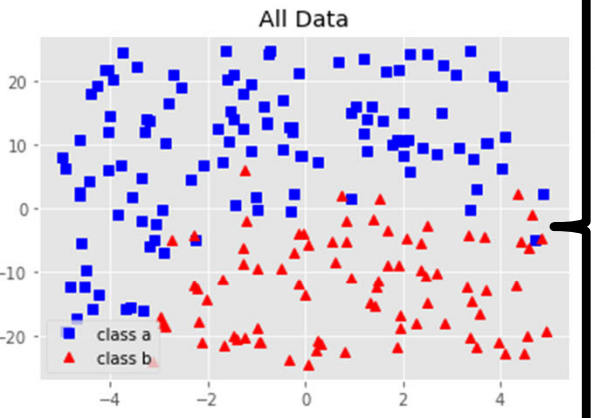
- Run KNN code posted on CANVAS (Classification_KNN_LogisticRegression)

KNN code (continued)



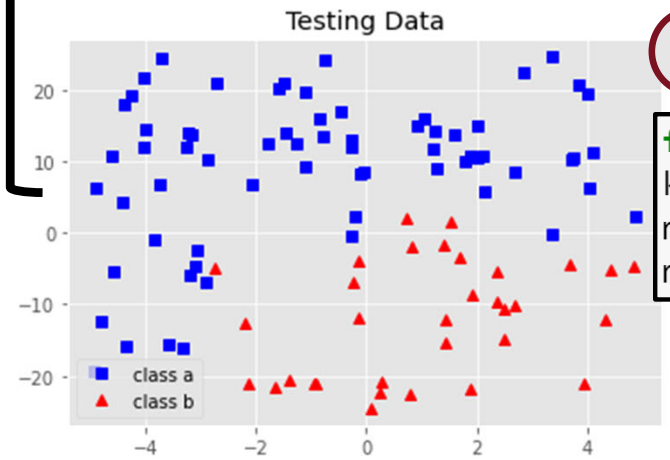


KNN code (continued)



1

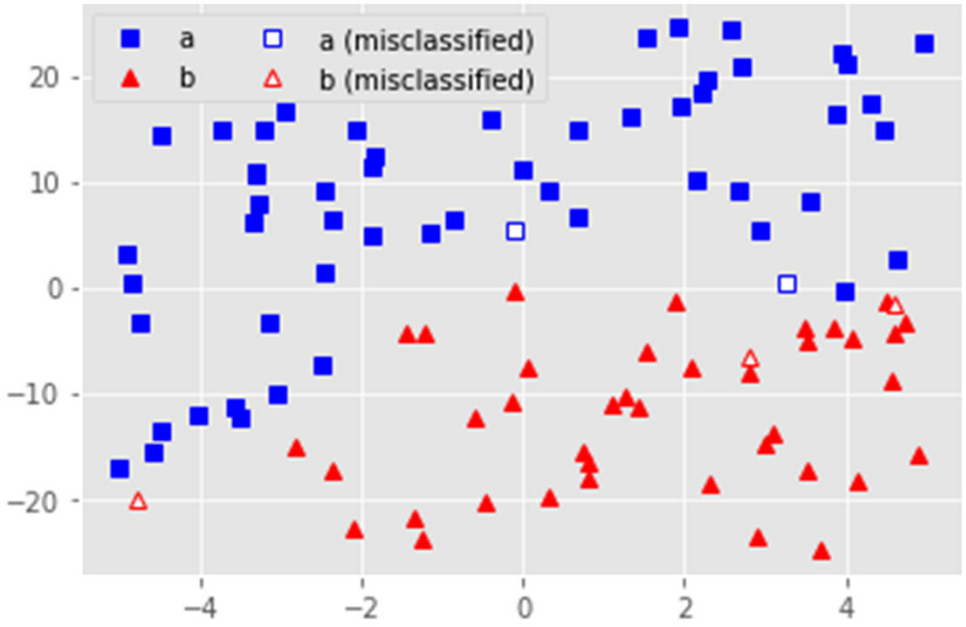
```
train, test = train_test_split(data, test_size = 0.5)
```



2

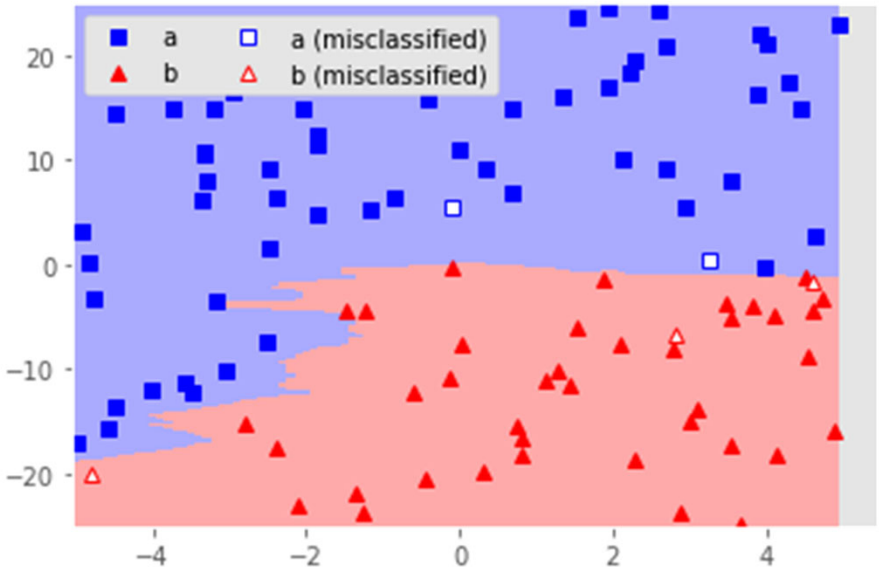
```
from sklearn import neighbors
k = 5
model = neighbors.KNeighborsClassifier(k)
model.fit(train[['x','y']], train['class'])
```

KNN code (continued)




```
def plot_predicted_1(model, test):
    predicted = model.predict(test[['x','y']])
    correct = test[test['class'] == predicted]
    correcta = correct[correct['class'] == 'a']
    correctb = correct[correct['class'] == 'b']
    incorrect = test[test['class'] != predicted]
    incorrecta = incorrect[incorrect['class'] == 'b']
    incorrectb = incorrect[incorrect['class'] == 'a']

    plt.plot(correcta['x'], correcta['y'], 'bs', label='a')
    plt.plot(correctb['x'], correctb['y'], 'r^', label='b')
    plt.plot(incorrecta['x'], incorrecta['y'], 'bs',
             markerfacecolor='w', label='a (misclassified)')
    plt.plot(incorrectb['x'], incorrectb['y'], 'r^',
             markerfacecolor='w', label='b (misclassified)')
    plt.legend(loc='upper left', ncol=2, framealpha=1)

plot_predicted_1(model, test)
```

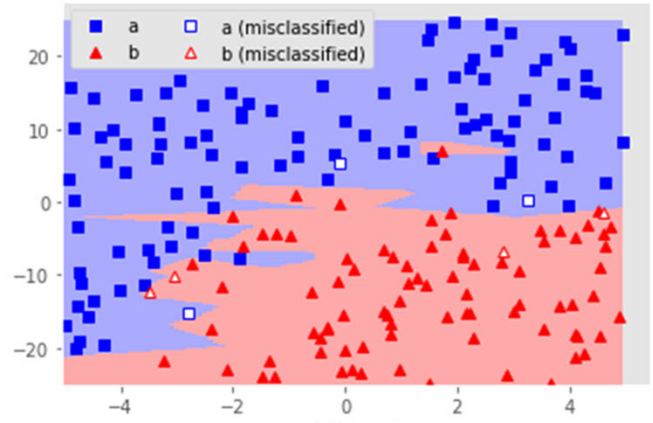
KNN code (continued)



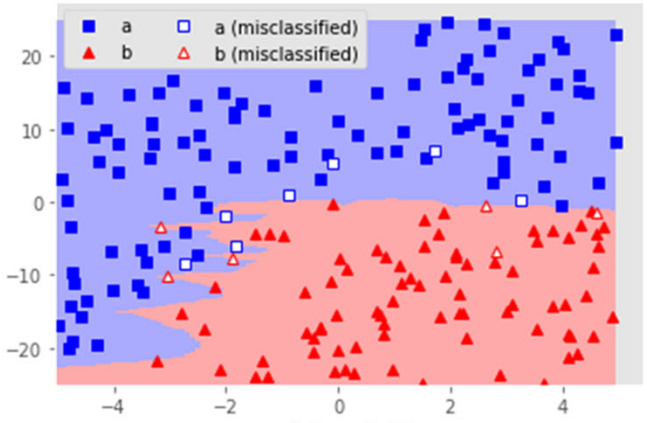

```
def plot_predicted_2(model, test):
    cmap = ListedColormap(['#AAAAFF','#FFAAAA'])
    xmin, xmax, ymin, ymax = -5, 5, -25, 25
    grid_size = 0.1
    xx, yy = np.meshgrid(np.arange(xmin, xmax, grid_size),
                         np.arange(ymin, ymax, grid_size))
    pp = model.predict(np.c_[xx.ravel(), yy.ravel()])
    zz = np.array([{'a':0,'b':1}[ab] for ab in pp])
    zz = zz.reshape(xx.shape)
    plt.figure()
    plt.pcolormesh(xx, yy, zz, cmap = cmap)
    plot_predicted_1(model, test)

plot_predicted_2(model, test)
plt.show()
```

Model Complexity with k-Nearest Neighbor

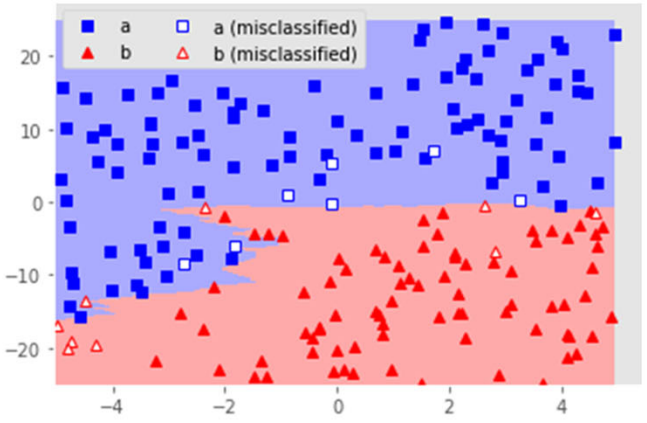
K=1



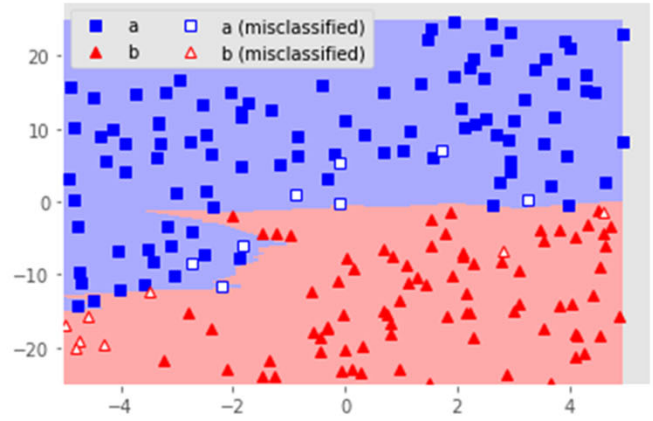
K=3



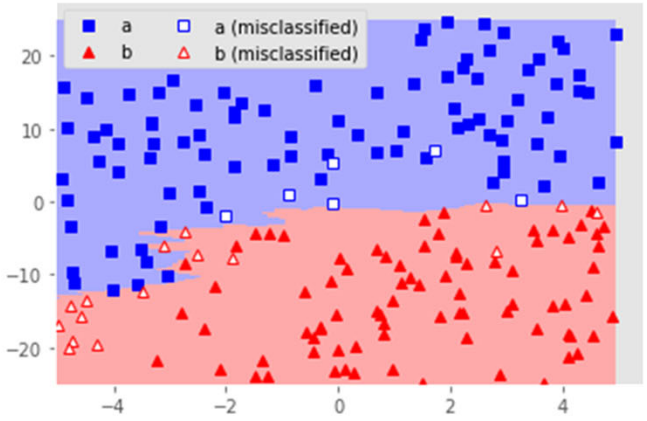
K=7



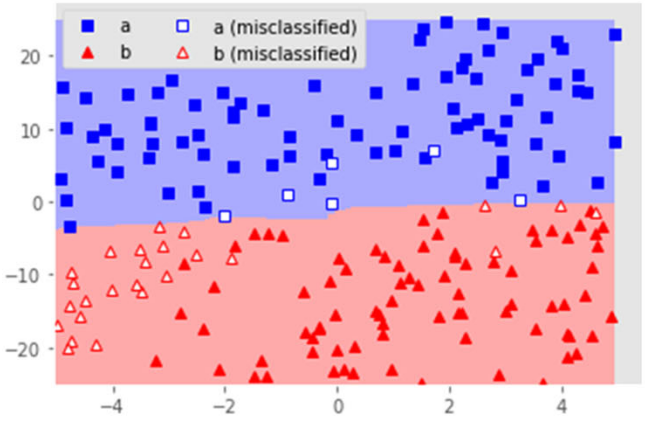
K=9



K=15



K=21



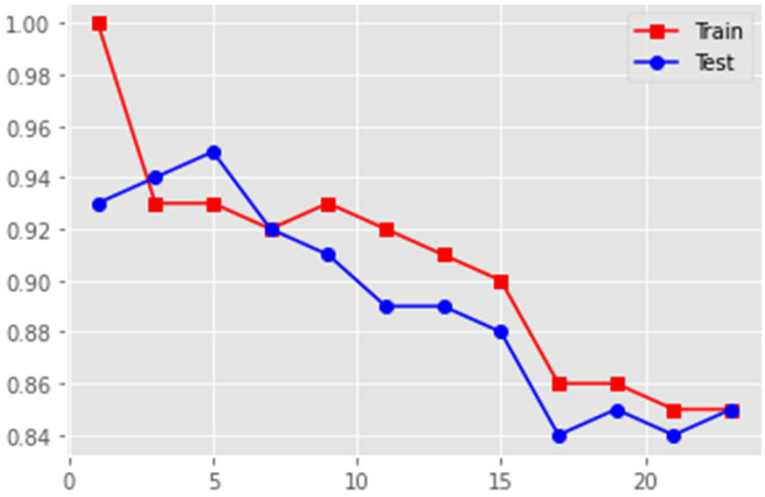
Overfitting vs underfitting in classification

- Similar to the regression, classification is sensitive to the complexity of your model

- In KNN, you overfit by using too few neighbors and underfit by using too many neighbors

Model Score

- The scikit-learn nearest neighbor model can generate a score based on the accuracy of our prediction.




```
kvals = range(1,25,2)
train_score = []
test_score = []

for k in kvals:
    model = neighbors.KNeighborsClassifier(k)
    model.fit(train[['x','y']], train['class'])
    train_score.append(model.score(train[['x','y']], train['class']))
    test_score.append(model.score(test[['x','y']], test['class']))

plt.plot(kvals, train_score, 'r-s', label='Train')
plt.plot(kvals, test_score, 'b-o', label='Test')
plt.legend()
plt.show()
```

Logistic Regression

Logistic Regression

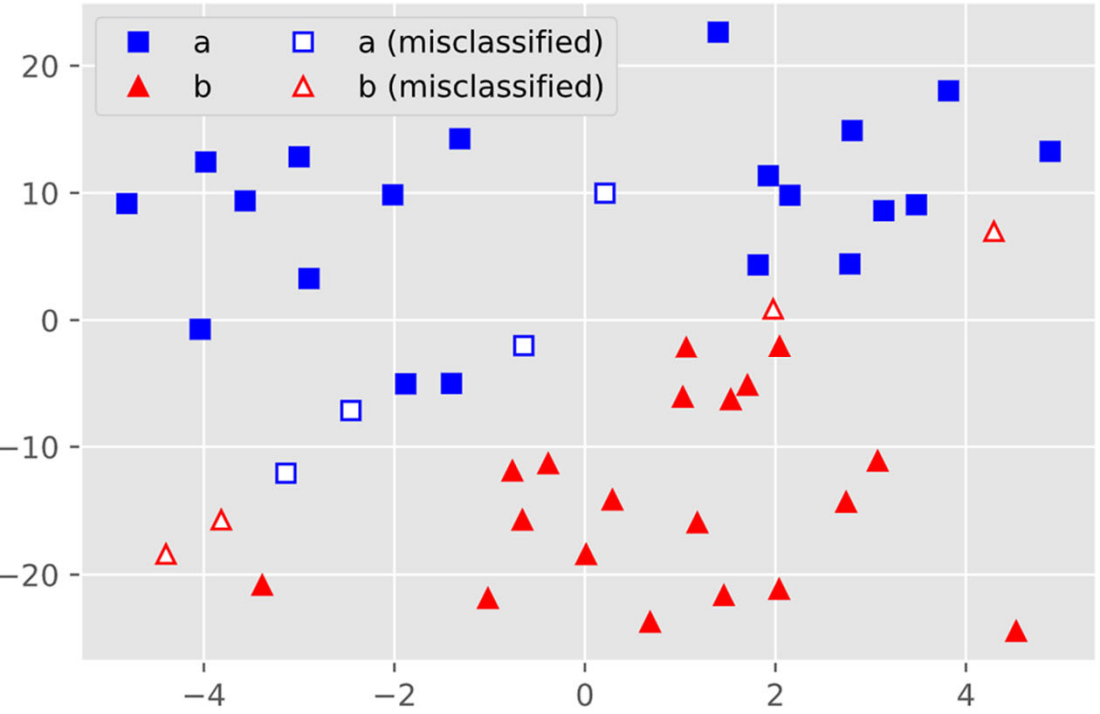
- Logistic regression takes a different approach than KNN
- Rather than modeling the two classes as numbers, it predicts the probability of belonging to a specific group
- The best way to think about logistic regression is that it is a linear regression, but for classification problems
- We fit the model using the logistic function:

$$\text{Logistic function} = \frac{1}{1 + e^{-x}}$$

Common vs acceptable threshold in linear regression

- We often use the probability of 0.5 to map the outcome of logistic function to binary classes (common threshold)
- The threshold can be set equal to 0.7, 0.8, or any other value in the (0,1) range (acceptable threshold)
- The outcome of the sigmoid function does not really become equal to 0 or 1. This means there is always some uncertainty

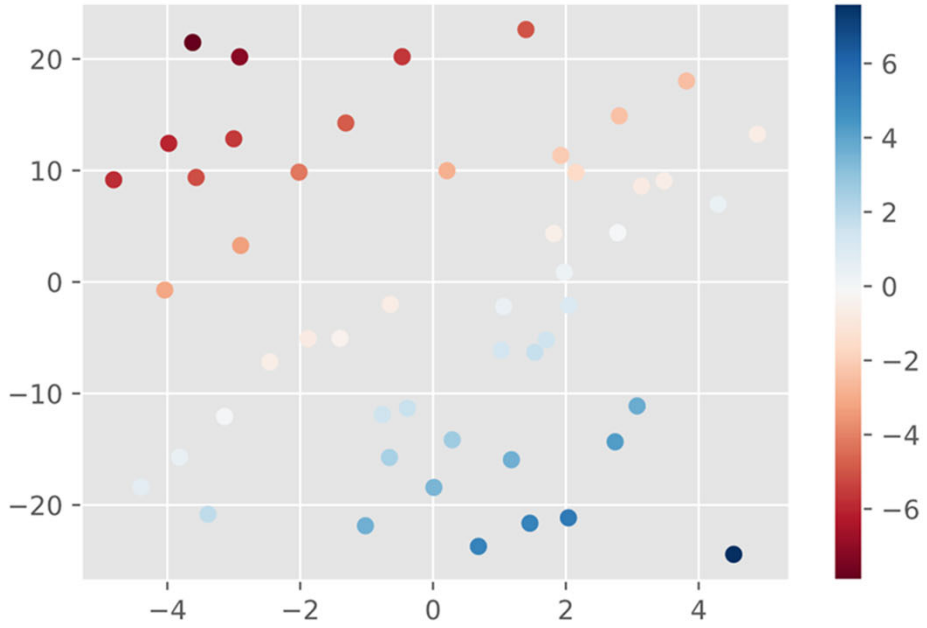
Logistic Regression (results from code)




```
model = linear_model.LogisticRegression()
model.fit(train[['x','y']], train['class'])

def plot_predicted_1(model, test):
    predicted = model.predict(test[['x','y']])
    correct = test[test['class'] == predicted]
    correcta = correct[correct['class'] == 'a']
    correctb = correct[correct['class'] == 'b']
    incorrect = test[test['class'] != predicted]
    incorrecta = incorrect[incorrect['class'] == 'b']
    incorrectb = incorrect[incorrect['class'] == 'a']

    plt.plot(correcta['x'], correcta['y'], 'bs', label='a')
    plt.plot(correctb['x'], correctb['y'], 'r^', label='b')
    plt.plot(incorrecta['x'], incorrecta['y'], 'bs',
             markerfacecolor='w', label='a (misclassified)')
    plt.plot(incorrectb['x'], incorrectb['y'], 'r^',
             markerfacecolor='w', label='b (misclassified)')
    plt.legend(loc='upper left', ncol=2, framealpha=1)

plot_predicted_1(model, test)
```

Decision function of Logistic Regression

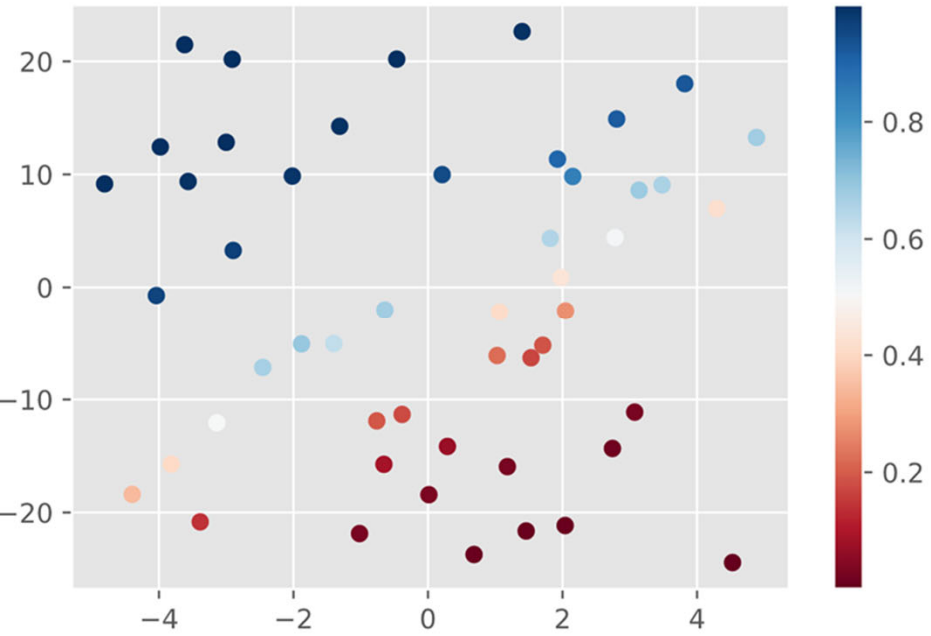
- The logistic regression model also yields a decision function that determines the distance for each point from the dividing hyperplane




```
plt.figure(figsize=(6,4))
plt.scatter(test['x'], test['y'],
            c=model.decision_function(test[['x','y']]),
            cmap='RdBu')
plt.colorbar()
plt.show()
```

Generated probabilities of Logistic Regression

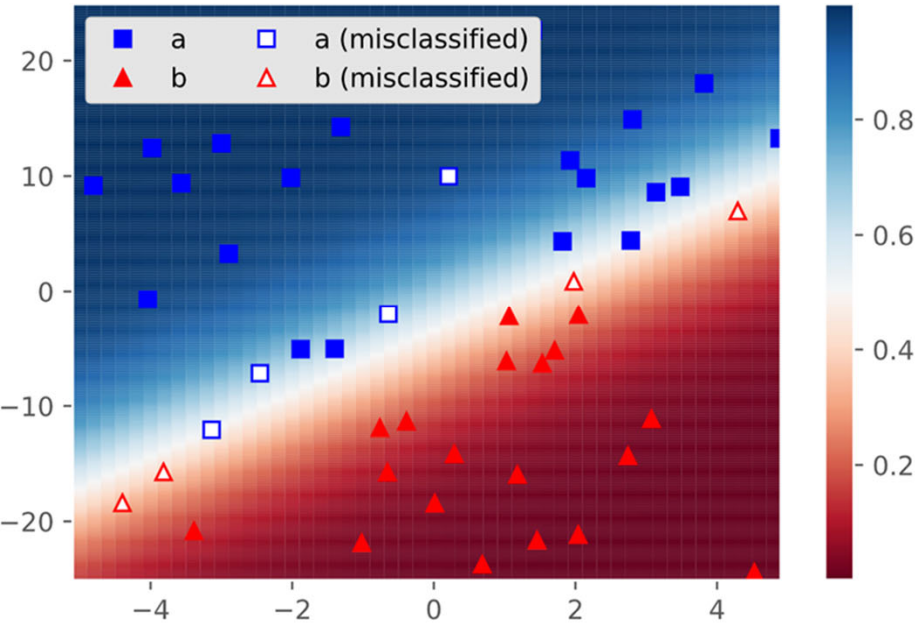
- We were estimating class membership probabilities
- This means we can retrieve these probabilities




```
plt.figure(figsize=(6,4))
plt.scatter(test['x'], test['y'],
            c=model.predict_proba(test[['x','y']])[:,0],
            cmap='RdBu')
plt.colorbar()
plt.show()
```

Separating plane of Logistic Regression

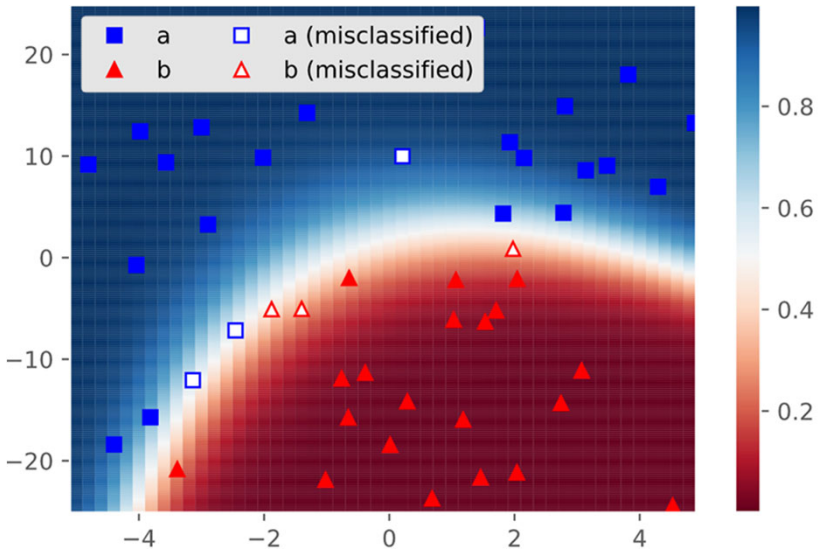
- We can also plot the probabilities in the plane




```
def plot_probabilities(model, test):
    cmap = 'RdBu'
    xmin, xmax, ymin, ymax = -5, 5, -25, 25
    grid_size = 0.2
    xx, yy = np.meshgrid(np.arange(xmin, xmax, grid_size),
                         np.arange(ymin, ymax, grid_size))
    pp = model.predict_proba(np.c_[xx.ravel(), yy.ravel()])[:,0]
    zz = pp.reshape(xx.shape)
    plt.figure()
    plt.pcolormesh(xx, yy, zz, cmap = cmap)
    plt.colorbar()
    plot_predicted_1(model, test)

plot_probabilities(model, test)
plt.show()
```

Logistic Regression with 2nd order polynomial

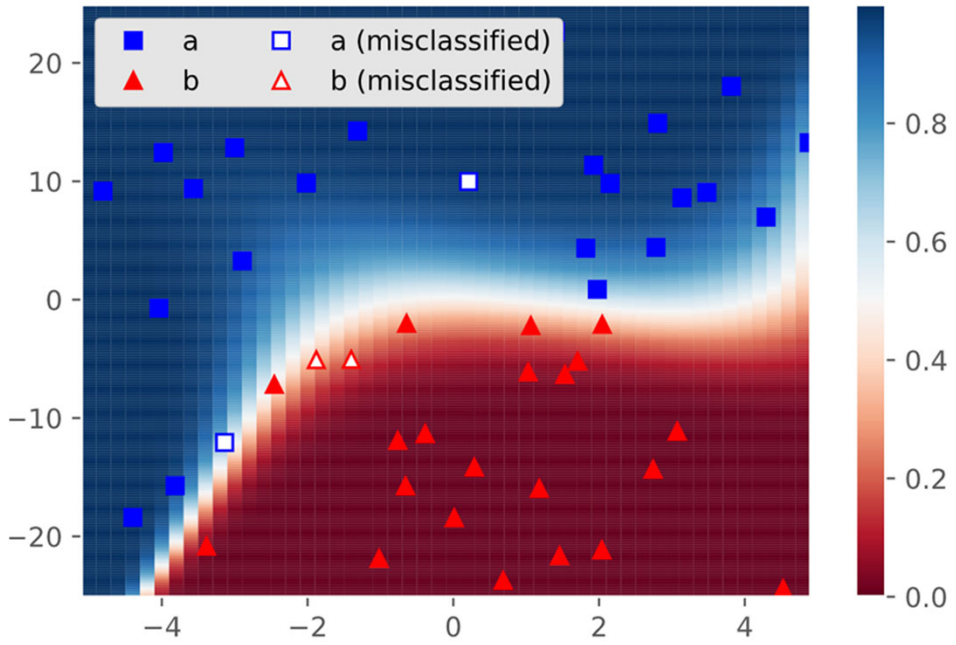
- Simple polynomial model:
  - Using sklearn.preprocessing.PolynomialFeatures We can generate polynomial expansions of our base features to any degree desired
  - New features $1, x, y, x^2, xy, y^2$



Test score: 0.88

To-Do: Extend the code yourself

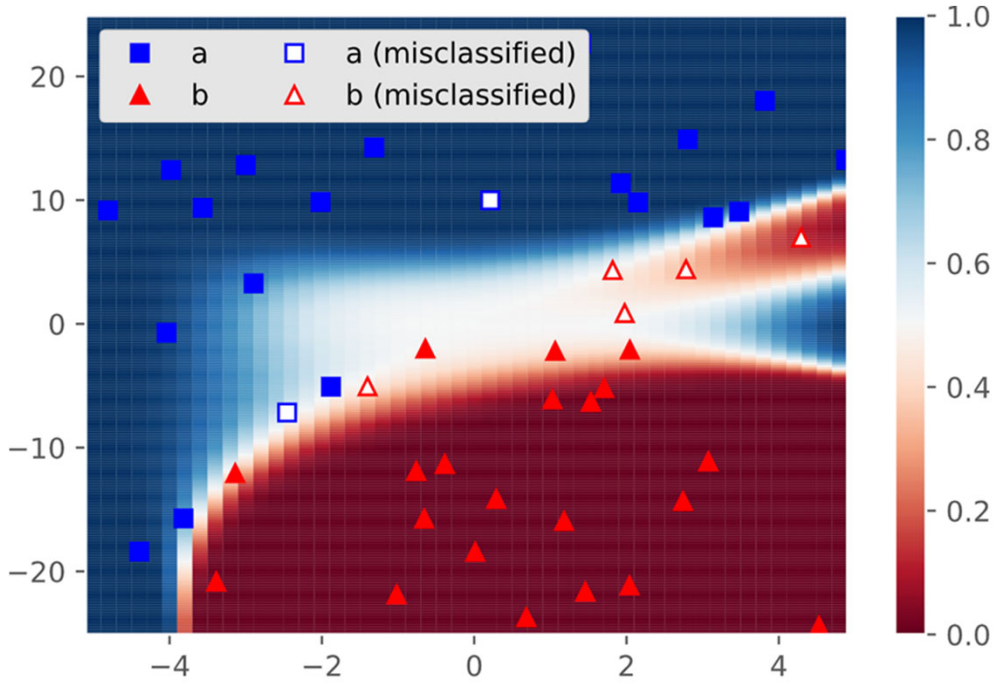
Logistic Regression with 3rd order polynomial



Test score: 0.92

To-Do: Extend the code yourself

Logistic Regression with 4th order polynomial

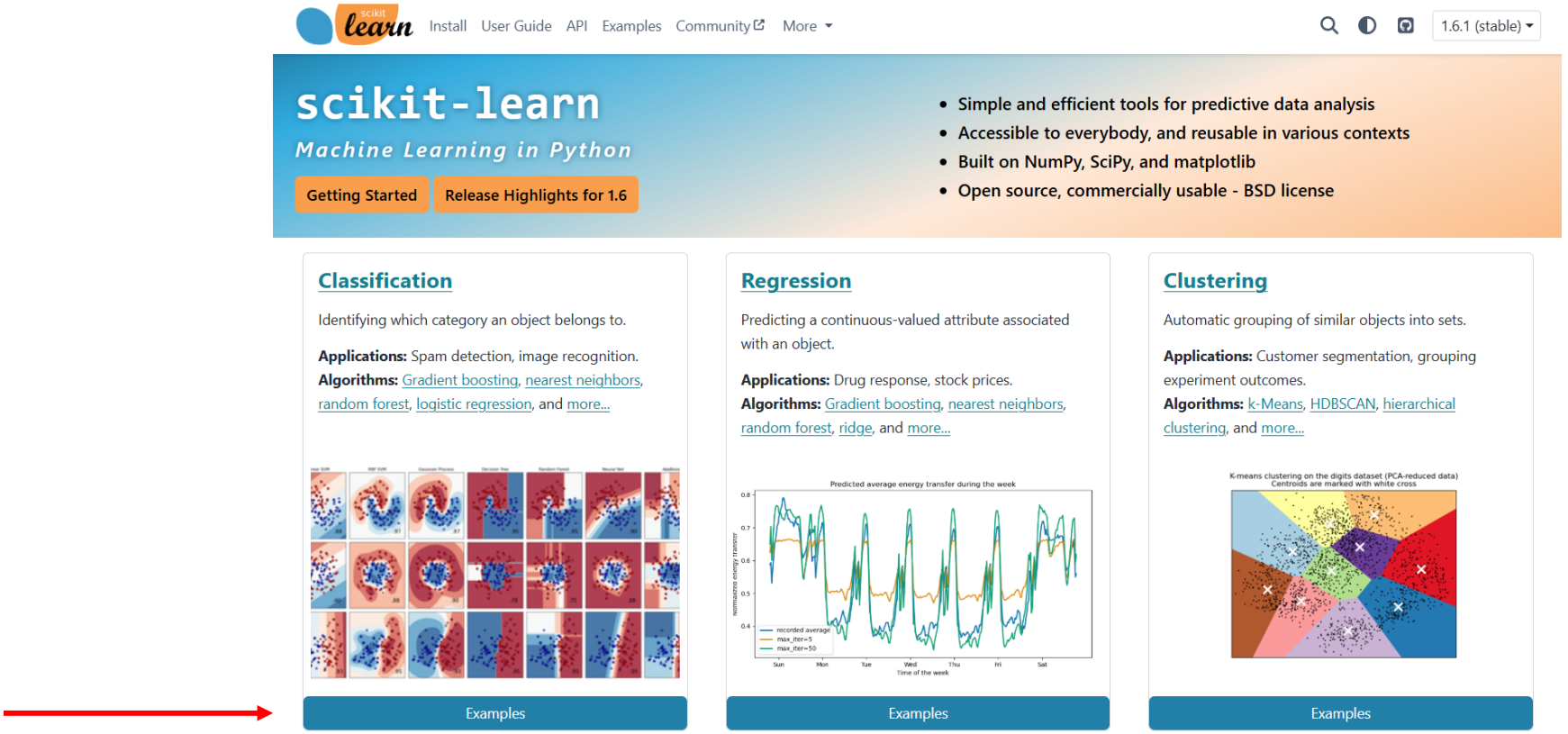


Test score: 0.86

How can we run another classifier that is not discussed in class?

Check sample codes on scikit-learn

- Go to scikit-learn main page (https://scikit-learn.org/stable/index.html)

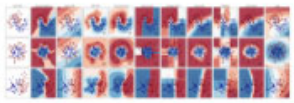


Go to the classifier comparison page

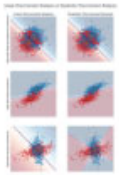
- Download the code accessible from the link below

Classification

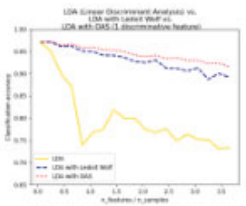
General examples about classification algorithms.



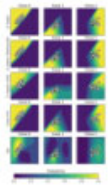
Classifier comparison



Linear and Quadratic Discriminant Analysis with covariance ellipsoid



Normal, Ledoit-Wolf and OAS Linear Discriminant Analysis for classification



Plot classification probability

Recognizing hand-written digits

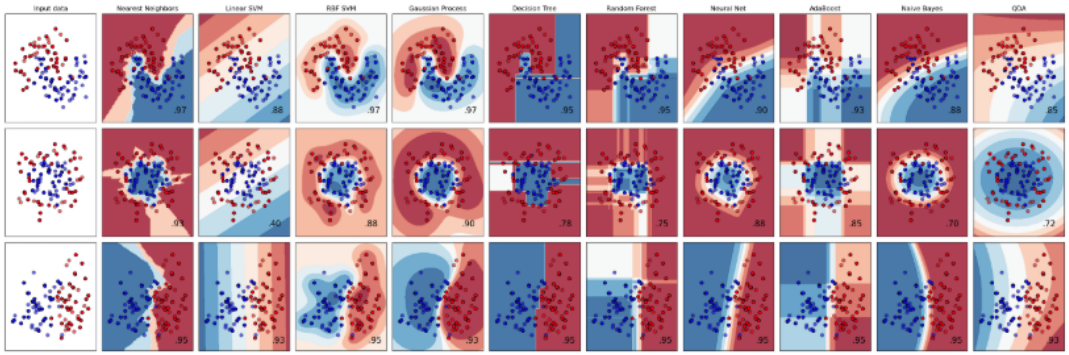
Run the downloaded code on your machine to compare them

Classifier comparison

A comparison of several classifiers in scikit-learn on synthetic datasets. The point of this example is to illustrate the nature of decision boundaries of different classifiers. This should be taken with a grain of salt, as the intuition conveyed by these examples does not necessarily carry over to real datasets.

Particularly in high-dimensional spaces, data can more easily be separated linearly and the simplicity of classifiers such as naive Bayes and linear SVMs might lead to better generalization than is achieved by other classifiers.

The plots show training points in solid colors and testing points semi-transparent. The lower right shows the classification accuracy on the test set.

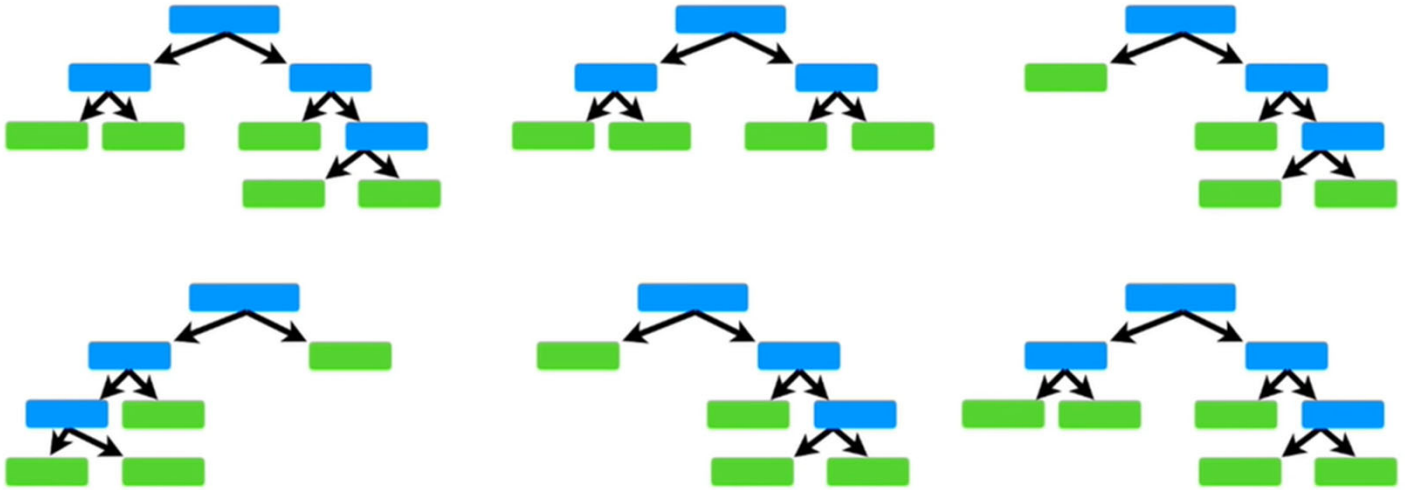


- Q: How would you apply different classifiers to your data?

Random Forest

Random Forest Classification are Ensemble Machine Learning

- Weak Learners form Strong Learns with lower variance and better performance
- Reduces variance with minimal increase in bias
- Difficult to interpret and lack of transparency as compared to decision tree (discussed later in the course)



Access the code on CANVAS

- Code name: RandomForest
- Also upload the data file: TOC_Prediction_DataSet
- Discuss the results

Data Analytics (PETR 6397)

Support Vector Machines, Decision Trees, and Random Forests

Dr. Ahmad Sakhaee-Pour

Petroleum Engineering Department

University of Houston

Spring 2025

Discussed topics

- Support Vector Machines
- Decision Trees
- Ensemble Learning
- Random Forests

Support Vector Machine (SVM)

- SVM is a supervised algorithm for classification and regression
- Its objective is to maximize the margin between support vectors through a separating hyperplane
- The objective of SVM is NOT minimizing the cost function used in many other ML algorithms

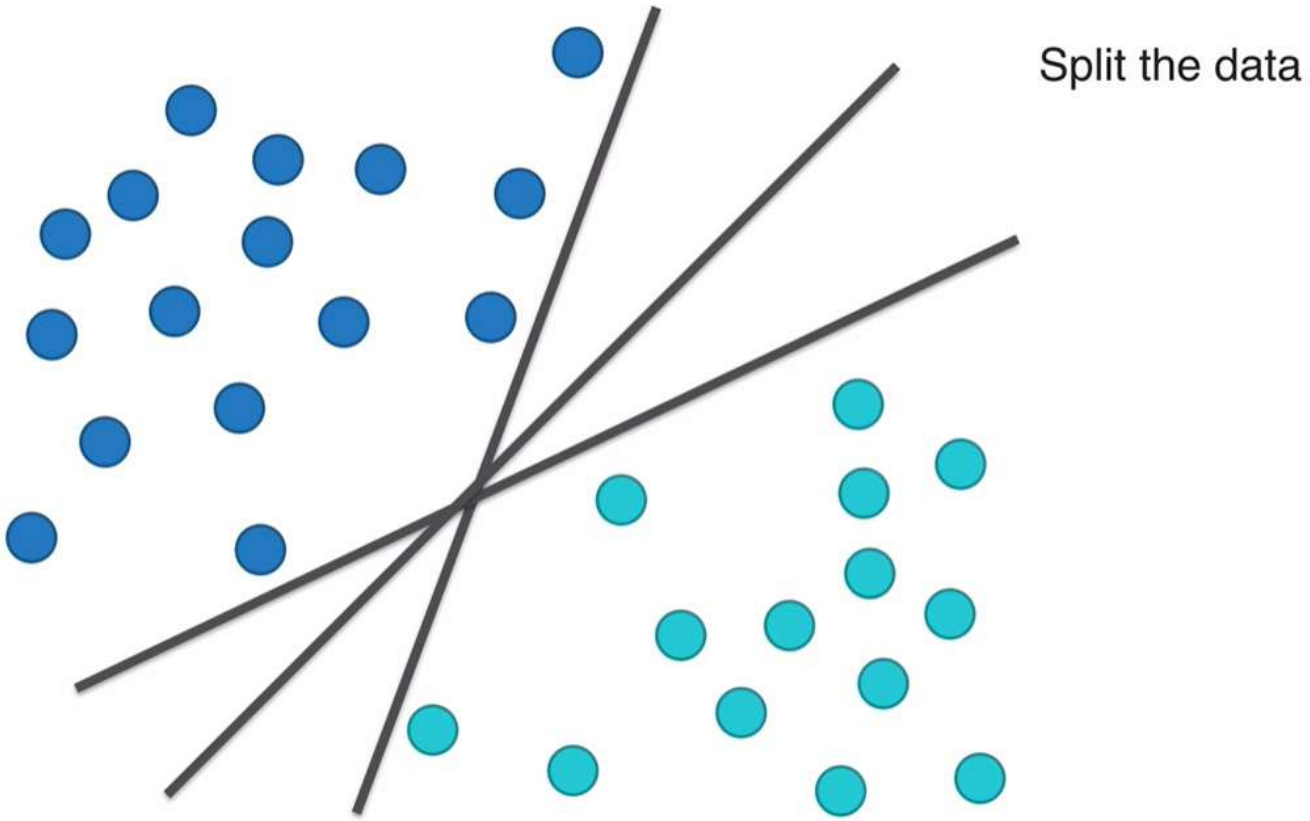
Fundamental properties of SVM

- It performs linear and non-linear classification and regression

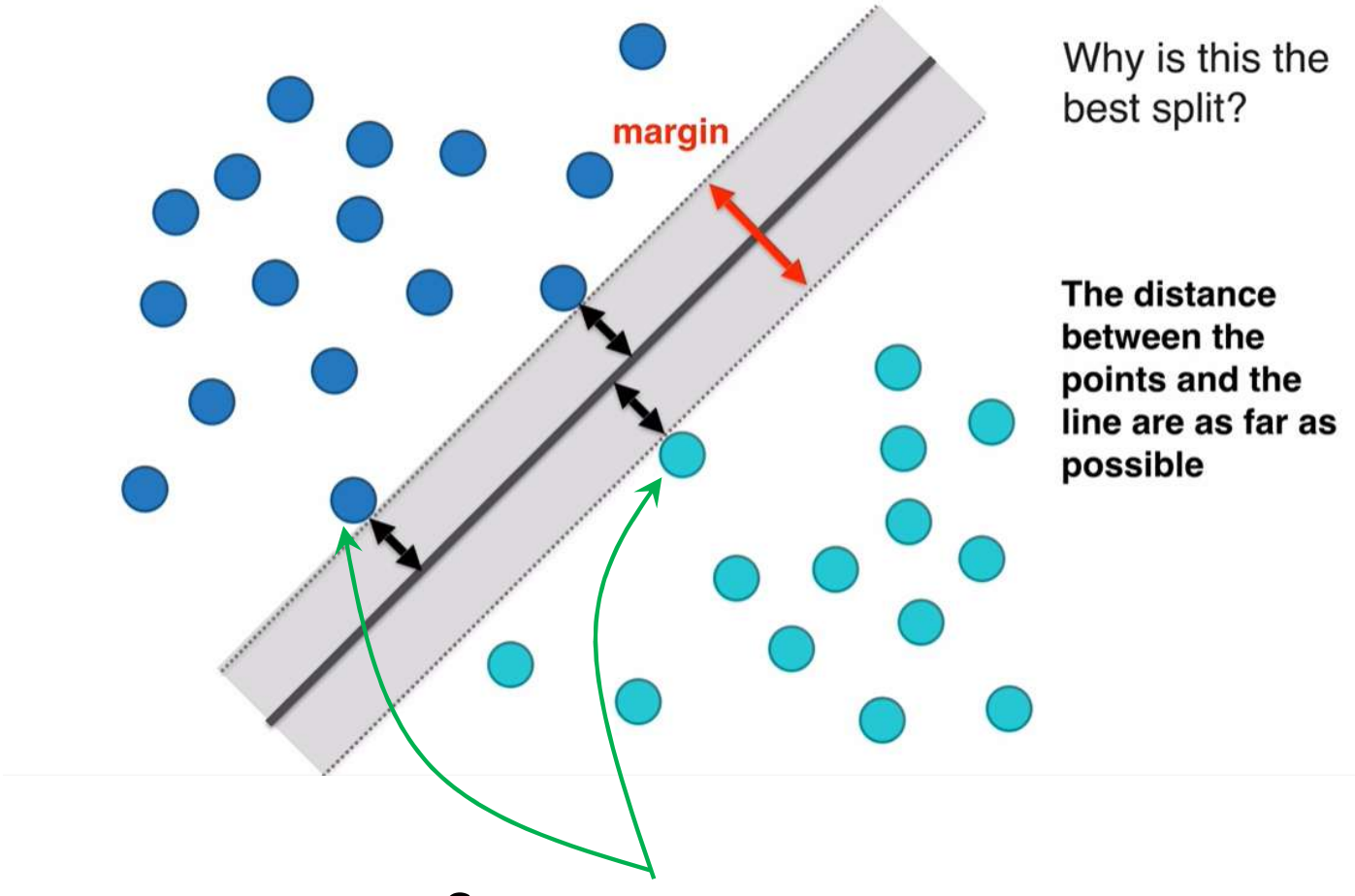
- SVM is called large margin classification because it fits the widest possible street between the classes

- They do not scale well (a major shortcoming when dealing with large datasets)

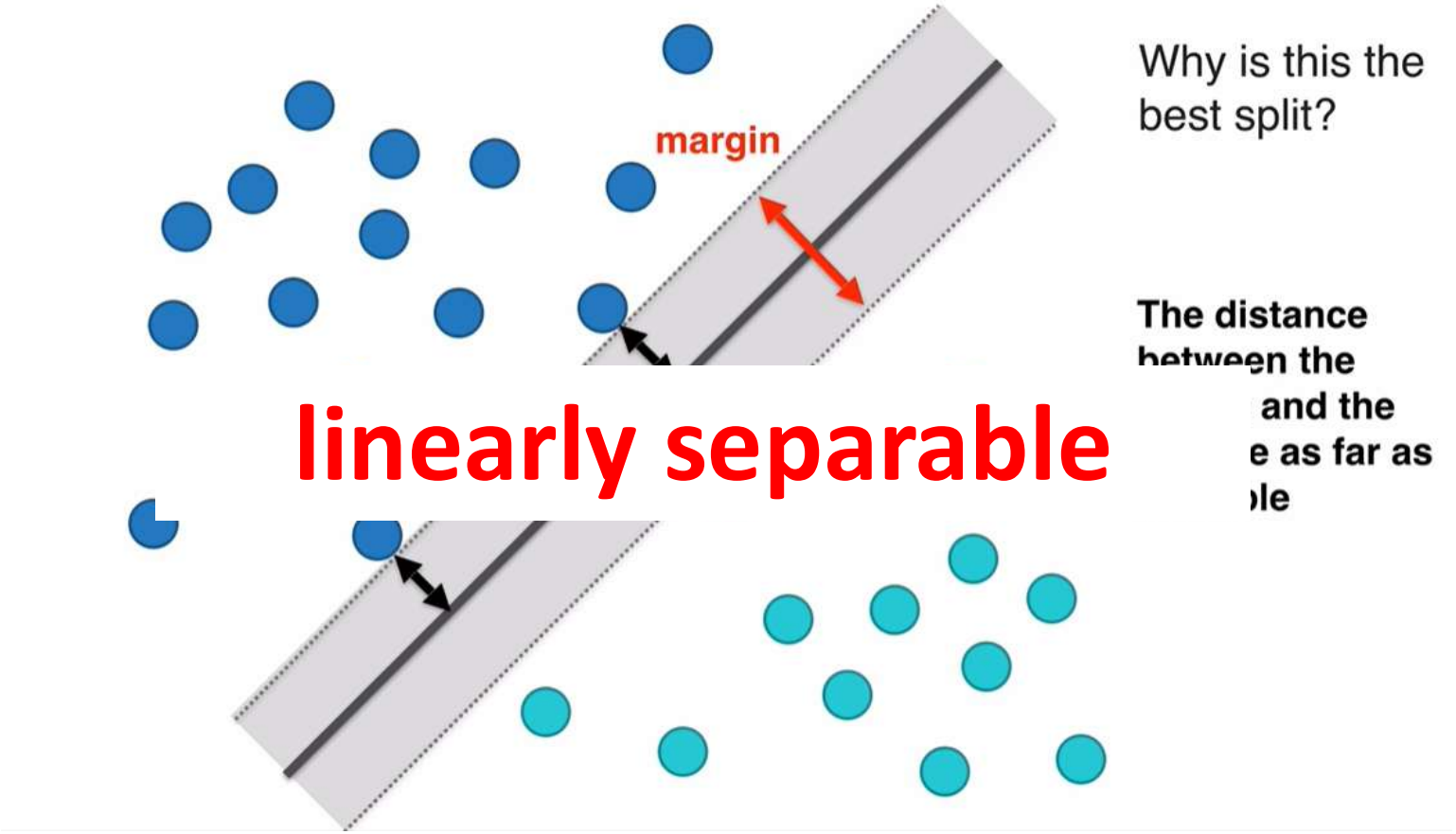
There are different ways to split (divide) the data



SVM goal is to maximize the margin between the support vectors



Original SVM works when data are linearly separable



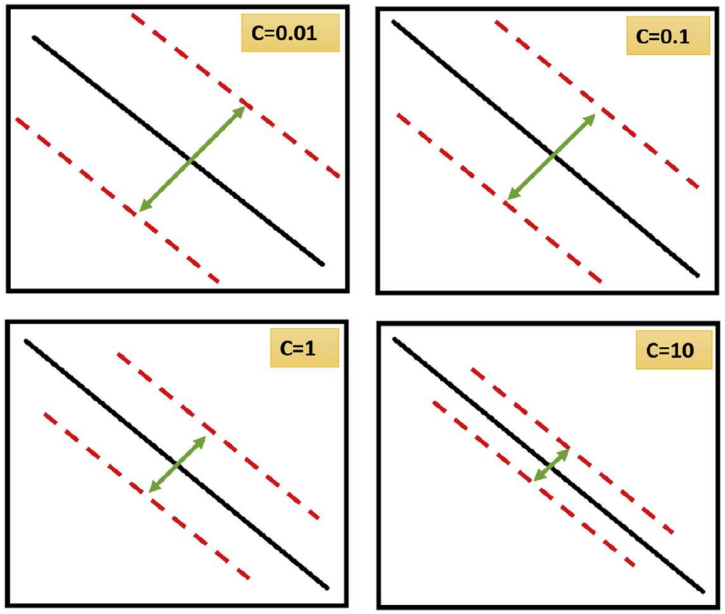
Hard margin versus soft margin

- Hard margin: all instances are off the street
- Hard margin works well if data are linearly separable
- Hard margin is sensitive to outliers (consider a scenario where one sample from one group is in the middle of another)

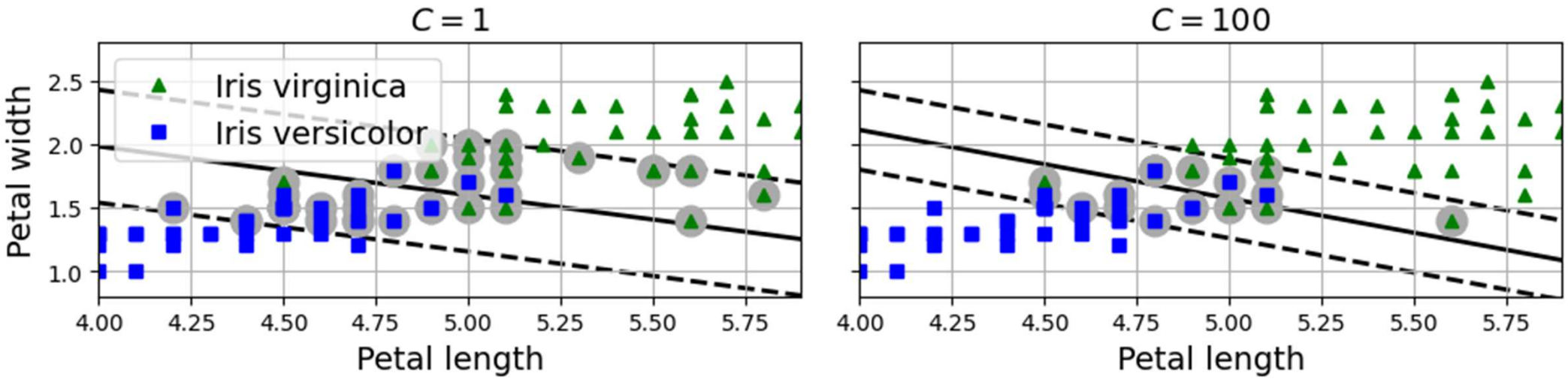
- Soft margin: we allow some samples to violate the margin but increase the street as wide as possible
- Soft margin addresses the hard margin shortcoming of dealing with non-linear data and outliers

Degree of tolerance of soft margin is captured by C (Cost)

- Smaller C:
  - Larger margin
  - Smaller penalty when misclassified
  - Lower training accuracy
  - More general
- Larger C:
  - Smaller margin
  - Larger penalty when misclassified
  - Higher training accuracy
  - Less general



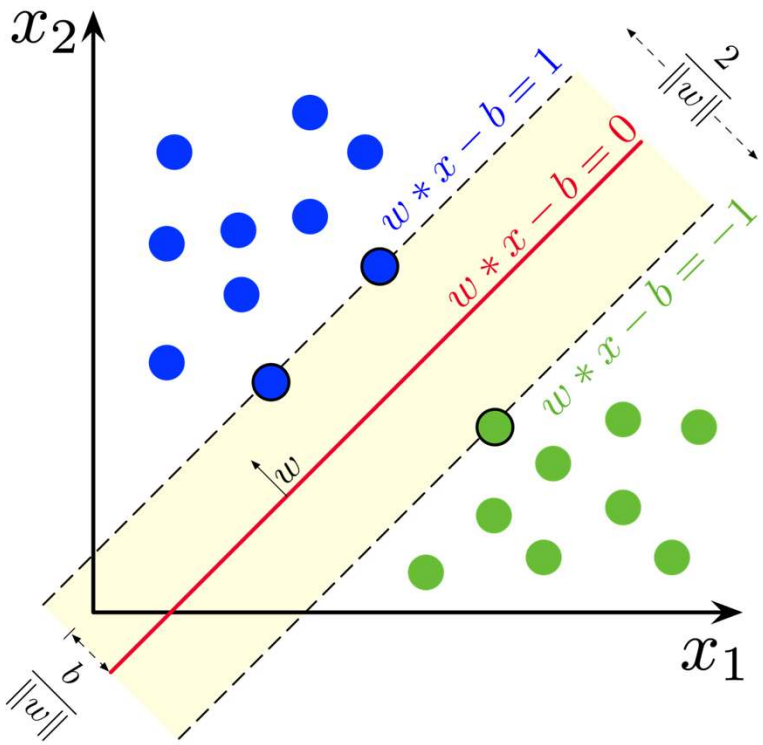
Which scenario is better?



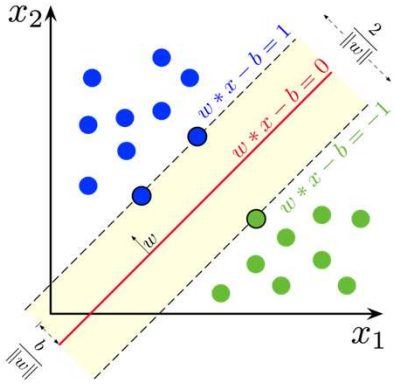
How does Linear SVM work?

- **w** is the vector normal to the plane
- **w**=(w1,w2,…,wn)
- b is the bias

- The goal is to maximize the margin ($2/\|\mathbf{w}\|$). So we minimize $\|\mathbf{w}\|$ based on hard margin and soft margin (next slide)



Objectives of linear SVM classifiers



- Hard margin:

  Minimize $\frac{1}{2}\ \mathbf{w}^{\mathrm{T}}\mathbf{w}$

  Subject to $t^{(i)}\left(\mathbf{w}^{\mathrm{T}}\mathbf{w} + \mathbf{b}\right) \geq 1$ for $i = 1,2, \ldots, m$

- Soft margin:

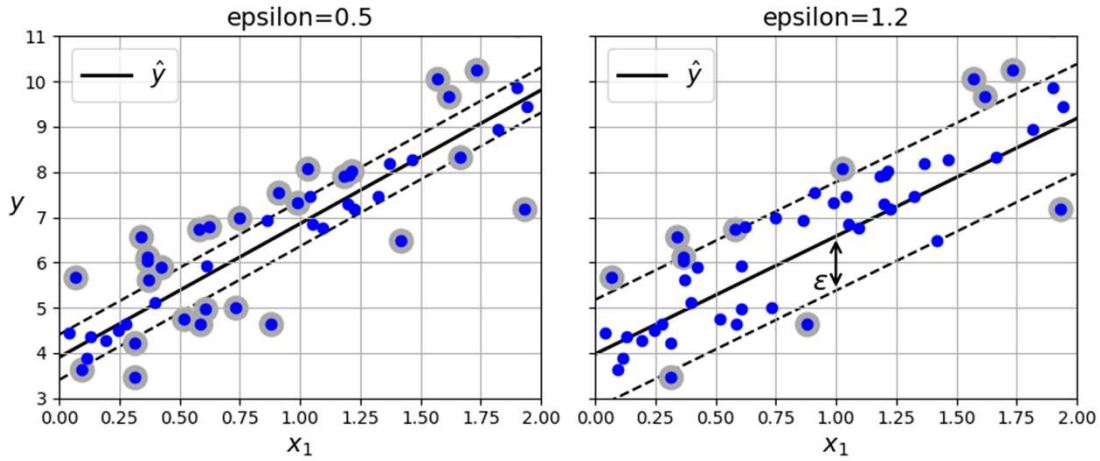
  Minimize $\frac{1}{2}\ \mathbf{w}^{\mathrm{T}}\mathbf{w} + C \sum_{i=1}^{m} \zeta^{i}$

  Subject to $t^{(i)}\left(\mathbf{w}^{\mathrm{T}}\mathbf{w} + \mathbf{b}\right) \geq 1 - \zeta^{i}$ for $i = 1,2, \ldots, m$

- $t^{(i)}$ is +1 if the sample is above the plane
- $t^{(i)}$ is -1 if the sample is below the plane

SVM regression

- In regression, SVM fits as many instances as possible on the street while limiting margin violations
- The width is controlled by epsilon



- Reducing epsilon increases the number of support vectors
- Adding more training examples within the margin will not impact the prediction (model is not sensitive to epsilon)

Non-linear SVM

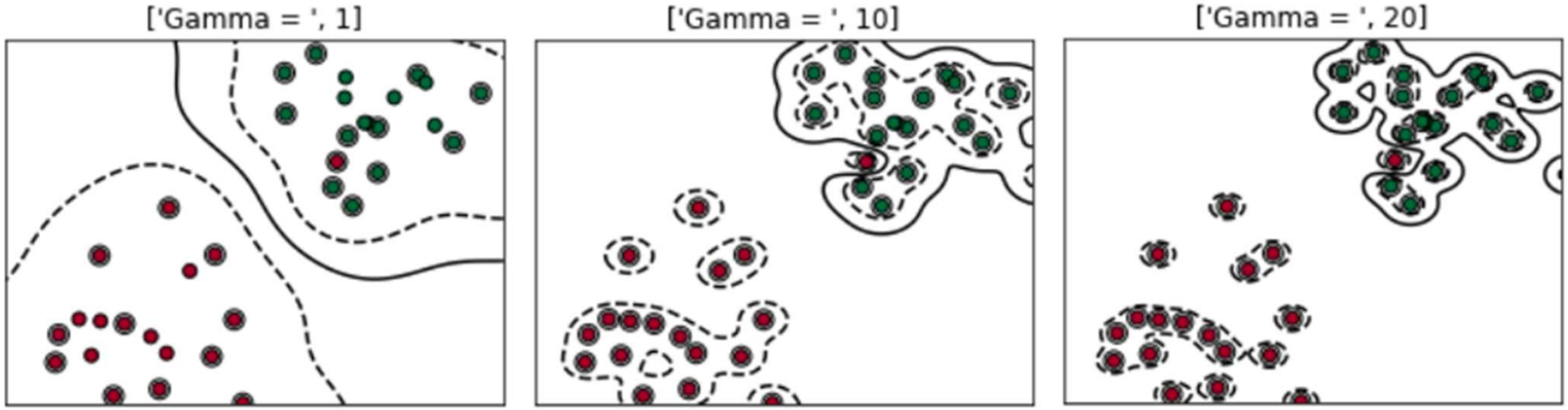
- We can add more features (such as polynomials) and use SVM for non-linear problems. We used this trick for regression in this course.

- We can also apply SVM using kernels (Polynomial kernel, Gaussian RBF kernel)

- Vapnik (and his colleagues) proposed linear SVM in 1963 and extended it to non-linear scenarios in 1992.

Non-linear SVM with Radial Basis Function (RBF) kernel

- Gamma: the inverse of the radius of influence of samples selected as support vectors. Low gamma means a large radius of influence.
  - Low gamma creates a simple model
  - High gamma creates a complex model that may overfit and be sensitive to noise



Decision Trees

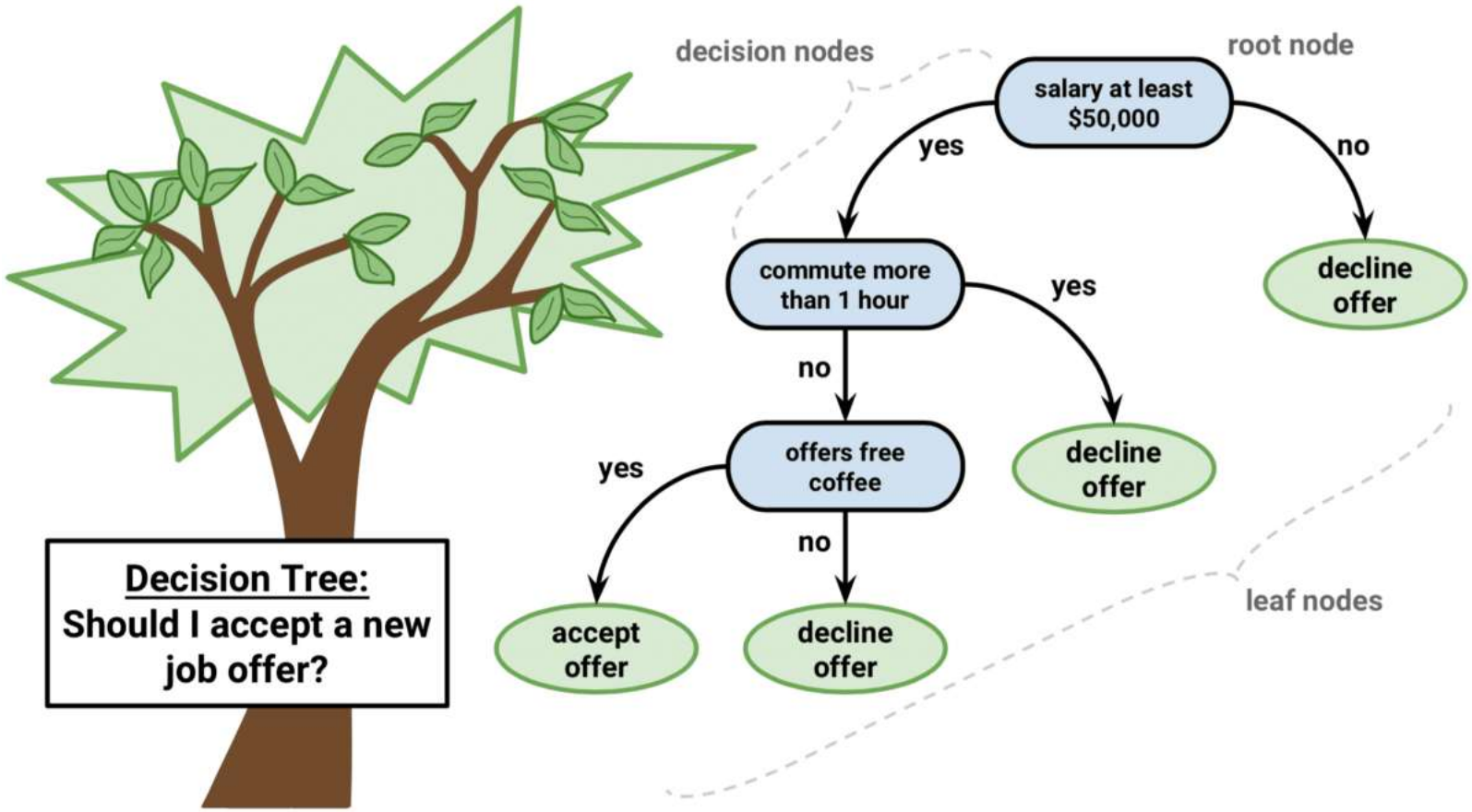
Decision Tree

- Decision tree is also a supervised algorithm for classification and regression
- It is not very powerful
- But it is easy to understand and interpret
- It is expanded with random forest, bagging, and boosting to increase its capabilities

Interesting properties of decision trees

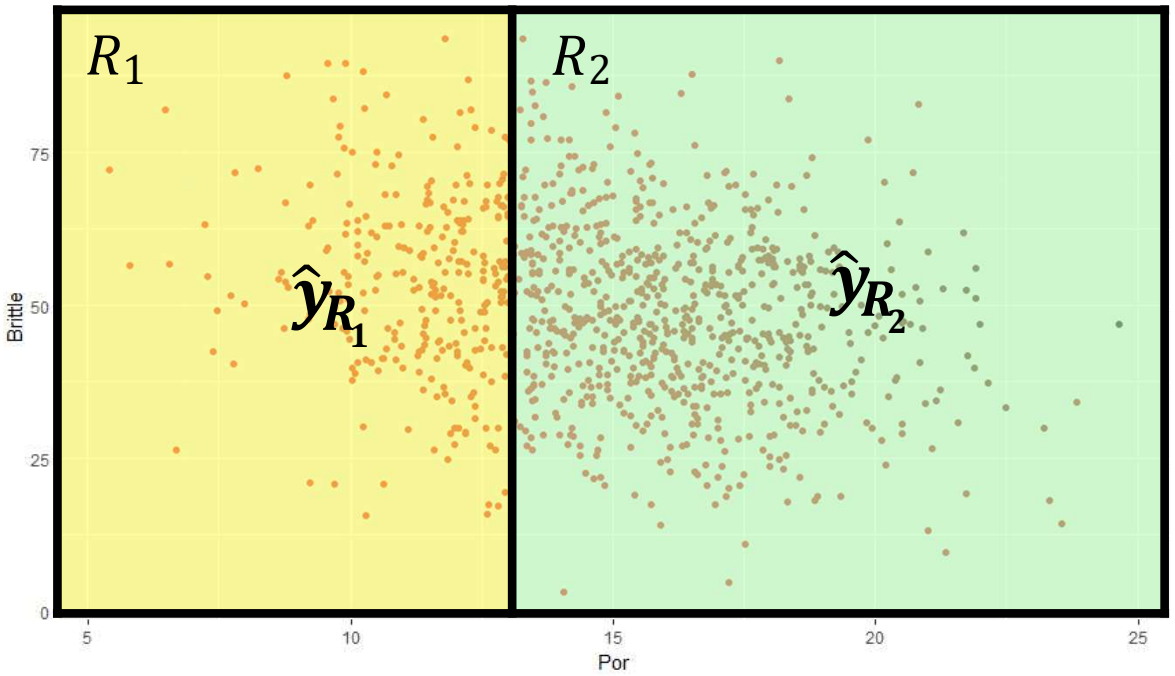
- They require little or no data preparation
- They do not require feature scaling
- Easy to explain
- Similar to how humans make decision
- They are white box models
- Lower predictive accuracy than other machine learning methods

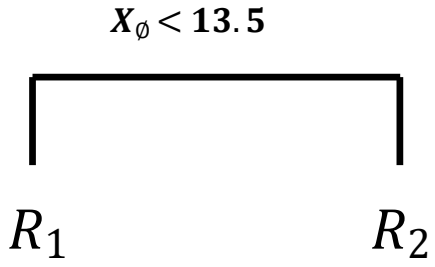
Decision Tree is similar to how we make a decision



Decision Tree for dividing samples based on porosity (1)

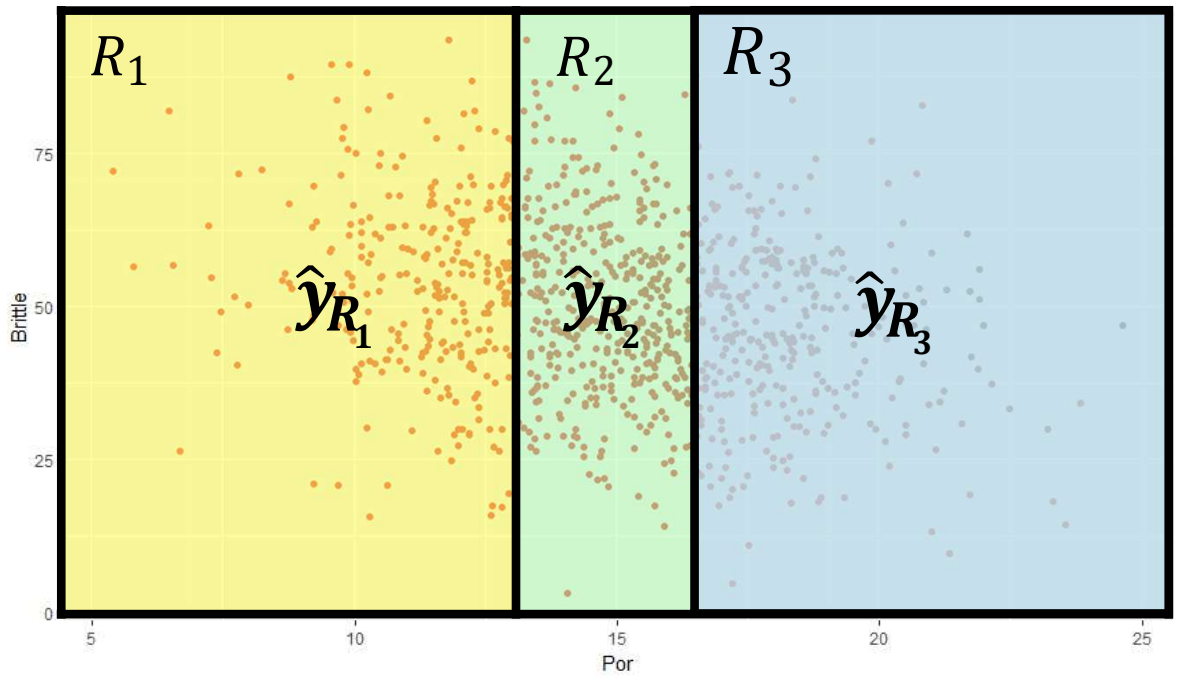
- How do we construct the Regions R_1, R_2, …, R_J?

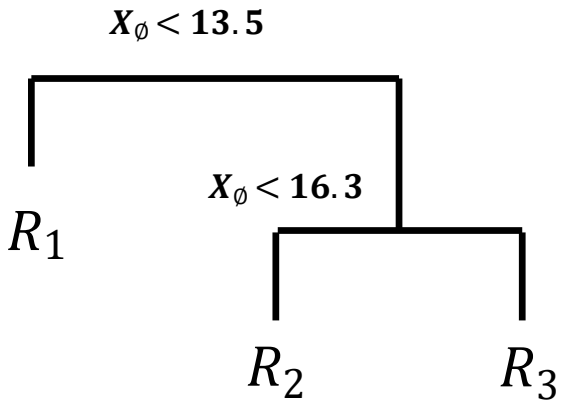




Decision Tree for dividing samples based on porosity (2)

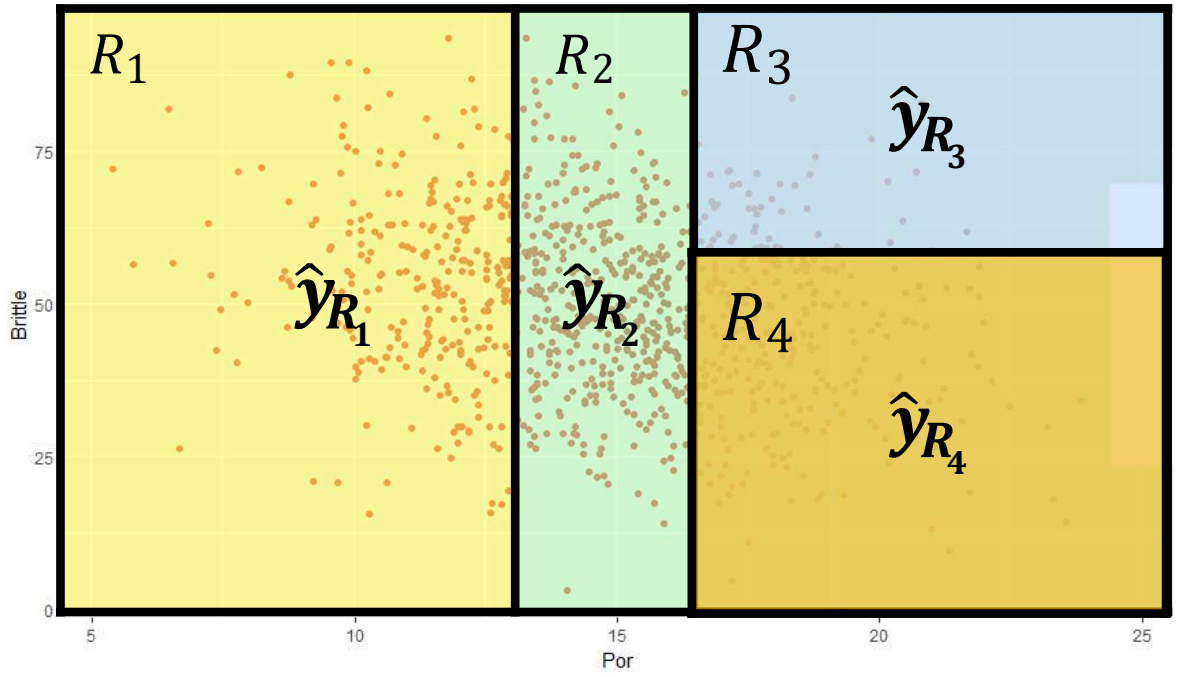
- How do we construct the Regions R_1, R_2, …, R_J?

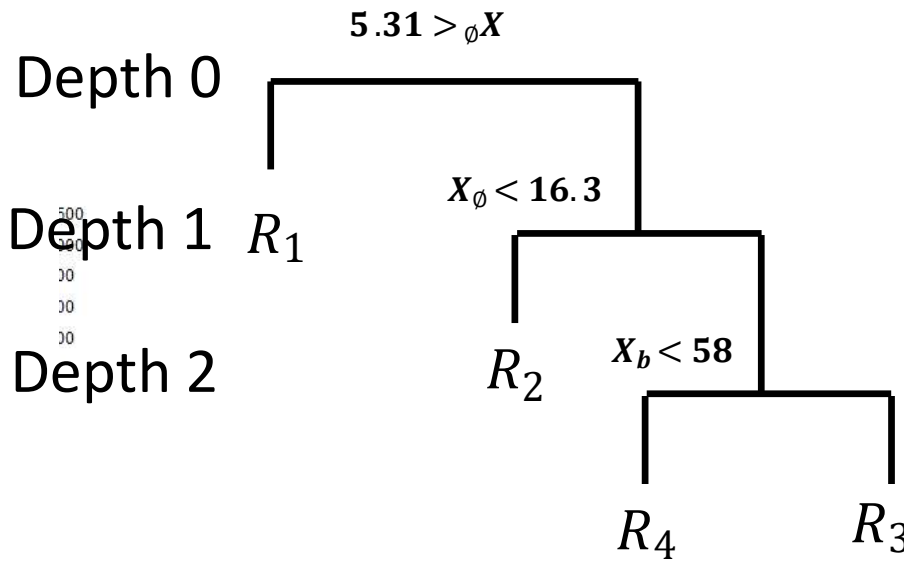




Decision Tree for dividing samples based on porosity (3)

- How do we construct the Regions R_1, R_2, …, R_J?





Depth 0,1,.. shows the penetration into the model

We have different types of nodes in the tree

First node is the root node

Q: what is child node? leaf node? split node?

Termination of decision tree

- When do we stop splitting?
  - We could continue until each datum has its own box!
    - This would be over fit!
  - Typical approach applies a minimum training data in each box criteria
    - It stops when all boxes have reached the minimum
  - We could continue until we cannot reduce Residual Sum of Squares (RSS)
    - But the current split could lead to an even better split → short sighted

We prefer a simplified decision tree

- The splitting process is deterministic, and it continues until there is no impurity (obvious overfit)
- Decision trees, if allowed to grow complicated, will generally overfit
- It is better to simplify the tree to a smaller tree with fewer splits
    - lower model variance
    - better interpretation
    - lower model bias
- Hyperparameter tuning is concerned with finding the optimal values using cross validation (an example is discussed in the second code in this lecture).

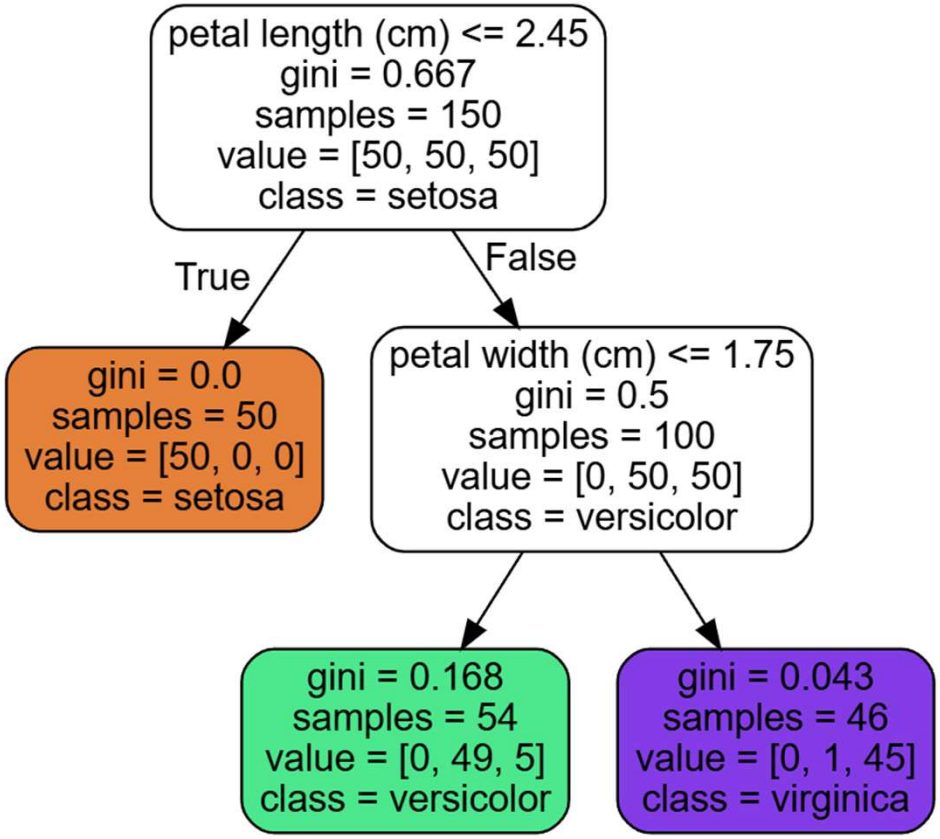
Gini impurity

- Gini impurity shows how often an element would be incorrectly labeled if it was randomly labeled
- gini=0 if all samples belong to a single class (a node is pure)

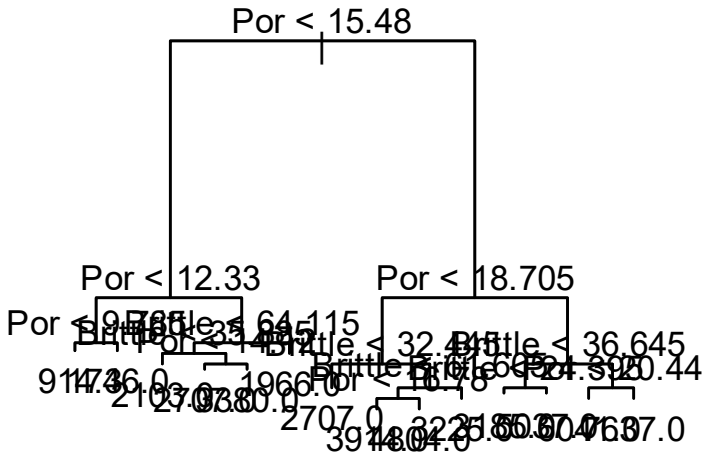
$$G_i = 1 - \sum \left(p_{i,k}\right)^2$$

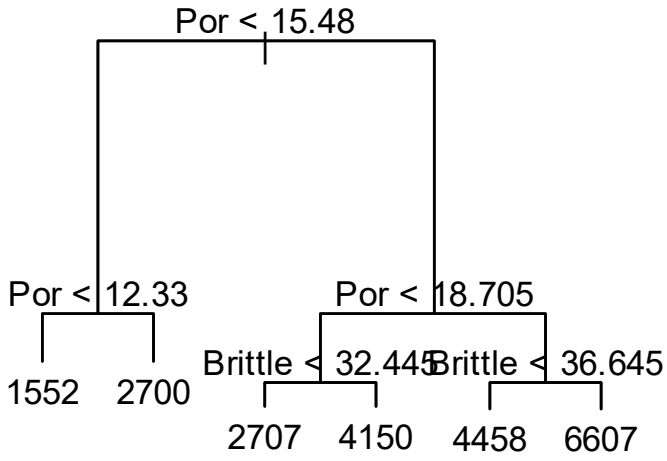
- G_i: gini impurity of the ith node
- $p_{i,k}$: the ratio of class *k* instances among all data in the ith node

Calculate the gini impurities



Is it possible to have more than two child nodes?





Cost function of the decision tree

- Classification and regression tree (CART) searches for pair (k,t_k) to generate the purest subsets possible by minimizing the following cost

$$J(k, t_k) = \frac{m_{left}}{m} G_{left} + \frac{m_{right}}{m} G_{right}$$

G: Gini impurity

$m_{left(or\ right)}$: Number of instances in left or right

- It keeps splitting the data until some criterion stops it

Stopping criterion

A. Classification and Regression Tree (CART) stops if it cannot find a split to lower the impurity (this rarely happens in practice)

B. CART stops splitting when it reaches preset criteria:

B1. It reaches the maximum depth of the model (we set max_depth)

B2. It reaches the max_leaf_nodes

B3. It reaches max_features

….

Note: CART is a greedy algorithm

Entropy is also used in the cost function

- Inspired by the entropy property in thermodynamics (or information theory):

$$H_i = -\sum p_{i,k} \log(p_{i,k})$$

- Entropy and gini impurity usually generate similar decision trees
- When they differ, entropy generates a more balanced decision tree
- Gini is slightly faster (notice the log function in the entropy)

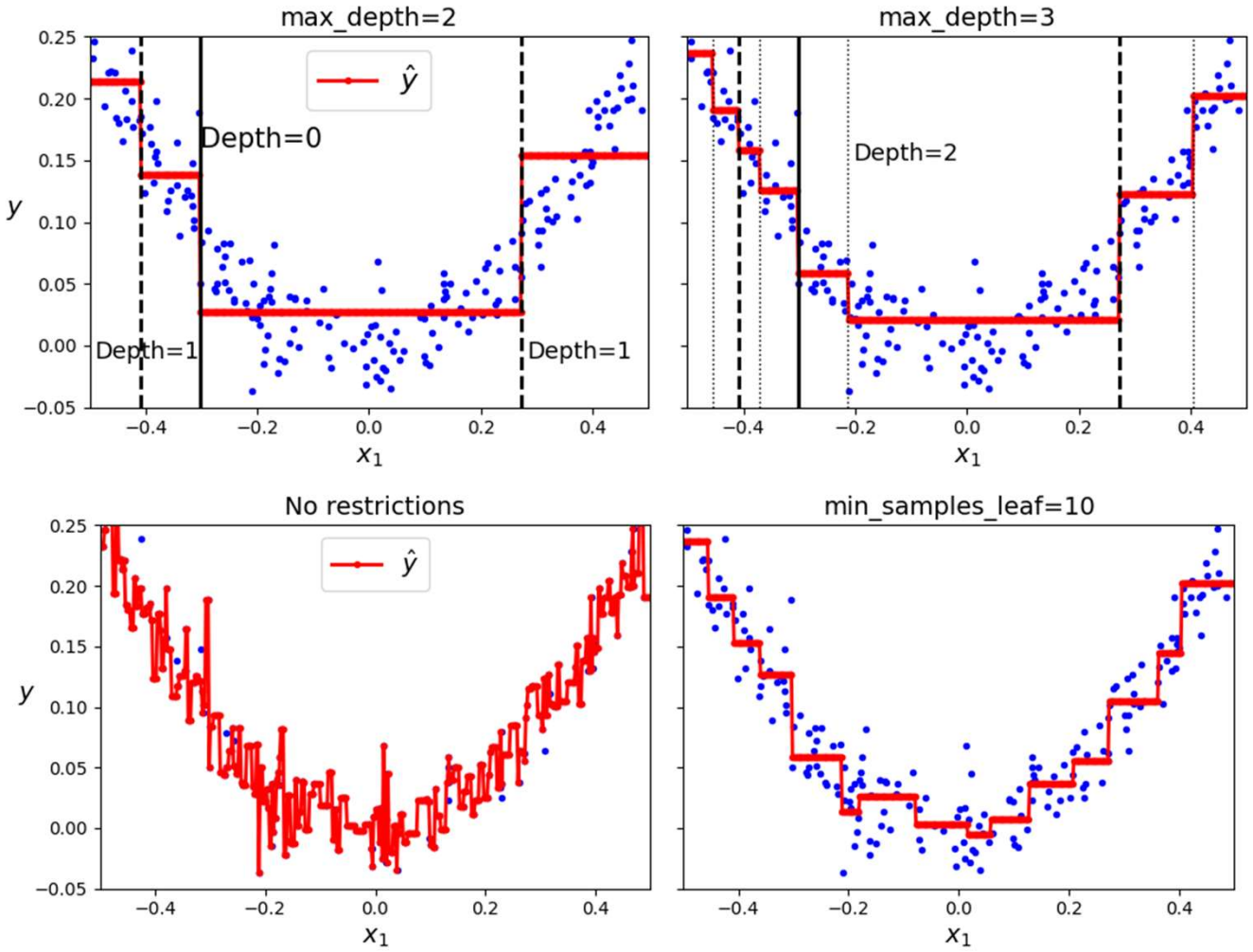
Decision tree also applies to regression but requires a relevant cost function

- It splits the data by minimizing the MSE (instead of gini)

$$J(k, t_k) = \frac{m_{left}}{m} \mathrm{MSE}_{\mathrm{left}} + \frac{m_{right}}{m} \mathrm{MSE}_{\mathrm{right}}$$

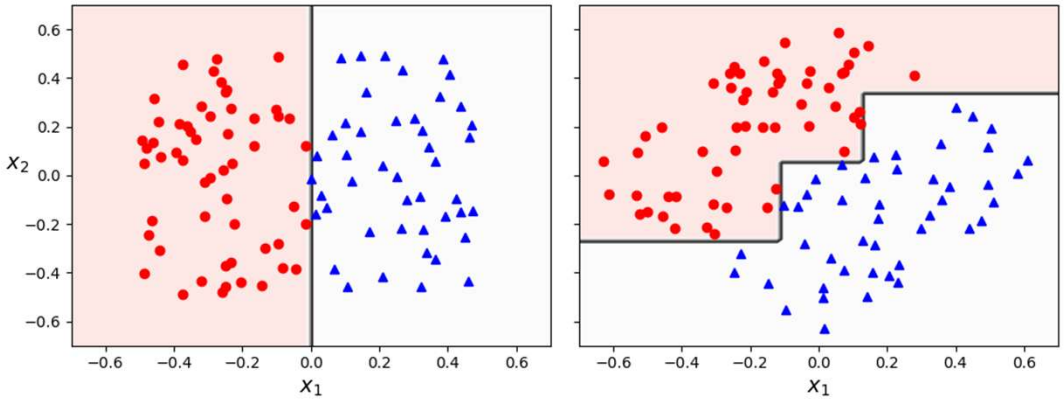
$\mathrm{MSE}_{\mathrm{node}}$ is the mean squared error that captures the difference between the predicted value and the **AVERAGE of actual values** in each node

Example of overfitting with no regularization



Two major shortcomings of decision tree

- They are very sensitive to data orientation because of the orthogonal boundaries



- Small changes in hyperparameters produce different results

Effects of the stochasticity on decision tree

- The training algorithm is stochastic because it randomly selects the features to evaluate at each node

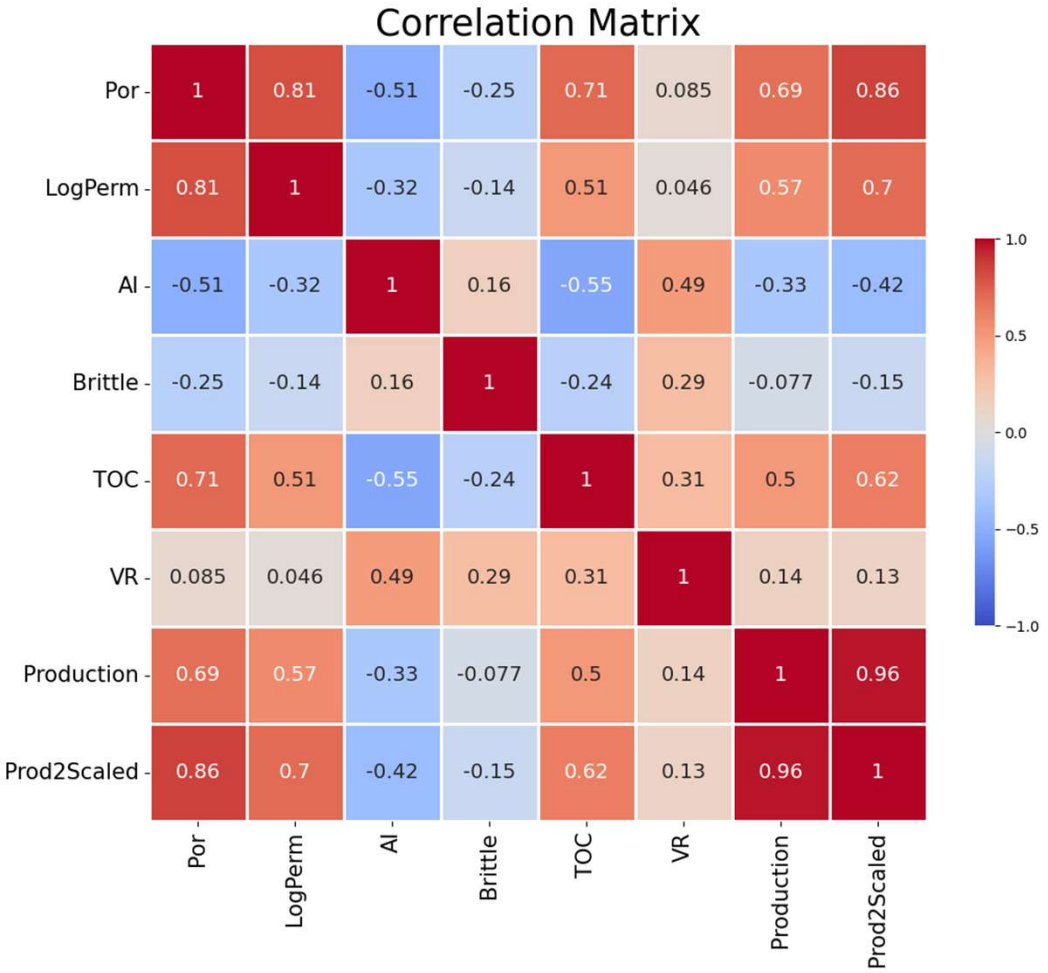
- This means that we may get very different decision trees for the same dataset (unless you set the random_state)

- Remember that the decision tree is a greedy algorithm

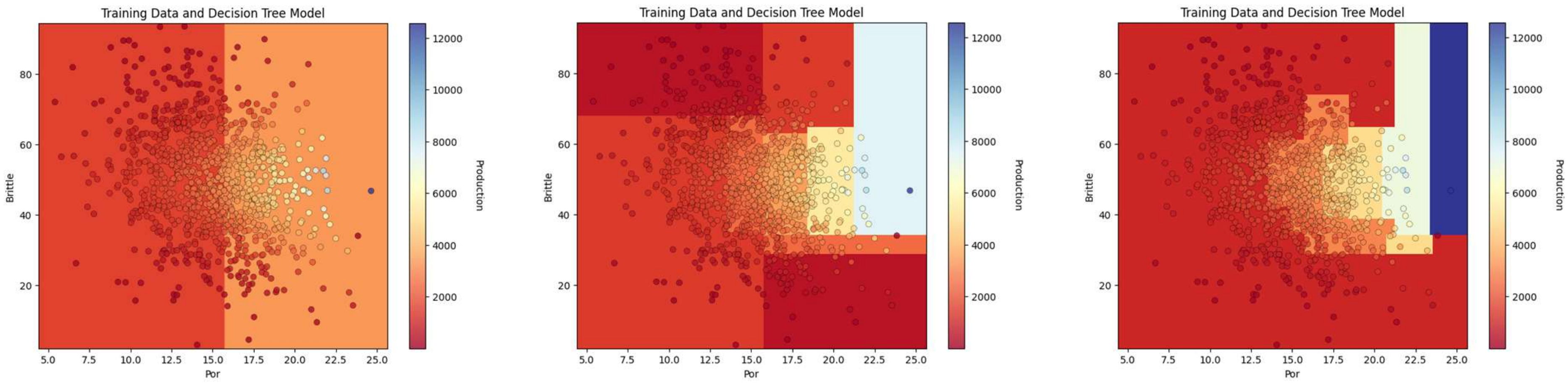
- Q: how do we address the negative impact of stochasticity on the decision tree?

Code 1

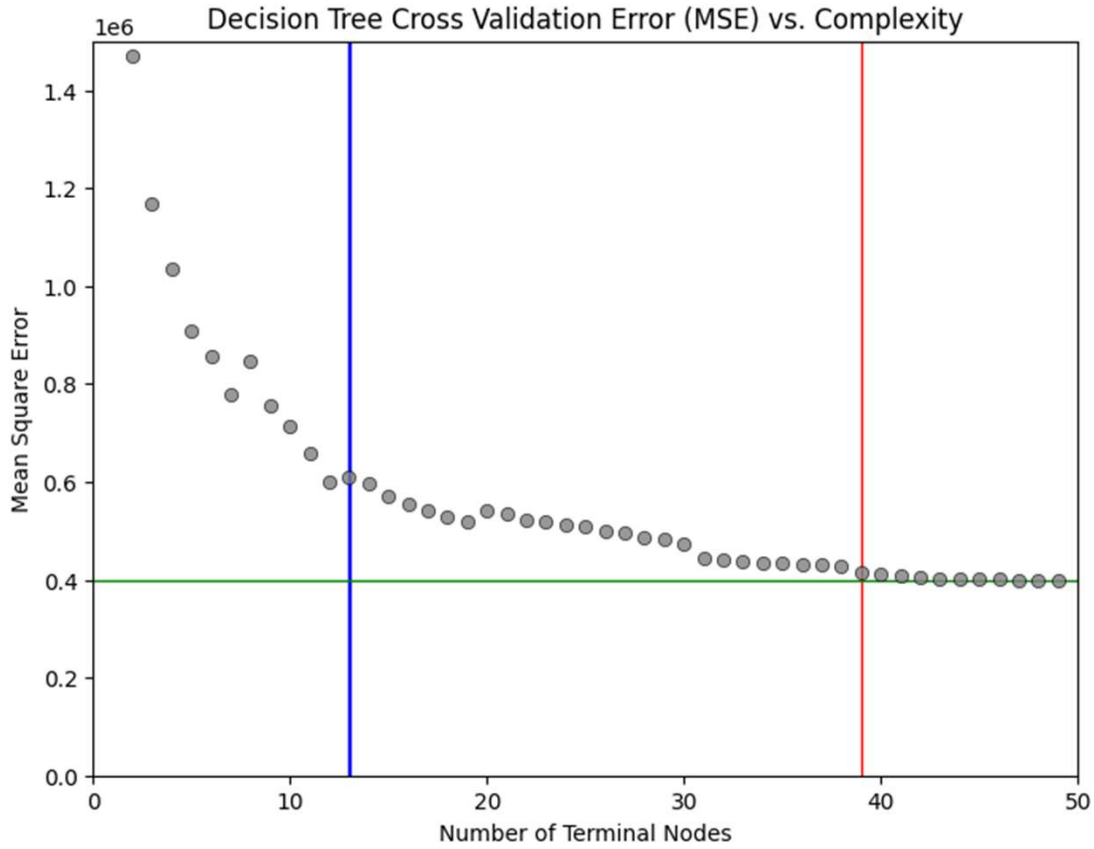
- Run “Decision_Tree” after uploading the data (unconv_MV_v2)
- How would you interpret the correlation matrix



Evolution of the decision tree with increasing the number of nodes



Determine and justify the best number of terminal nodes



- Ensemble Learning and Random Forests

Fundamental idea of ensemble learning

- Main idea: a group of people is smarter than a single person (wisdom of the crowd)
- The same applies to machine learning: a group of classifiers (or regressors) is better than a single classifier
- Ensemble is simply a group of classifiers (or regressors)
- Ensemble learning (sometimes called ensemble method) is based on ensemble models
- Q: how do we choose from the predictions?

Voting classifiers

- Hard voting: choose the outcome with the highest vote (similar to electing people based on the majority vote)
- The outcome is a strong learner even if we use weak learners, especially if there are many

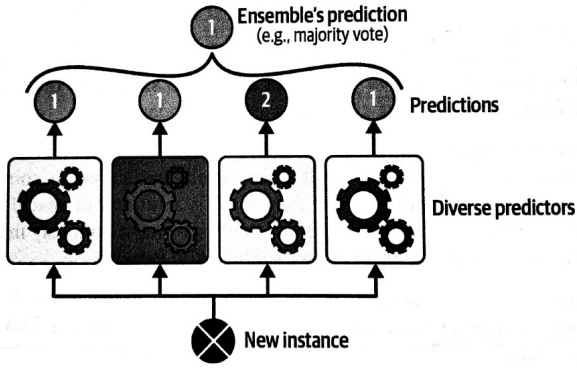


Fig. 7.2 of the reference

- The ensemble (group of classifiers) performs better than its best classifier, again similar to how a society with many people behave
- Soft voting: gives more weight to the models with higher scores (performs better than hard voting)

Methods to improve the ensemble performance

- In society, it would be better to have diverse ideas from different people
- Example of hard coding: In solving data analytics problems, it is better to form a diverse team (scientists, engineers,…)
- Example of soft voting: In solving data analytics problems, it is better to form a diverse team with good scientists and good engineers
- Ensemble methods work best where the predictors are independent

Methods to create diverse classifiers

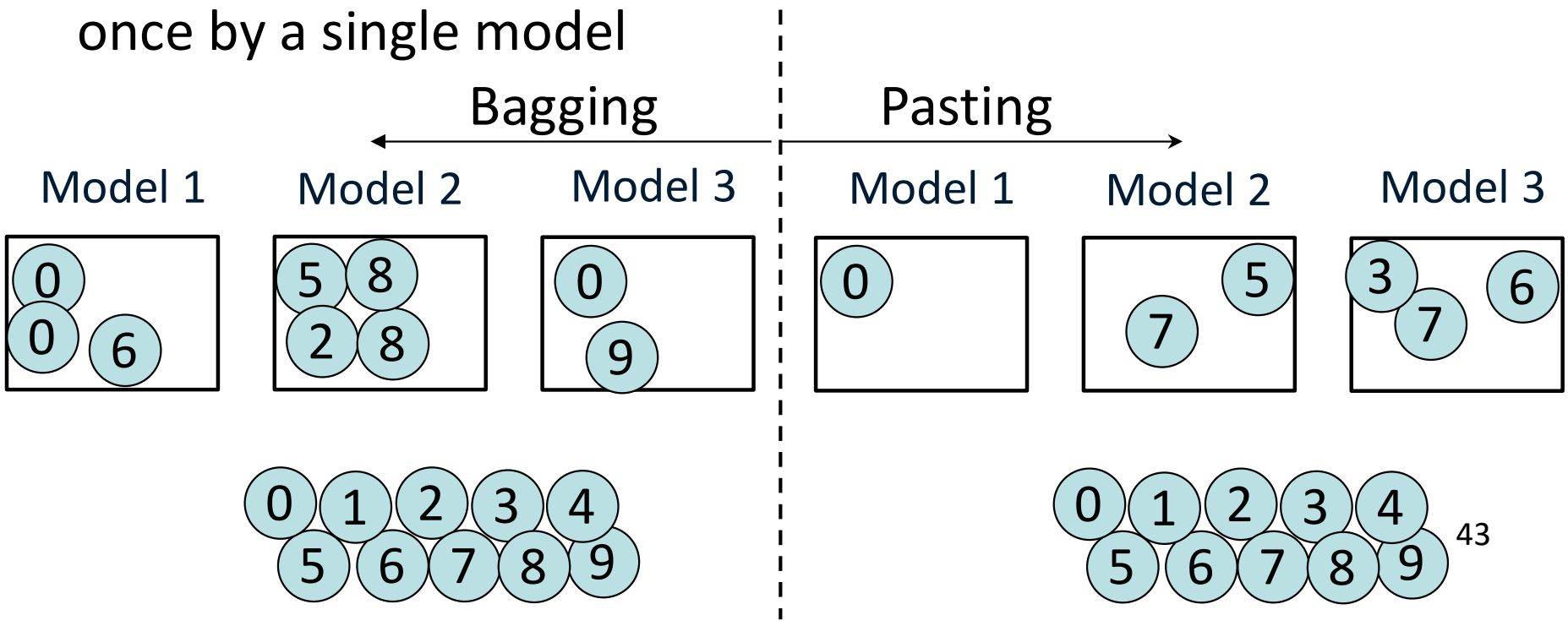
A. Use different algorithms (SVM, Decision Tree,…)

B. Use different subsets of data (Bagging, Pasting)

C. Apply both

Bagging versus Pasting

- In Bootstrap aggregating (Bagging), we sample with replacement
- In Pasting, we sample without replacement
- Bagging allows a training instance to be sampled more than once by a single model

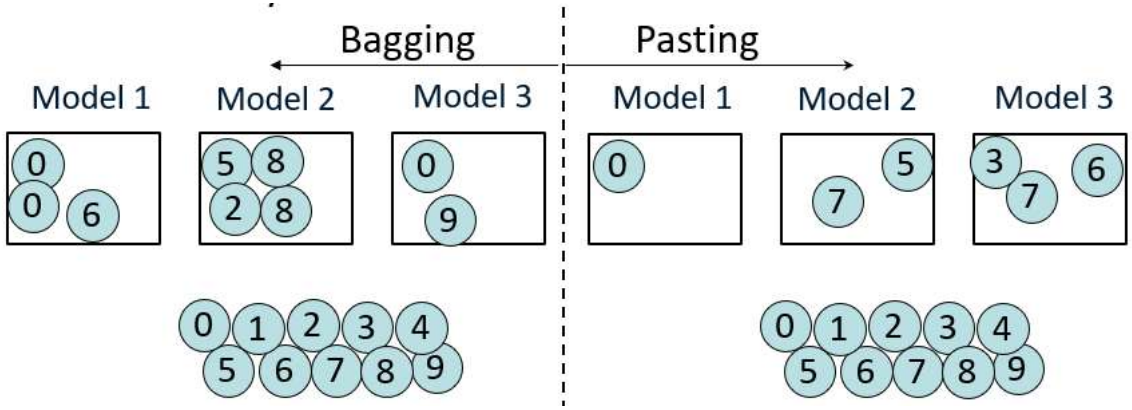


Aggregating the results

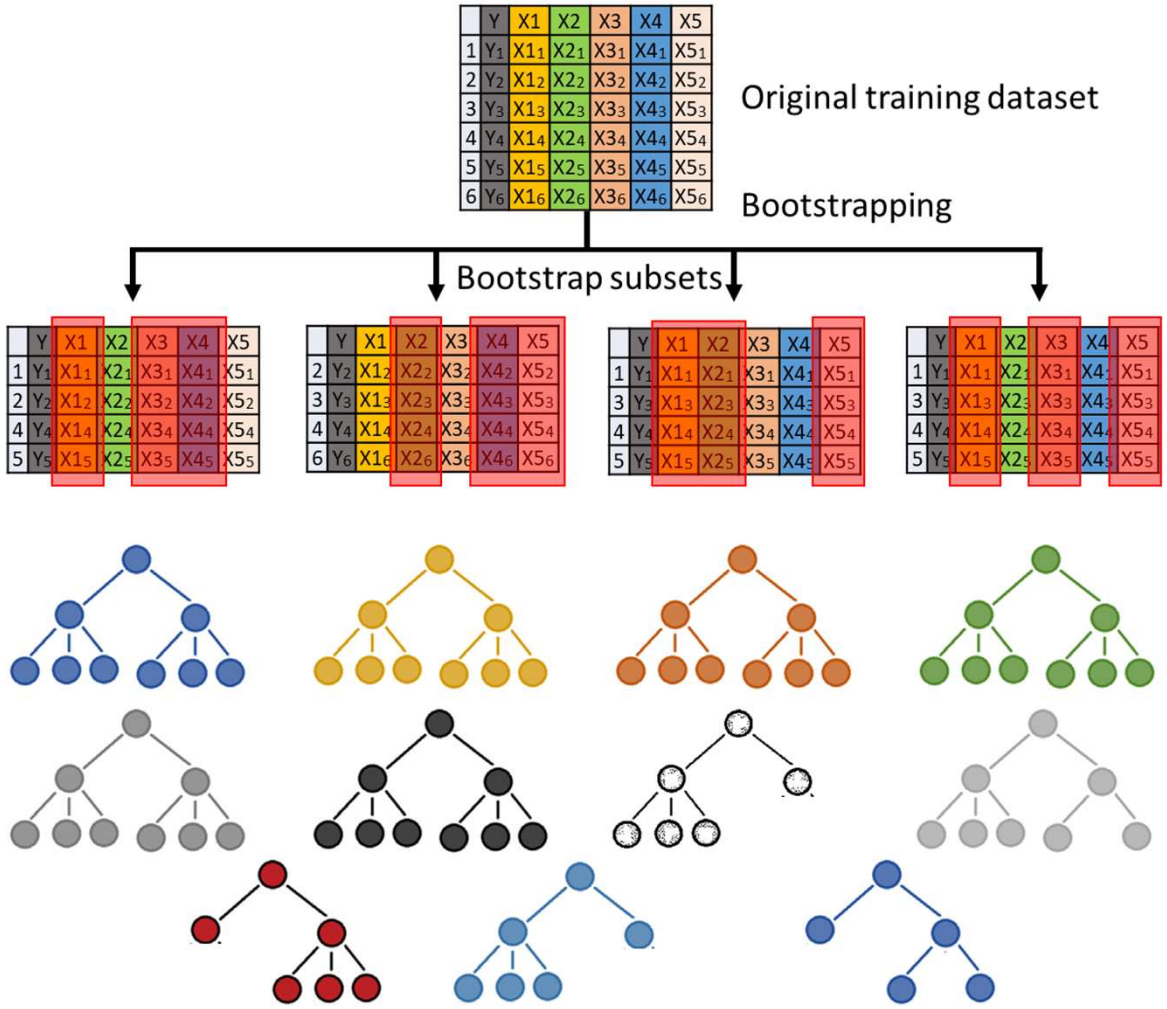
- Classification (usually) uses the most frequent answer
- Regression (usually) averages the results
- Aggregated results generalize better
- Again, remember that the ensemble model usually performs better than a single predictor (classifier or regressor)

Performance of bagging vs pasting

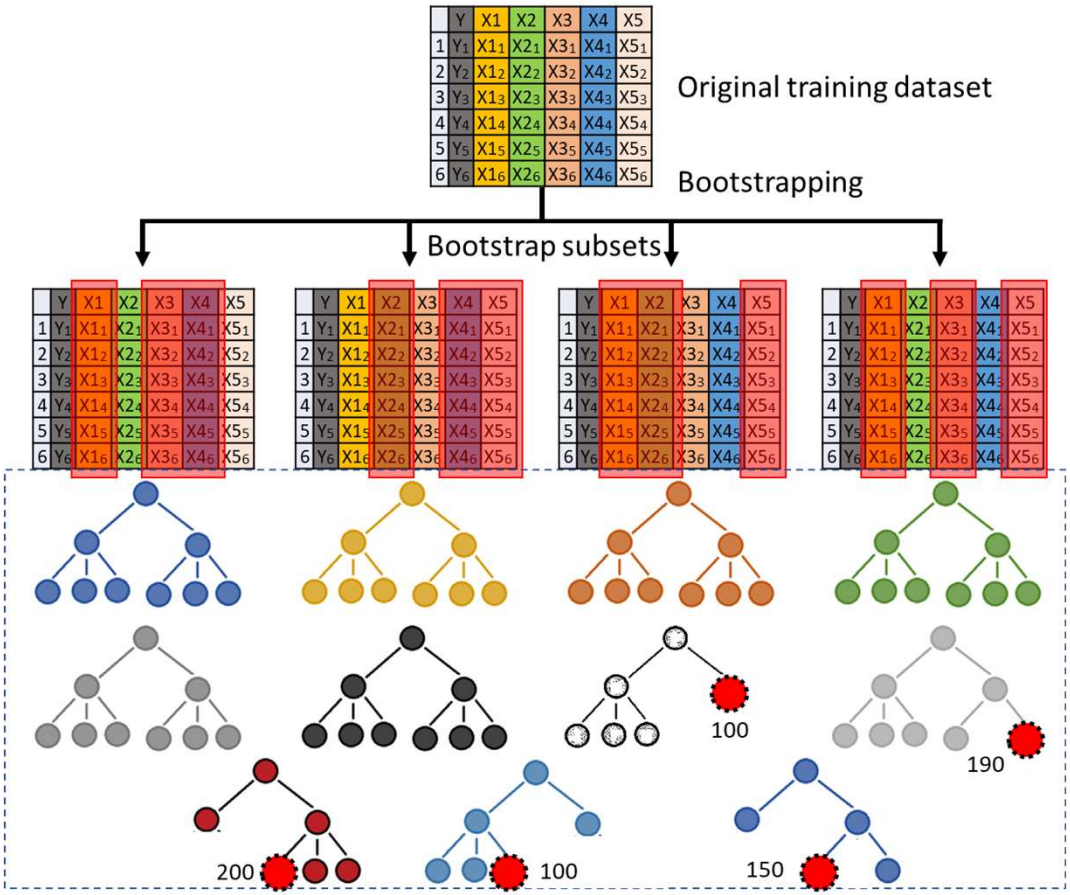
- Bagging is usually preferred in practice because it performs better with more diversity and lower variance
- Q1 (answer after reading the book): what are the effects of bagging and pasting on the ensemble bias?
- Q2 (answer after reading the book): which fraction of the training instances are not sampled for each predictor in bagging?



Random Forest is an ensemble of decision trees used for classification or regression



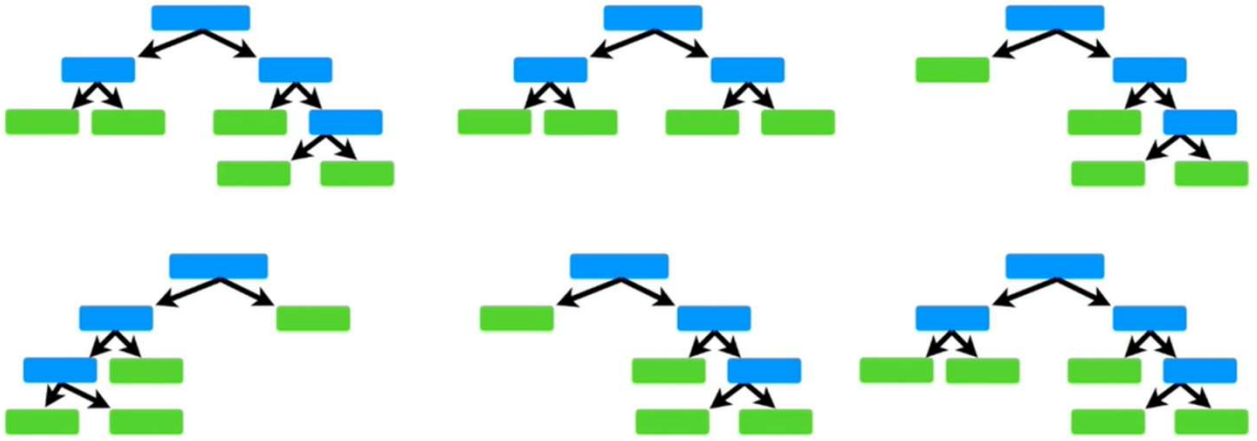
Example of random forest regression



- What is the predicted value in this example?

Random Forest vs Decision Tree Classification

- Random forest reduces variance with minimal increase in bias
- It is difficult to interpret random forest
- There is a lack of transparency in the random forest



Extra Trees

- Extra trees (or extremely randomized trees) is a supervised algorithm similar to random forest
- Extra trees algorithm uses the entire original sample (it is not based on bagging)
- Extra trees are used for classification and regression

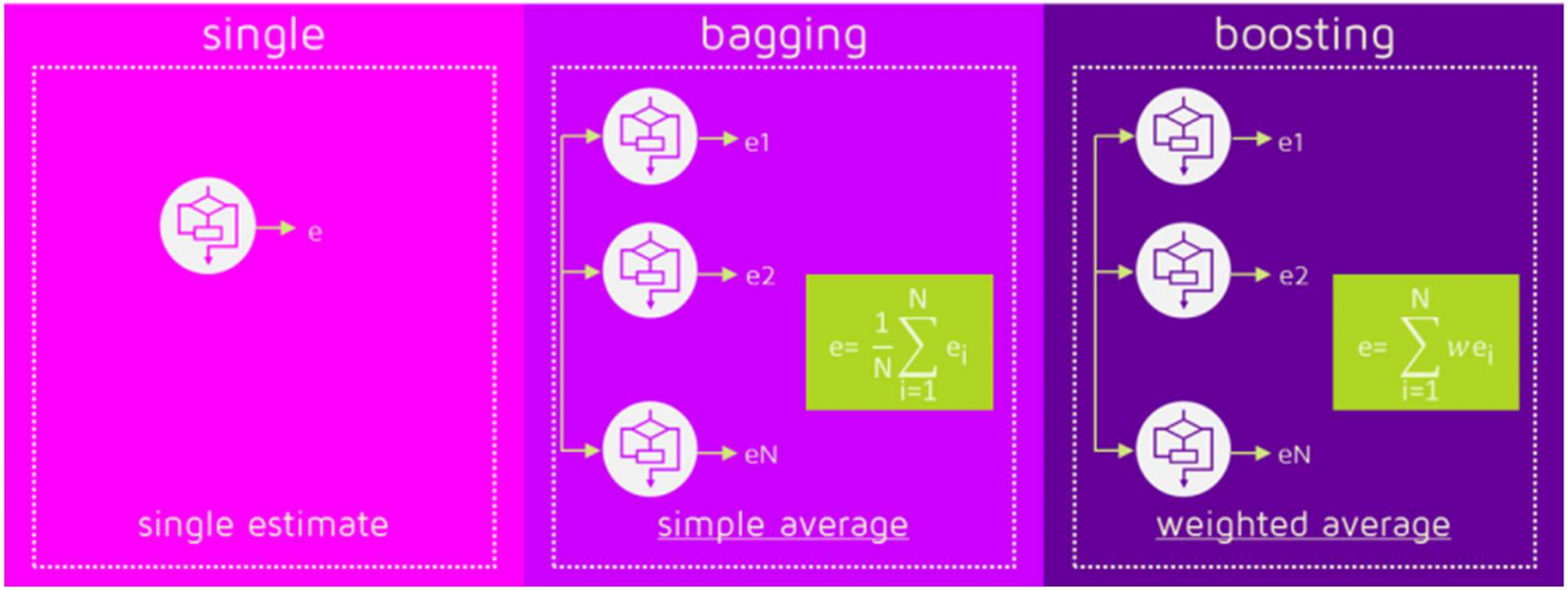
Main features of Decision Tree, Random forest, and Extra tree

Metrics	Decision tree	Random Forest (RF)	Extra trees (ET)
Number of trees	1	Many	Many
Bootstrapping	N/A	Yes	No (option for bootstrapping is available in scikit-learn)
# of features for the split at each decision	All features	Random subset of Features	Random subset of features
Splitting criteria	Best split	Best split	Random split

Boosting

- Boosting is a process where we combine several learners to create a stronger learner
- Adaptive Boosting (AdaBoost) improves the performance of a predecessor that underperforms
- Gradient Boosting minimizes a loss function by optimizing the weights (placing more weights on instances with erroneous predictions)

Gradient Boosting vs. Bagging



Gradient Boosting

- In gradient boosting, after building the weak learners, the predictions are compared with actual values
- The difference between prediction and actual values represents the error rate
- The error rate is used to calculate the gradient, which is the partial derivative of the loss function
- The gradient is used to find the direction that the model parameters would have to change to reduce the error in the next round of training.

Boosting Workflow

- Boosting applies multiple weak learners to build a stronger learner
  - A weak learner offers predictions slightly better than random selection
  - A weak learner builds a simple model with a high error rate, the model can be quite inaccurate, but moves in the correct direction
  - Calculate the error from the simple model
  - Fit another model to the error
  - Calculate the error from the addition of the first and second model
  - Repeat until the desired accuracy is obtained or some other stopping criteria

eXtreme Gradient Boosting (XGBoost)

- XGBoost is a more regularized form of Gradient Boosting
- XGBoost uses $\ell 1$ and $\ell 2$ regularization to improve model generalization and overfitting reduction
- XGBoost is usually faster than gradient boosting due to the parallelization
- XGBoost can handle missing values within a data set (data preparation is not as time-consuming)

When to use XGBoost?

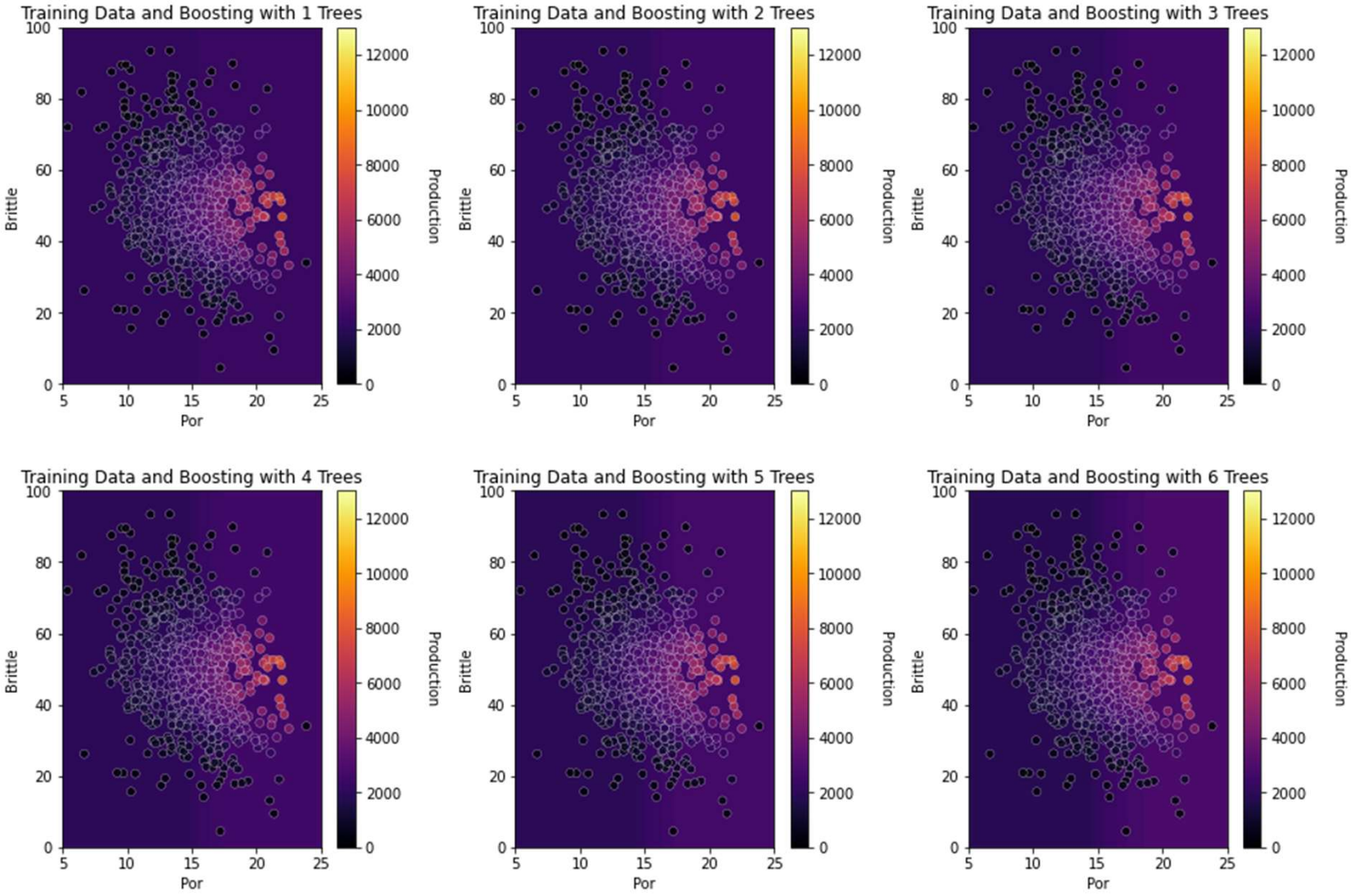
1. When there is a larger number of training samples: Ideally, more than 1000 samples and less than 100 features or we can say when the number of features < number of training samples.

2. When there is a mixture of categorical and numeric features or just numeric features.

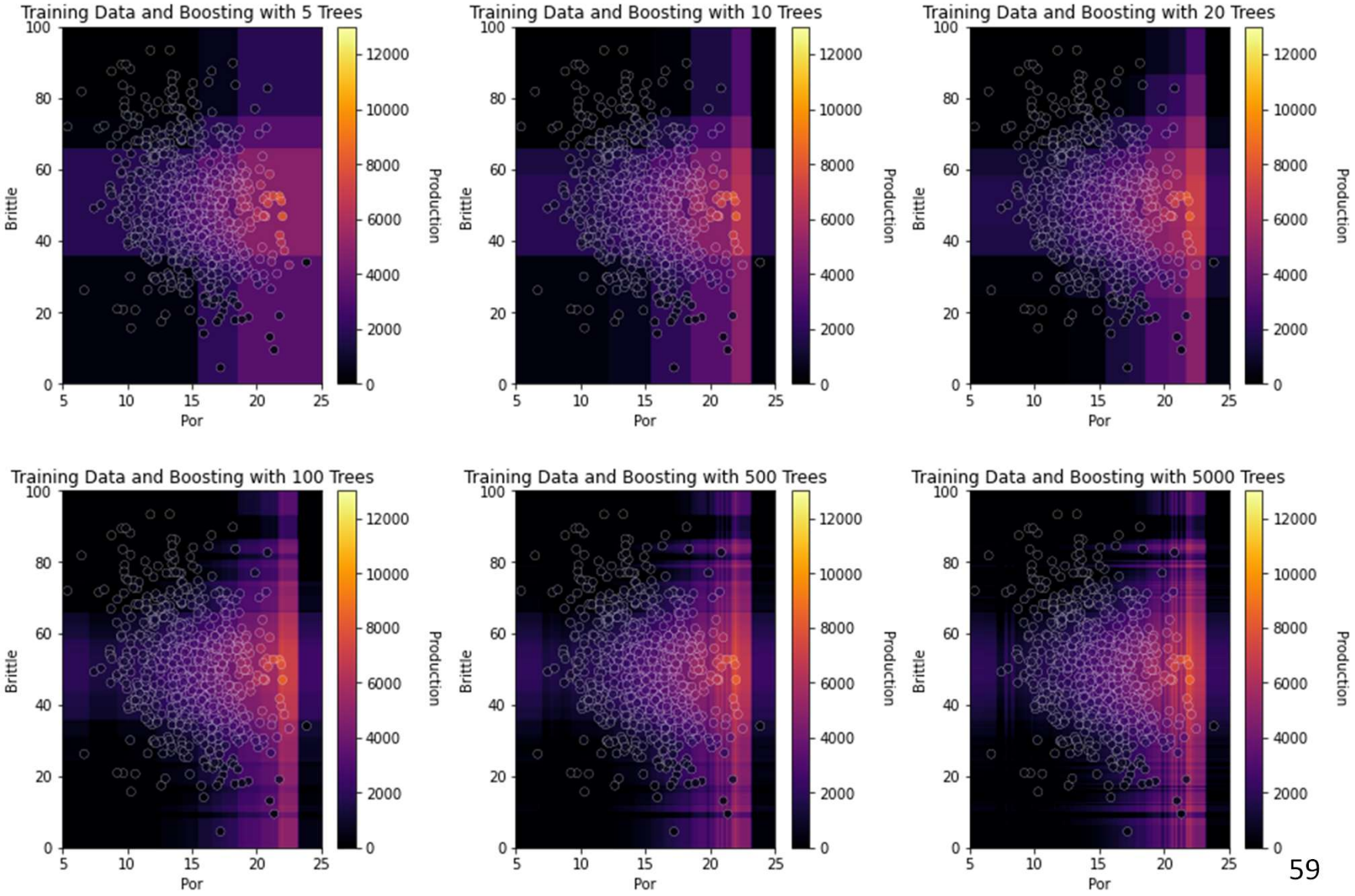
Code 2

- Run “GradientBoosting_and_XGBoost”

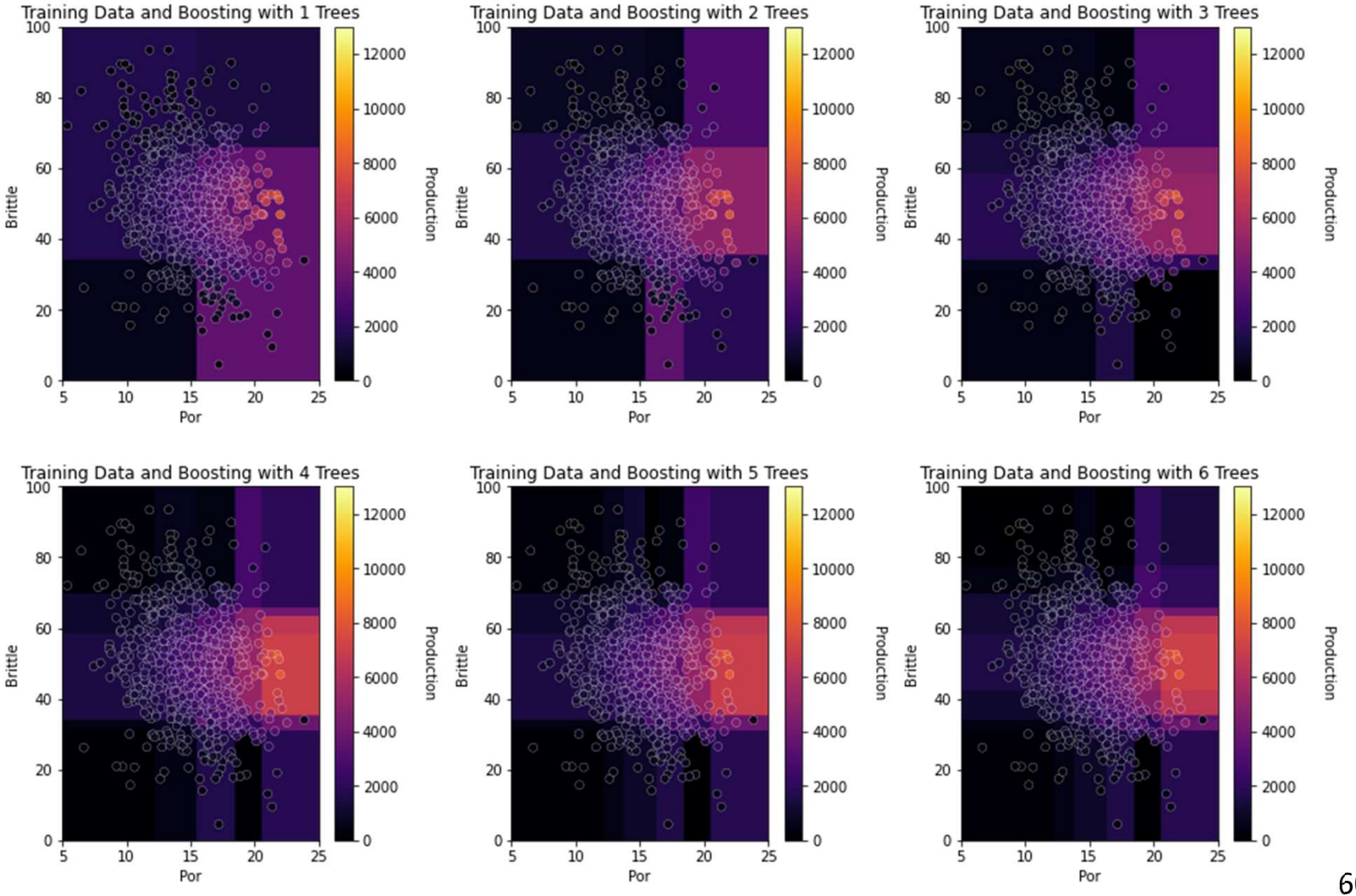
Significant misfit when we use up to 6 decision trees, **but each makes one decision**. **Each tree is just a "correction" to the previous one**. The combination of trees allows make more complex predictions. **Depth of each tree=1**



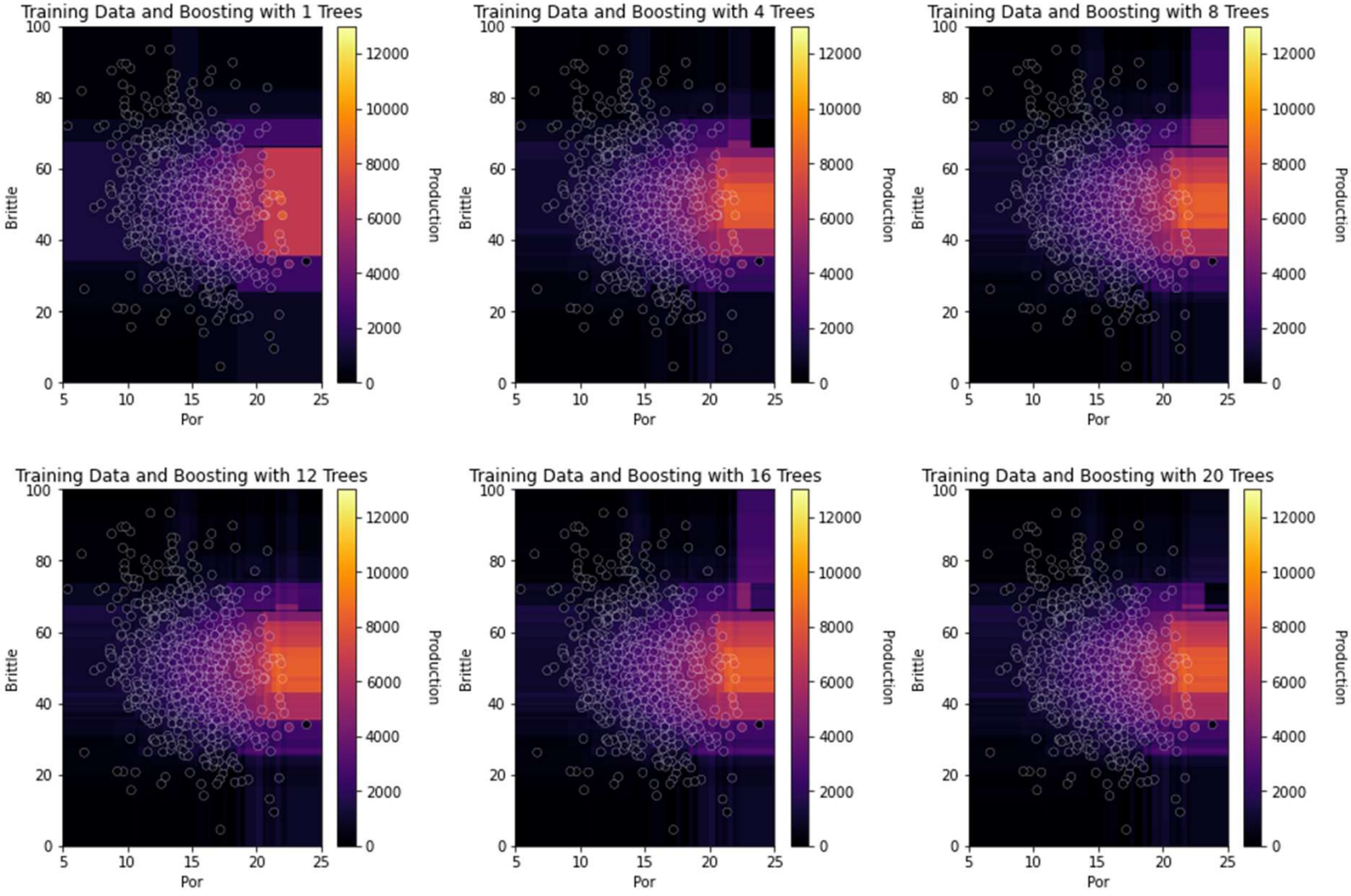
More trees but each has **depth = 1**



More patterns appear when **depth = 2** even with limited trees



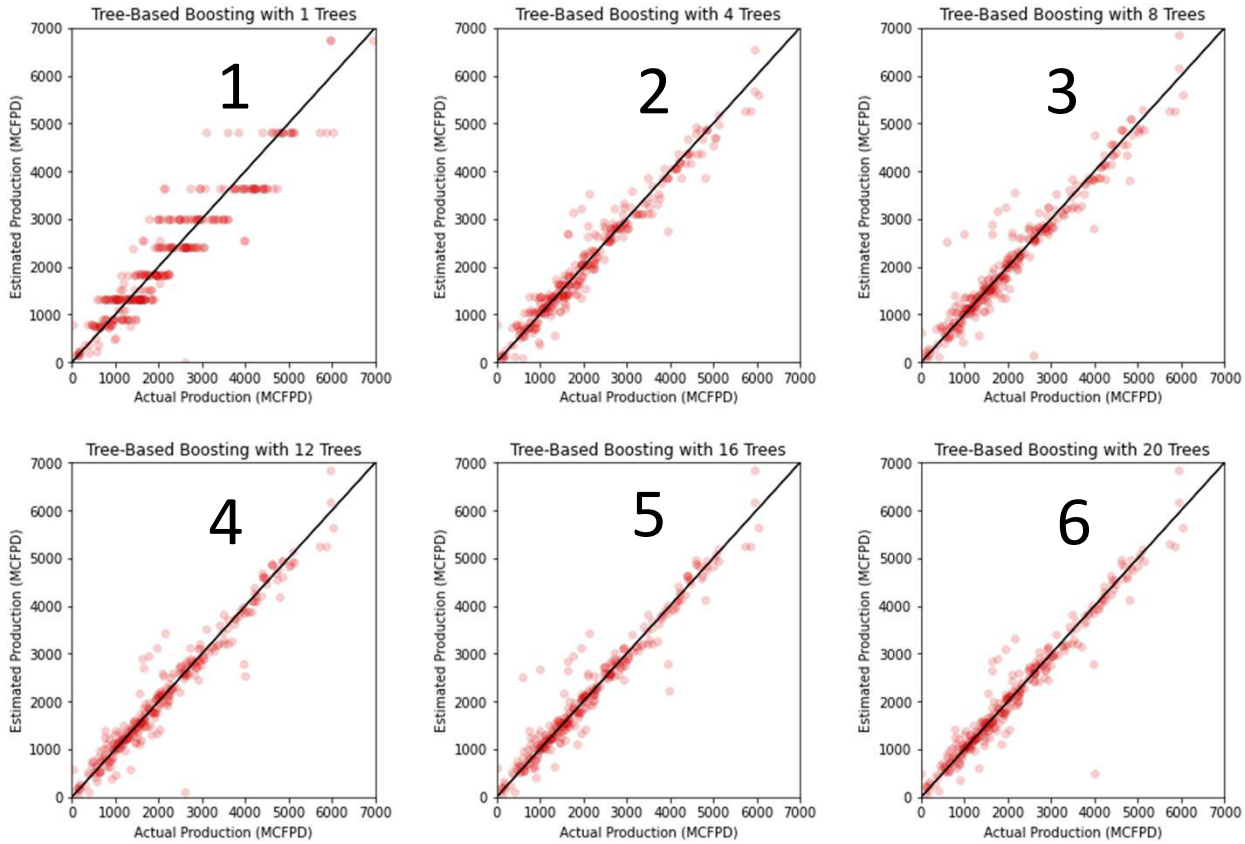
More trees with depth = 5



Finding the best solution

- The results look more interesting as we increase the number of trees and their depths
- We analyze the performance on test data to find the best scenario (cross validation)

Identify the best model



1, Mean Squared Error on Training = 345995.84 , Variance Explained = 0.85 Cor = 0.92

2, Mean Squared Error on Training = 209960.36 , Variance Explained = 0.91 Cor = 0.95

3, Mean Squared Error on Training = 188190.61 , Variance Explained = 0.92 Cor = 0.96

4, Mean Squared Error on Training = 164960.31 , Variance Explained = 0.93 Cor = 0.96

5, Mean Squared Error on Training = 166082.49 , Variance Explained = 0.93 Cor = 0.96

6, Mean Squared Error on Training = 176161.67 , Variance Explained = 0.92 Cor = 0.96

XGBoost

- TO-DO go through the rest of code and analyze the results

Data Analytics (PETR 6397)

Dimensionality Reduction

Dr. Ahmad Sakhaee-Pour

Petroleum Engineering Department

University of Houston

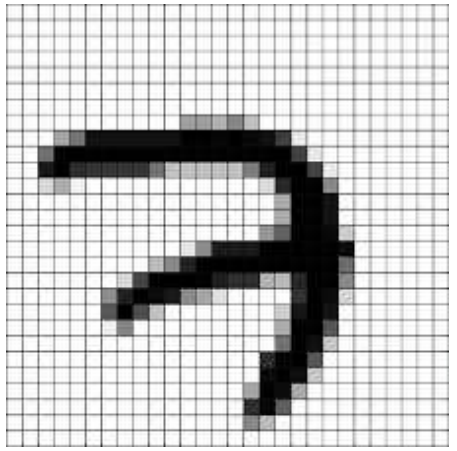
Spring 2025

Discussed topics

- Curse of dimensionality
- Dimensionality reduction
- Principal Component Analysis (PCA)
- Stochastic Neighbor learning (SNE)
- T-Distributed Stochastic Neighbor Embedding (t-SNE)

What is dimensionality

- Dimensionality is the number of dimensions (features) of a sample data

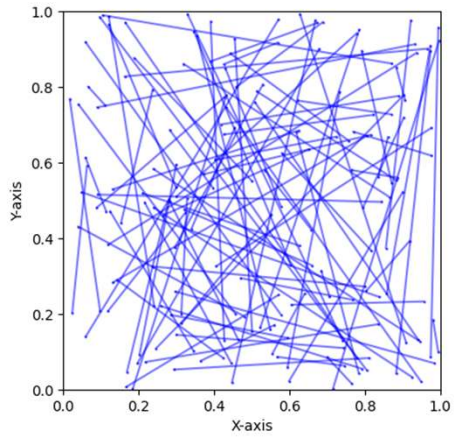


- What is the dimensionality of the MNIST image?

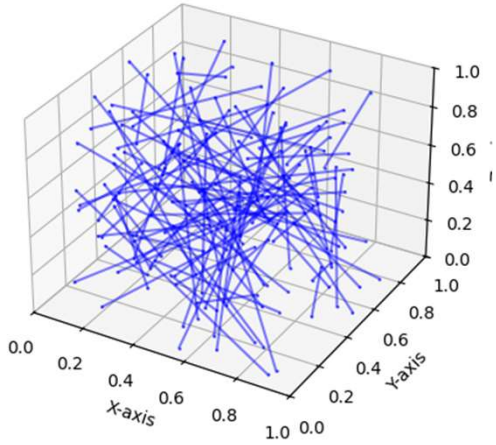
- What was the dimensionality of each rock sample in the previous homework?

Placing two points randomly in a unit cell

- Suppose we have a unit cell where we place two points randomly
- Determine the distance between the points
- Repeat this process and average the results



100 trials in 2d



100 trials in 3d

Average distance between the two points increases in higher dimensions

- We placed 10,000 points and calculated the average distance
  - 1d: 0.3364
  - 2d: 0.5203
  - 3d: 0.6657
  - 4d: 0.7760
  - 10d: 1.2667
  - 100d: 4.0734
  - 1000d: 12.9036

Curse of dimensionality

- Curse of dimensionality is a set of problems concerned with data in a higher-dimensional space:

  A. Data sparsity happens because data become sparser

  B. Visualization is more challenging

  C. Distance is not as informative as 2d or 3d

  …

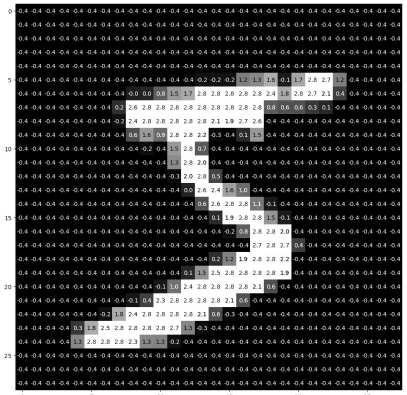
Data becomes sparser in higher dimensions

- Because of data sparsity, making valid correlations becomes more difficult

- We like to keep the useful information but minimize the negative effects of sparsity. This is the objective of dimensionality reduction

Benefits of the dimensionality reduction

- It helps the visualization by decreasing the number of dimensions (consider plotting your data in 2d or 3d versus 784d= 28x28 pixels)
- It may filter noise (many pixels are not important in MNIST data)
- It accelerates the process but does not necessarily improve performance because some information is removed
- Caution: Accelerating the training for low-dimensional data does not justify using the dimensionality reduction



Two applications of dimensionality reduction

1. Preprocessing and feature engineering: One of the standard methods of dimensionality reductions is the Principal Component Analysis (PCA), which extracts more important components

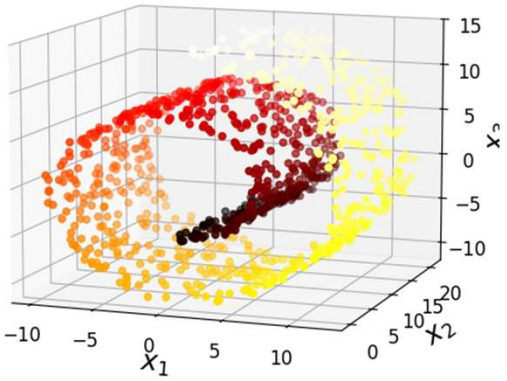
2. Generating plausible artificial datasets: PCA divides the dataset into components. We can then use this information to generate new samples (see the bonus homework problem)

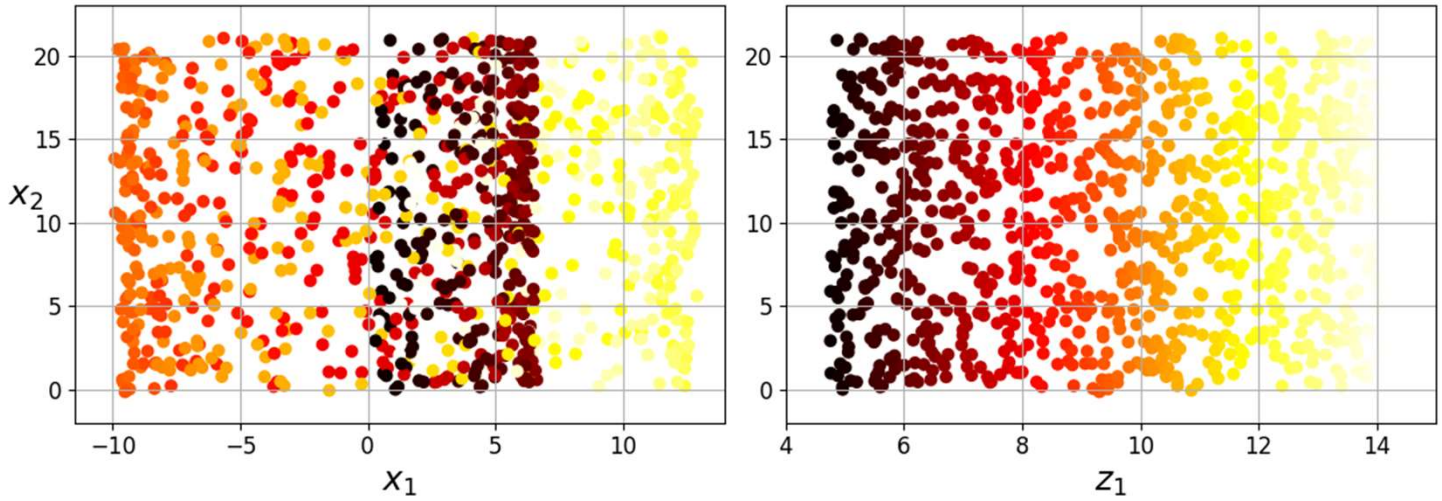
There are two main approaches for data reduction

- Projection: projects data from a higher-dimensional space to a lower-dimensional subspace to remove the sparsity
- Manifold learning: data exist near a much lower-dimensional manifold that is difficult to detect in a high-dimensional space
- Basically, we try to find the actual degree of freedom:
  - What is the degree of freedom of a straight line when it is not horizontal or vertical?
  - How about a non-curved plane?

Applicability depends on the dataset: Compare projection vs manifold learning

Swiss roll dataset



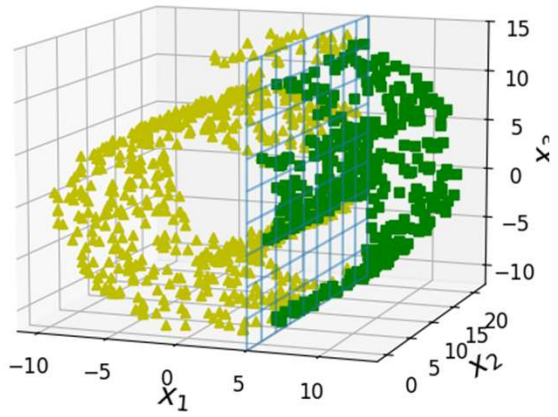


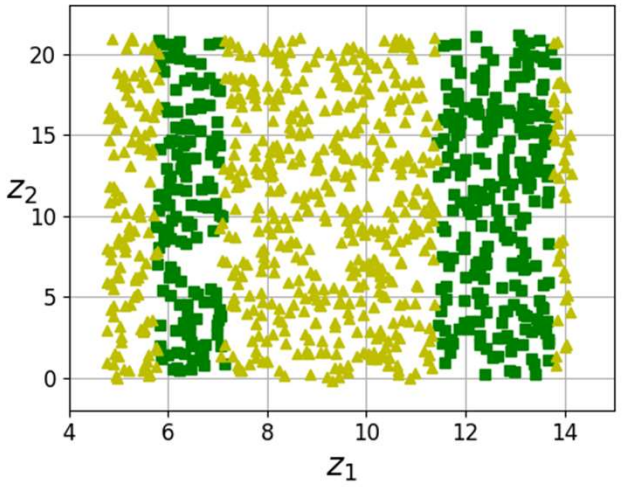
data projection

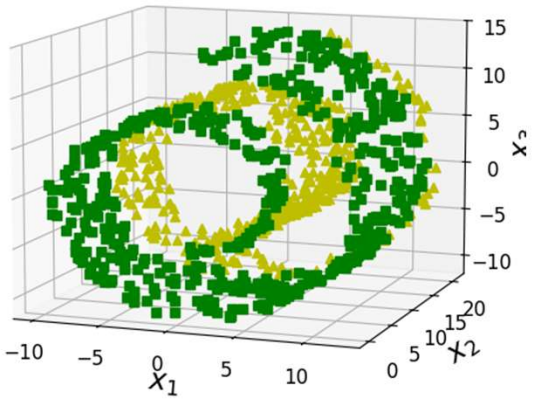
data unrolling
(manifold learning)

Fig. 8.5 (Reference)

Decision boundaries may become simpler or more complex depending on the data structure







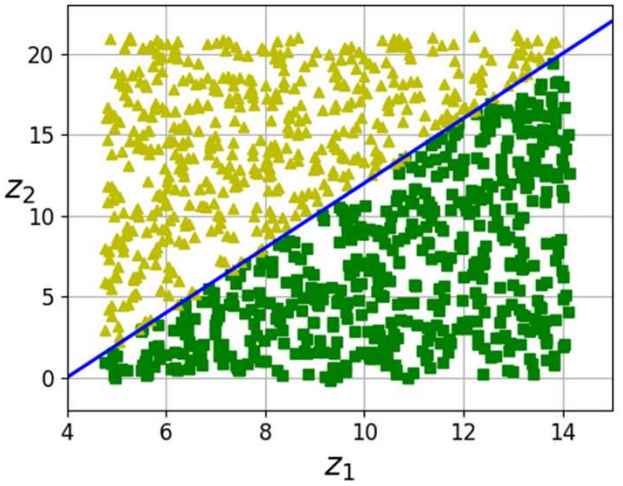
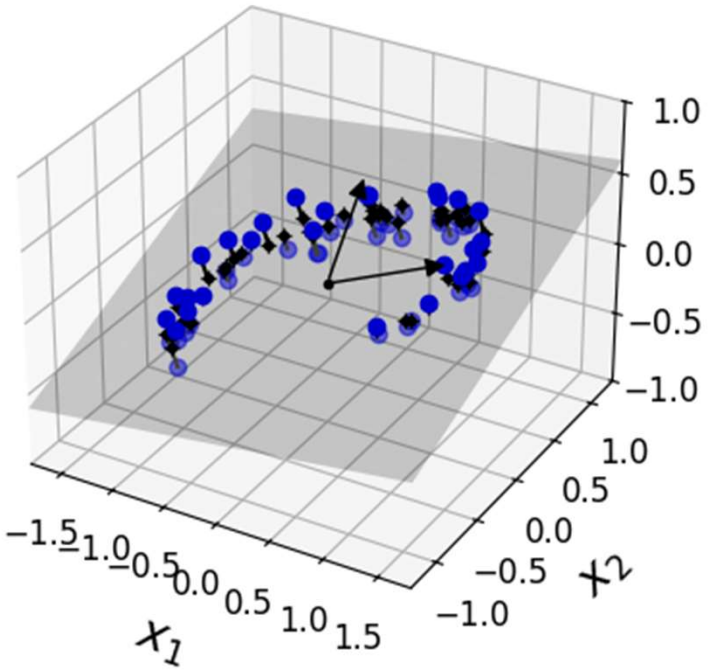
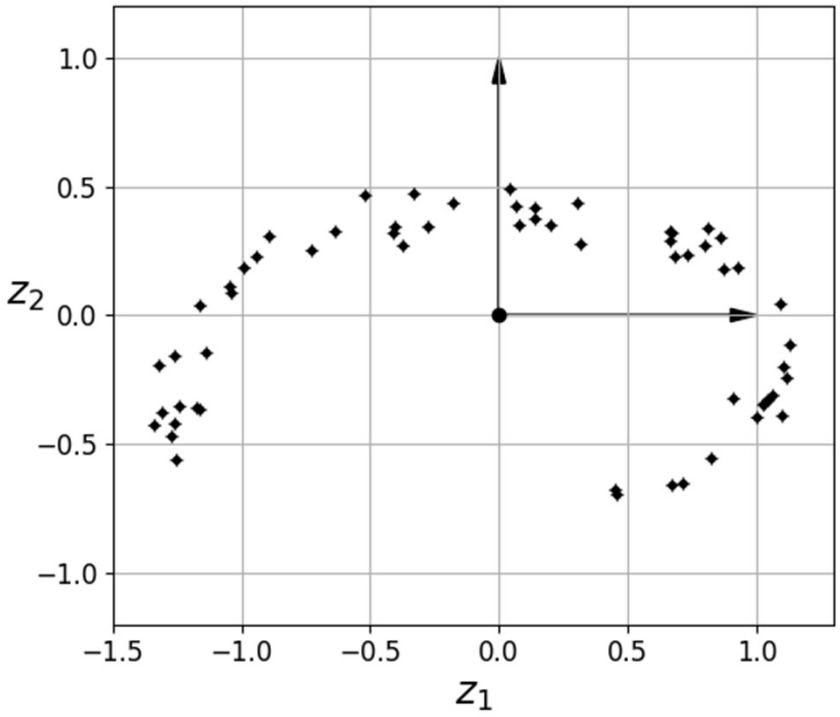


Fig. 8.6 (Reference)

Projection


1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
x_1
x_2


1.0
0.5
0.0
−0.5
−1.0
z_2
−1.5
−1.0
−0.5
0.0
0.5
1.0
z_1

Eigenvalue and eigenvector (reminder)

- Principal Component Analysis (PCA) is based on eigenvalue and eigenvector
- Eigenvalue (λ) and eigenvector (X) of a square matrix (A) are obtained from $A \cdot X = \lambda \cdot X$
- We first solve $\det(A - \lambda \cdot I) = 0$ to find the eigenvalues
- We then find the eigenvector for each eigenvalue
- Larger eigenvalues are more important and carry more information

Example of eigenvalue and eigenvector calculation

```
A = np.array([[3, 1, 4],
              [1, 5, 9],
              [2, 6, 5]])
```

Eigenvalues: [13.08576474 2.58000566 -2.66577041]

Eigenvectors: [[-0.31542644 -0.95117074 -0.32372474]
[-0.72306109 0.30781323 -0.70222933]
[-0.61456393 0.02291827 0.63409484]]

Note: eigenvectors are normal to each other

Q: Which eigenvalue is the least important?

PCA determines eigenvalues and eigenvectors using SVD

- Singular value decomposition (SVD)

$$A = U \times S \times V^T$$

- U contains the left singular vectors (columns of U)
- S is a diagonal matrix containing the singular values
- V contains the right singular vectors (columns of V)
- V^T: Is the transpose of V
- SVD applies to non-square matrix
- Its answer is different from the previous method

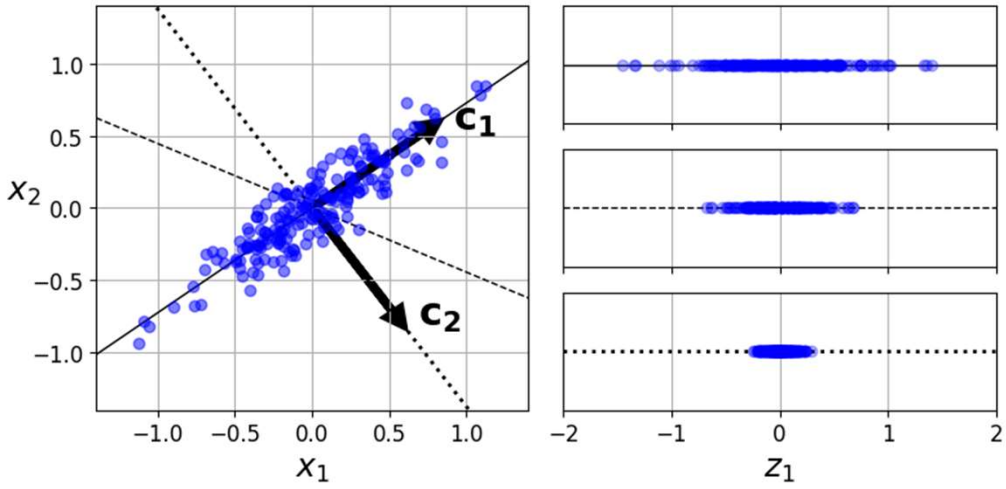
Example of SVD

```
A = np.array([[3, 1, 4],
              [1, 5, 9],
              [2, 6, 5]])
```

- U: [[-0.32463251 -0.79898436 -0.50619929]
  [-0.75307473 -0.1054674 0.64942672]
  [-0.57226932 0.59203093 -0.56745679]]
- S: [13.58235799 2.84547726 2.32869289]
- V: [[-0.21141476 -0.55392606 -0.80527617]
  [-0.46331722 0.78224635 -0.41644663]
  [-0.86060499 -0.28505536 0.42202191]]

Principal component analysis (PCA)

- PCA is the most popular dimensionality reduction
- The first component of PCA is called the principal component
- The first component accounts for the largest variance in data
- With each subsequent component, less information is contributed



Projecting down to *d* dimensions

- We can reduce the dimensionality to *d* by projecting the original data onto the hyperplane by the first *d* principal components:

$$\mathbf{X}_{\mathrm{d(projected)}} = \mathbf{X}\mathbf{W}_d$$

- $\mathbf{X}_{\mathrm{d(projected)}}$: Reduced dataset
- $\mathbf{X}$: Original data in a higher-dimensional space
- $\mathbf{W}_d$: Matrix containing the first *d* columns of the eigenvectors

Lost Information

- Explained ratio variance shows the proportion of variance captured by each principal component

```
A = np.array([[3, 1, 4],
              [1, 5, 9],
              [2, 6, 5]])
```

- U: [[-0.32463251 -0.79898436 -0.50619929]
  [-0.75307473 -0.1054674 0.64942672]
  [-0.57226932 0.59203093 -0.56745679]]
- S: [13.58235799 2.84547726 2.32869289]
- V: [[-0.21141476 -0.55392606 -0.80527617]
  [-0.46331722 0.78224635 -0.41644663]
  [-0.86060499 -0.28505536 0.42202191]]

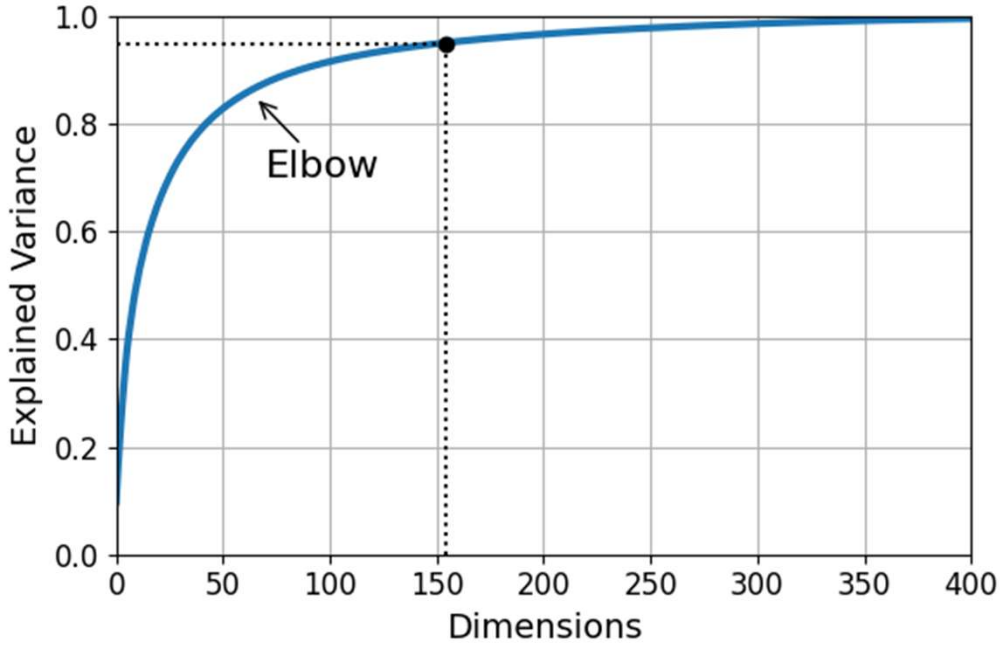
- `pca.explained_variance_ratio_`

[0.93171944 0.04089263 0.02738793]

- 2.73% of information is lost when we use the first two eigenvectors

The right number of *d* (lower dimension)

- Plot the explained variance versus the number of dimensions
- Pick the elbow (slope= 45 degrees)
- The following plot is based on SVD of MNIST data



Is it possible to recover the lost information

- Some information is lost when we project the data

$$\mathbf{X}_{\mathrm{d(projected)}} = \mathbf{X}\mathbf{W}_d$$

- Is it possible to recover the lost information by reversing the projection?

$$\mathbf{X}_{\mathrm{recovered}} = \mathbf{X}_{\mathrm{d(projected)}}\mathbf{W}_d^{\mathrm{T}}$$

- Let us check the next slide

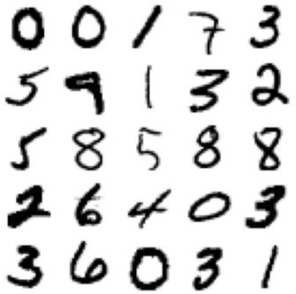
Determine whether we fully recovered the samples

- We applied PCA with a different number of components (d) but applied the reverse projection

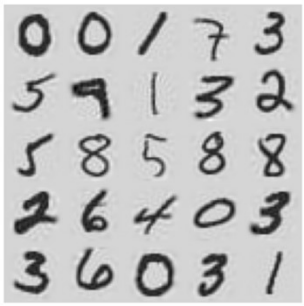
d=25

d=50

d=100



Original (n=784)



d=200

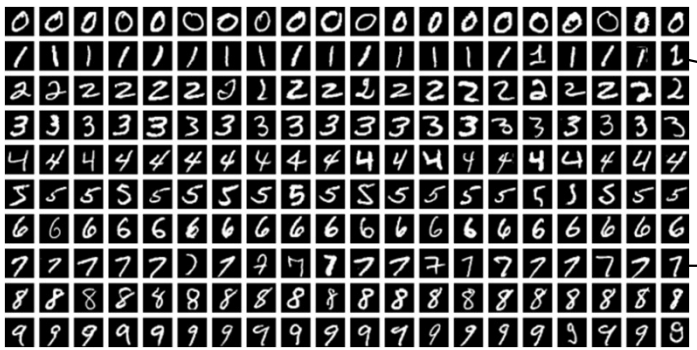
d=300

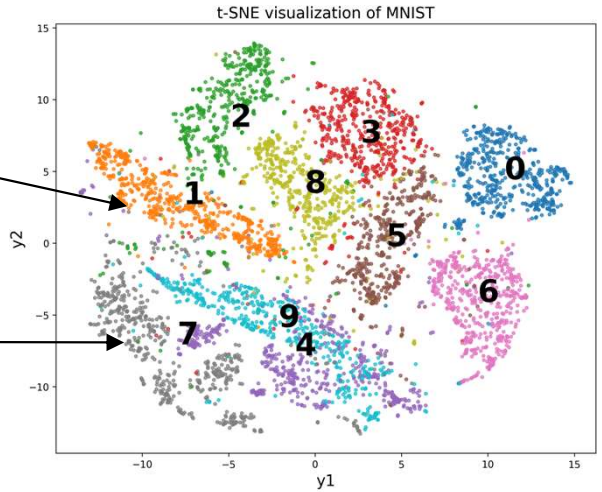
d=500

Manifold learning

Two methods of manifold learning

- First: Stochastic Neighbor Embedding (SNE)
- Second: t-Distributed Stochastic Neighbor Embedding (t-SNE)
- Both help interpret the underlying features of unlabeled data (unsupervised)
- Both are useful in handling high-dimensional data because represent them in 2d

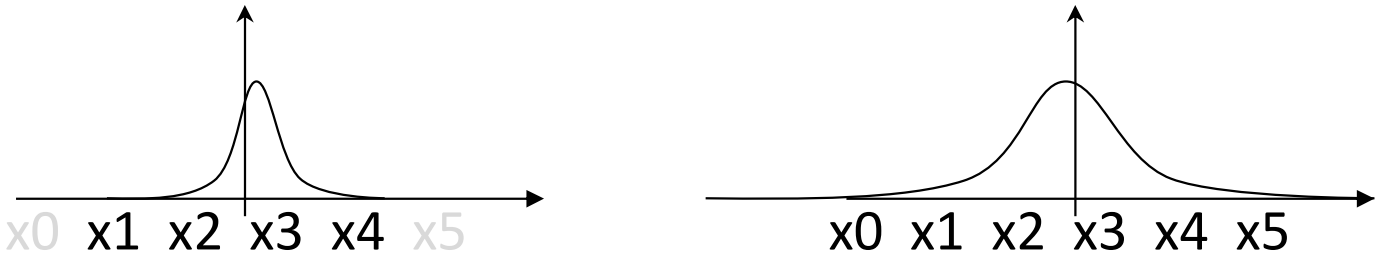




Stochastic Neighbor Embedding (SNE)

1. Converts the distance in a high-dimensional space into probabilities: It finds a Gaussian probability (pi,j) of two points (xi, xj) being neighbors. Closer points are more likely to be neighbors.

2. A wide probability picks distant points (x0, x5) as neighbors



3. It also finds the probability (qi,j) of the projected points (yi,yj) in a low-dimensional space

Main idea of SNE

- SNE minimizes the cost by reducing the difference between pi,j and qi,j using KL (Kullback-Leibler) divergence:

$$\text{cost} = \Sigma_{\text{i}}\Sigma_{\text{j}} p_{i|j} \log \frac{p_{i|j}}{q_{i|j}}$$

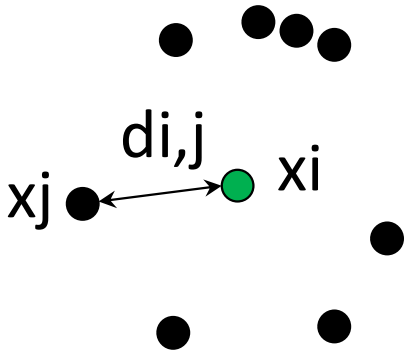
- We want neighbors (xi, xj) in a high-dimensional space to remain neighbors (yi,yj) in a low-dimensional space
- We tune the parameters of the probabilities (pi,j and qi,j)

Key properties of SNE

- SNE uses gradient descent to minimize the cost
- SNE asks us to specify the number of neighbors
- The number of neighbors is called **perplexity** in SNE (it is usually between 5 and 50)
- We should search over the tuning parameters to find the best answer (discussed later in this lecture)

Crowding problem in SNE

- Crowding problem happens when some points are equally distanced (di,j) from a specific point (xi)



- These equidistant points overwhelm SNE

- SNE struggles if we try to space neighboring points in a low-dimensional space

- t-SNE addresses the crowding problem (next slide)

t-Distributed Stochastic Neighbor Embedding (t-SNE)

- t-SNE addresses the crowding problem by using Student's t-distribution (Cauchy distribution) in a low-dimensional space

- The only difference between t-SNE and SNE is the probability distribution in a low-dimensional space

- Often, t-SNE outperforms in practice, but we can check both methods to find the best results

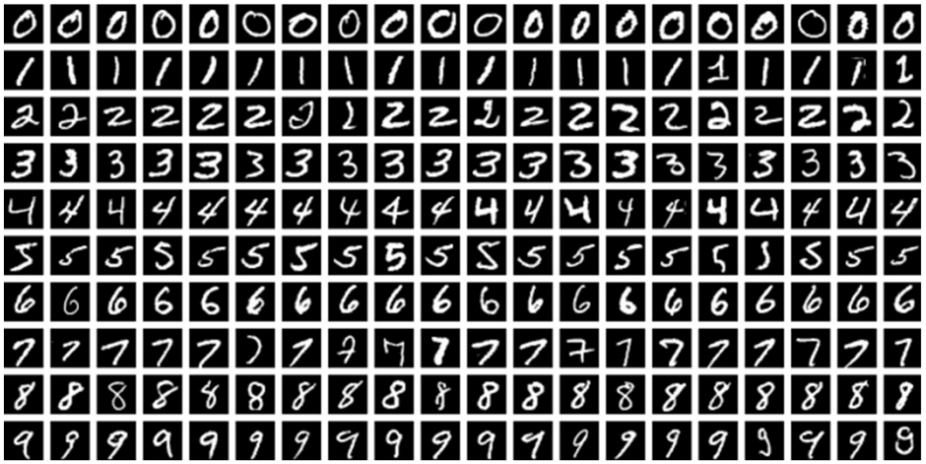
Implementation of t-SNE

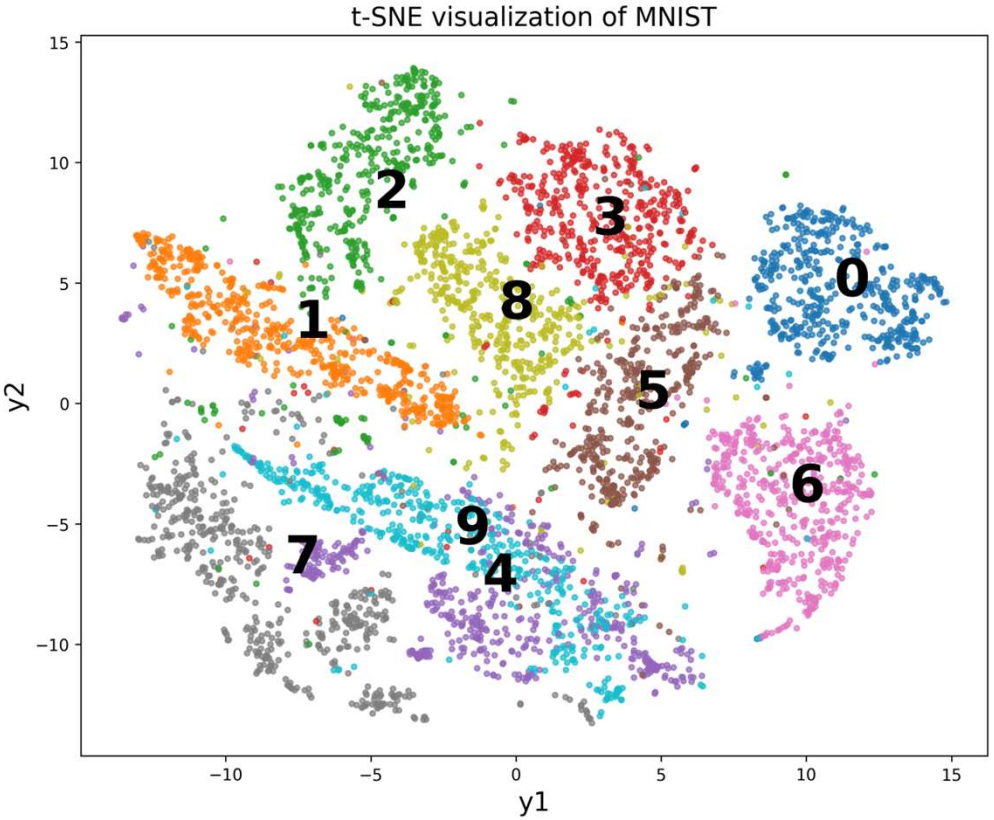
```
tsne = TSNE(n_components=2, perplexity=30, n_iter=300,
                    learning_rate=200)
```

- n_components shows the number of lower dimensions (2 or 3 is preferred for visualization)
- Perplexity is the number of neighbors, as discussed earlier
- n_iter is the maximum number of iterations
- learning_rate controls the step size (remember that T-SNE uses the gradient descent method)

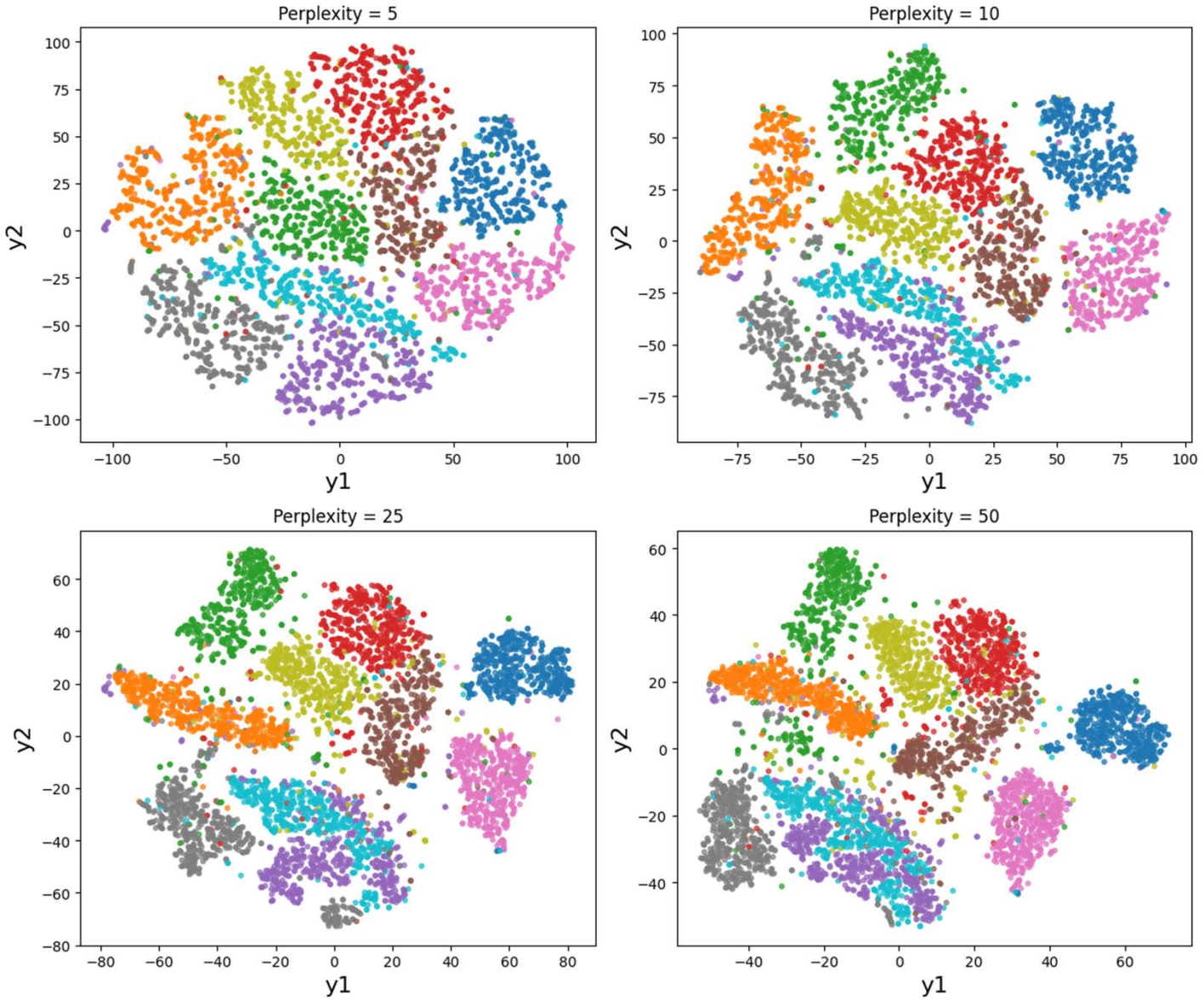
Analyzing 5000 handwritten digits of MNIST using t-SNE

- tsne = TSNE(n_components=2, perplexity=30, n_iter=300, learning_rate=200)

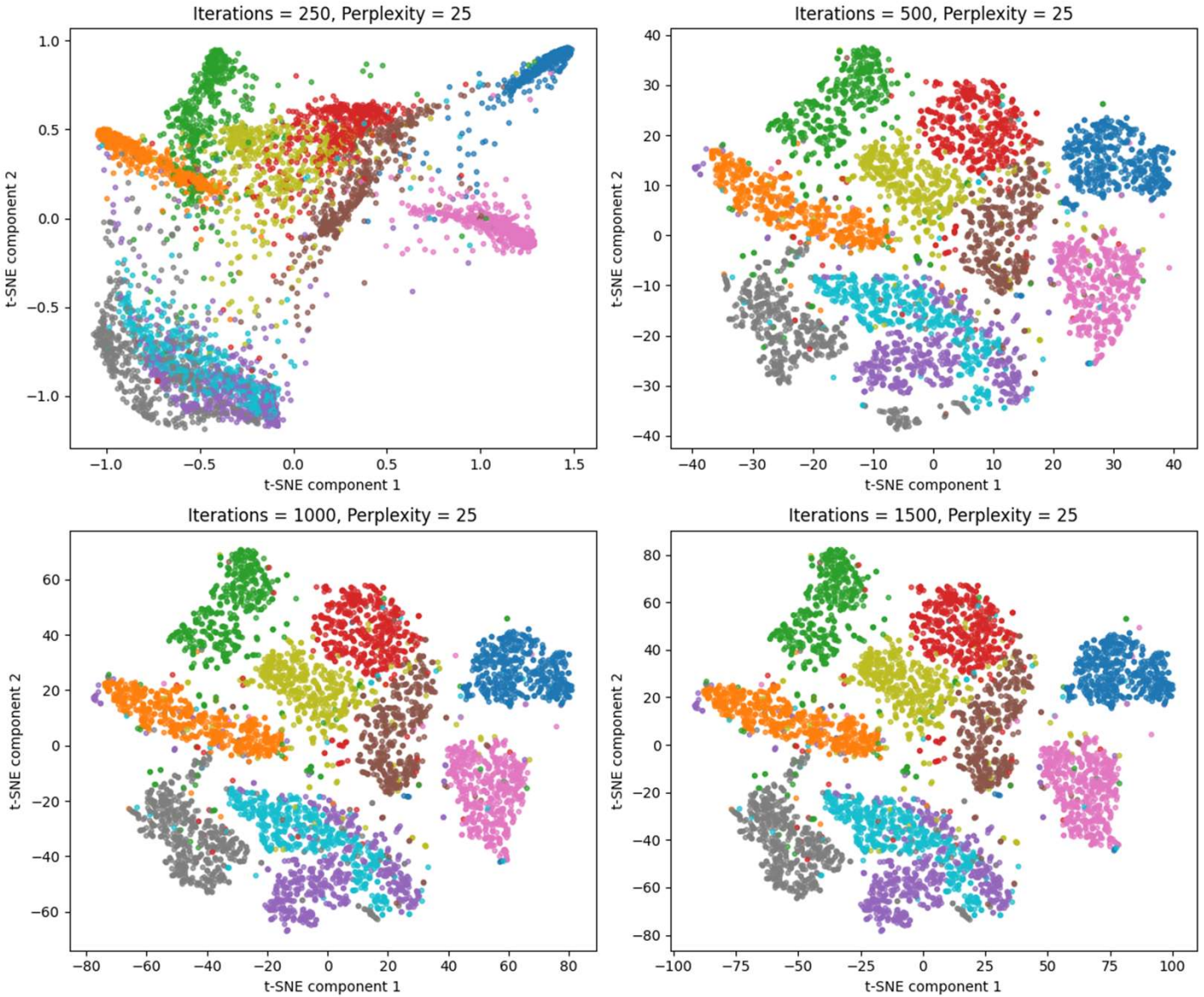




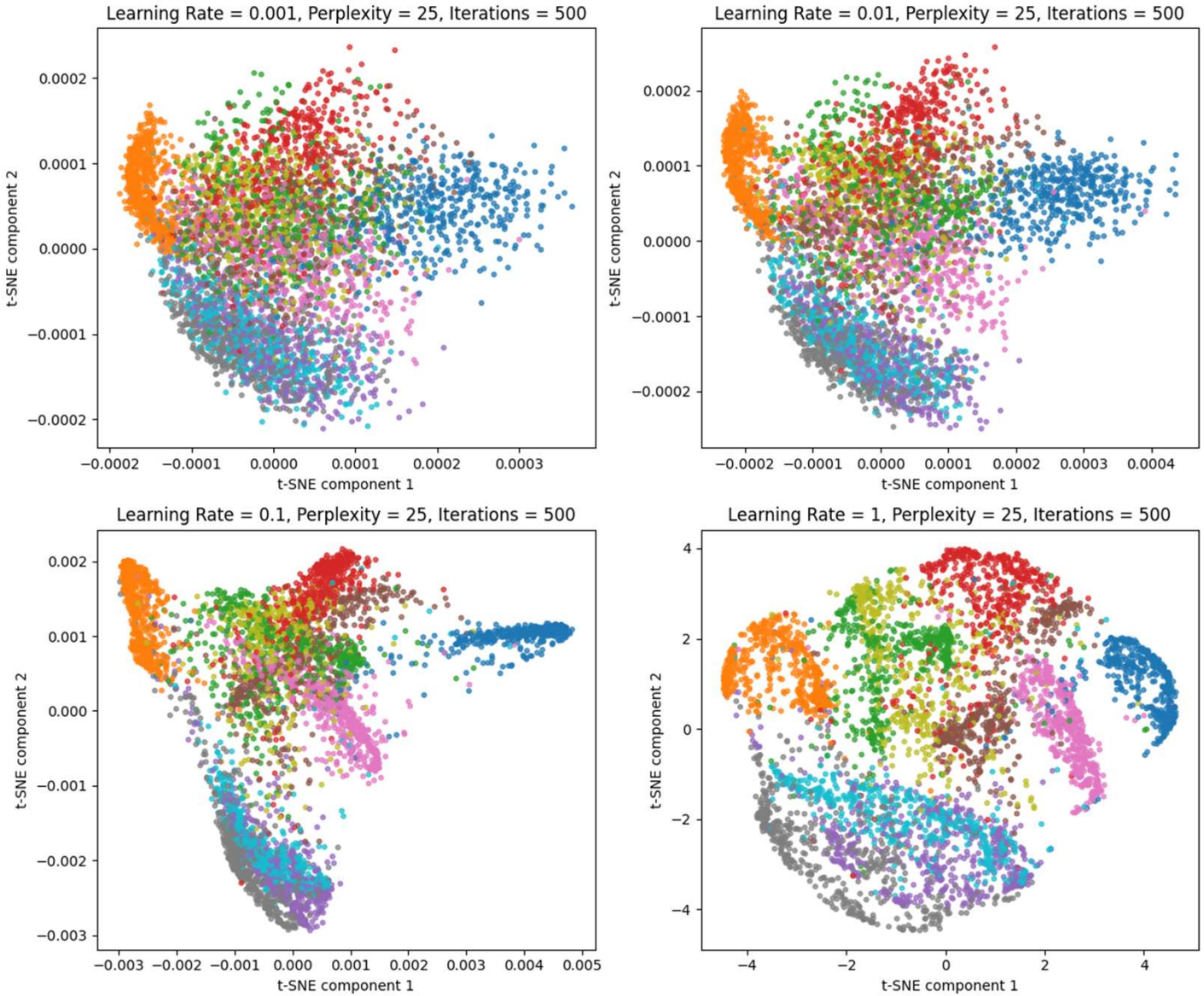
Effects of perplexity on t-SNE (choose the best performance)



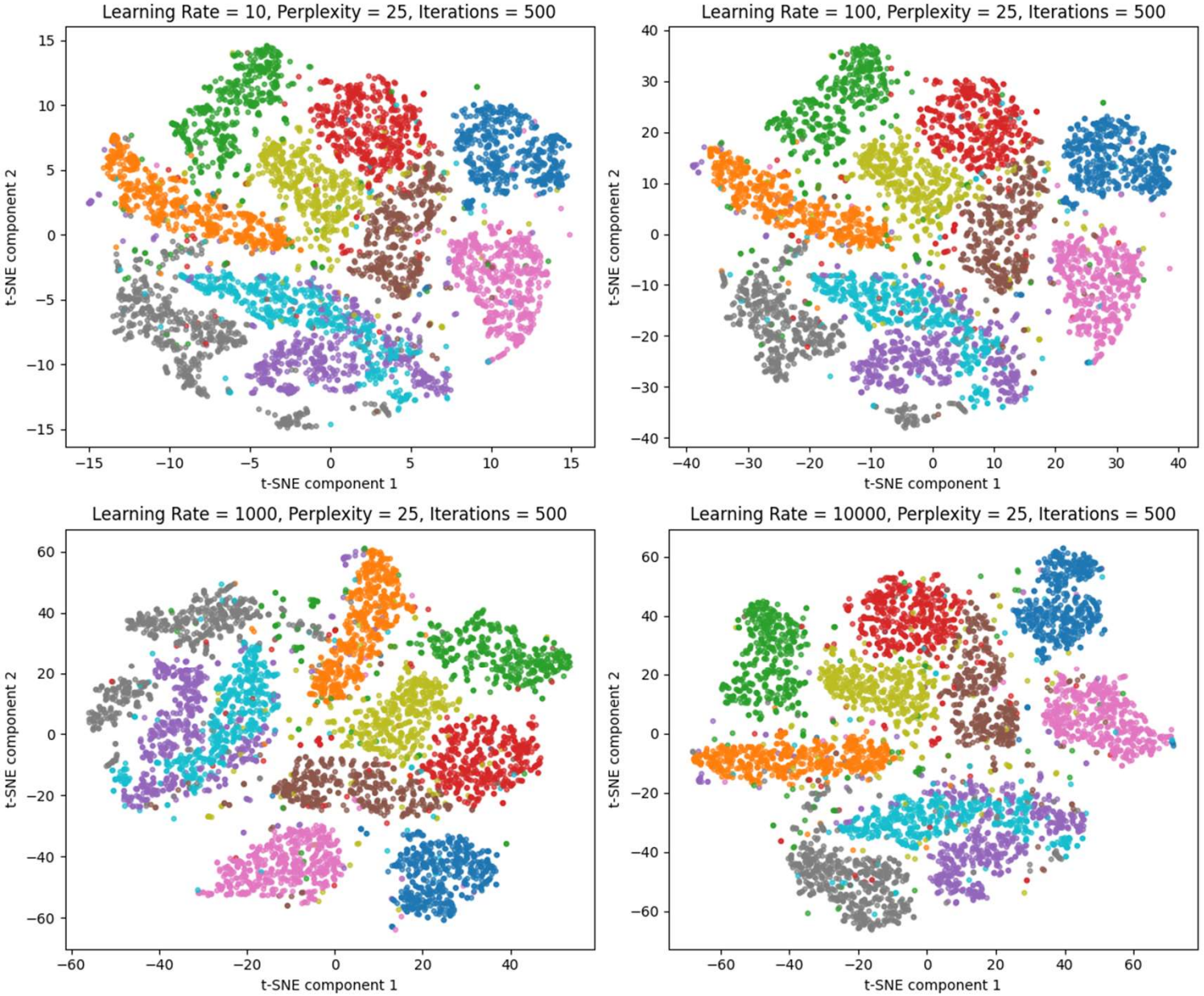
Effects of number of iterations on t-SNE (perplexity=25)



Effects of learning rate on t-SNE (perplexity=25, num_iteratations=500)



Effects of learning rate on t-SNE (continued)



Other methods of projection and manifold learning

- Random projection: briefly discussed next
- Locally linear embedding (LLE): briefly discussed next
- Multidimensional Scaling (MDS): check manifold.MDS

in scikit-learn

- Isomap Embedding: check manifold.Isomap in scikit-learn
- Linear discriminant analysis (LDA): check LinearDiscriminantAnalysis in scikit-learn

Random projection

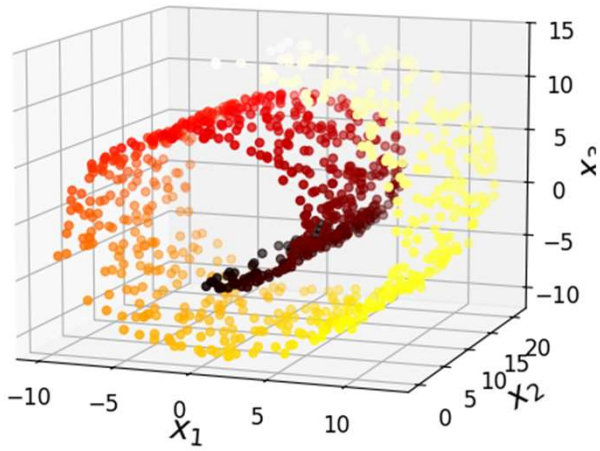
- It projects data using a **random** linear operation
- It uses a matrix P filled by a Gaussian distribution (mean=0, variance=1)
- Shape of P=[d,n]
- *d*: number of target dimensions

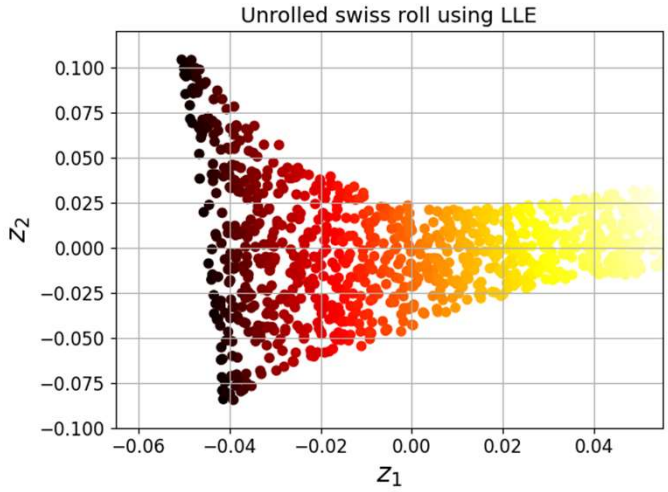
$$\mathrm{d} \geq 4\log(m)/(\frac{\varepsilon^2}{2} - \frac{\varepsilon^3}{3})$$

- *n*: number of original dimensions
- *m*: number of samples (instances)
- ε: tuning parameter (usually smaller than 10%)

Locally linear embedding (LLE)

- This is a nonlinear method
- It is a manifold learning technique
- It works well when noise is limited
- It finds the k-nearest neighbors of the data
- It does not preserve the global distances





Popup questions

- PCA is used for feature selection (True or False)
- PCA is used for visualization (True or False)
- T-SNE is used for feature engineering (True or False)
- T-SNE is used for feature selection (True or False)

Data Analytics (PETR 6397)

Clustering

Dr. Ahmad Sakhaee-Pour

Petroleum Engineering Department

University of Houston

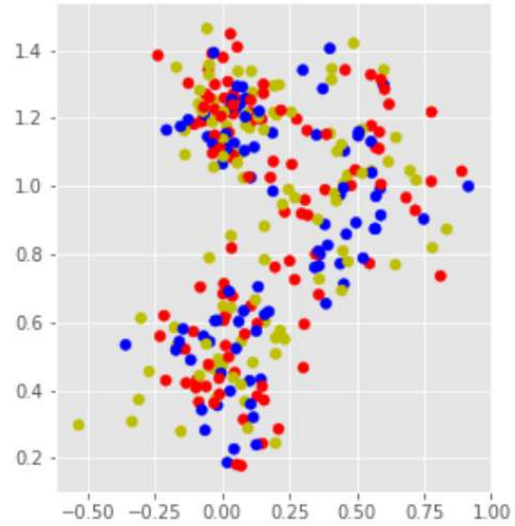
Spring 2025

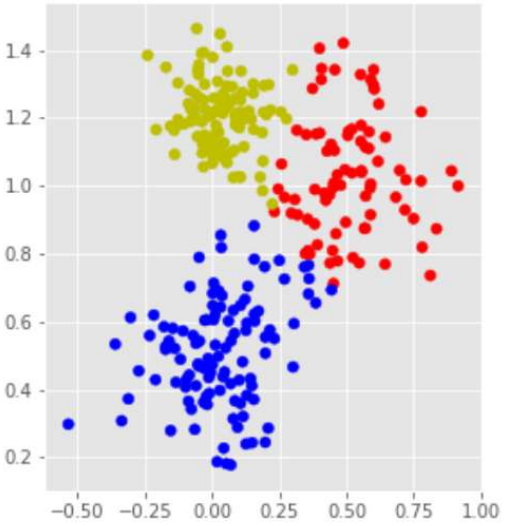
Topics

- Unsupervised learning
- Unsupervised tasks
- Clustering:
  - *K*-mean clustering
  - Gaussian mixture model (GMM)
  - Hierarchical clustering
  - Density-based spatial clustering of applications with noise (DBSCAN)

Unsupervised learning

- Unsupervised model groups similar points together
- Similarity is based on distance, density, or other statistical measures
- Unsupervised learning is used for exploratory data analysis (EDA) to find main characteristics and draw inferences without labels





Unsupervised tasks (1)

- Clustering: Groups samples based on some assumptions (discussed later)
  - –Example: customer segmentation, recommender system, search engine, and image segmentation

- Anomaly detection (outlier detection): The objective is to determine the “normal” data
  - –Example: fraud detection, detecting defective products

Unsupervised tasks (2)

- Dimensionality Reduction: Reduces number of variables by finding principal variables
  - Example: Improving *J* function representation (bonus homework)

- Density estimation: Obtains the probability density function (PDF) of how most data are distributed
  - Example: heights of people based on their gender and nationality

Clustering vs. classification

- Similar to classification, clustering groups samples based on the similarity

- In contrast with classification, clustering does not have access to the correct labels

Non-exhaustive list of applications of clustering

- Customer segmentation: Groups customers based on their purchases and activities on the internet

- Feature engineering: The cluster affinity is useful as an extra feature(s). They improve the performance

- Image segmentation: Clusters pixels based on their colors (examples discussed in detail later)

Definition of cluster

• We can group data differently; there is no universal definition of a cluster

Common criteria

1. Gathering around a particular point (centroid)

2. Continuous regions of densely populated (cluster shape has more freedom than the previous one)

3. Hierarchical structures (clusters of clusters)

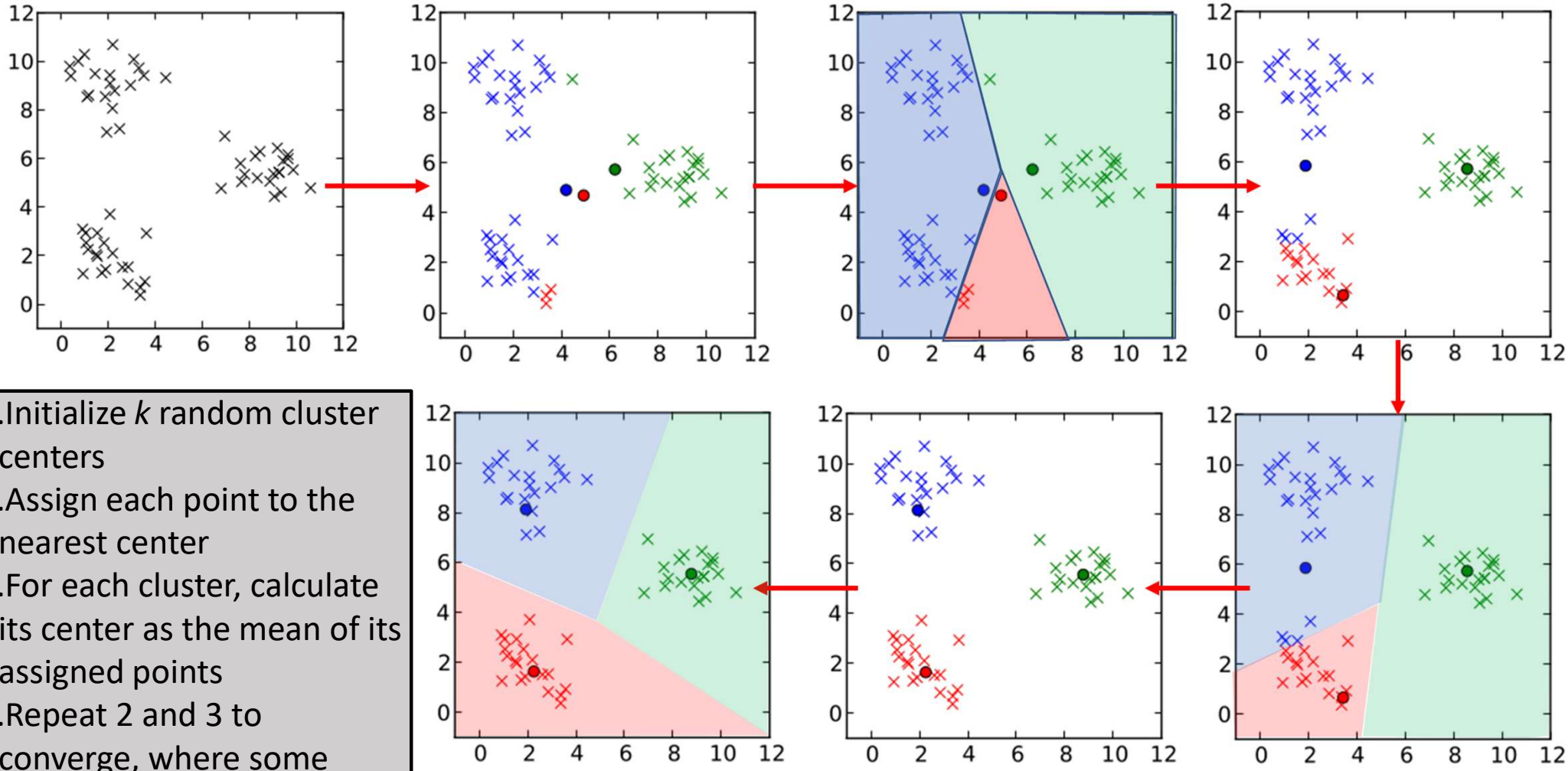
Clustering algorithms

K-means clustering

- This is the simplest and widely used clustering
- It divides data into different (disjoint) clusters, where each instance belongs to the cluster with the nearest mean

Pseudocode:

1. Initialize *k* random cluster centers
2. Assign each point to the nearest center
3. For each cluster, calculate its center as the mean of its assigned points
4. Repeat 2 and 3 to converge, where some criterion is satisfied



1. Initialize *k* random cluster centers
2. Assign each point to the nearest center
3. For each cluster, calculate its center as the mean of its assigned points
4. Repeat 2 and 3 to converge, where some criterion is satisfied

Main characteristics of *k*-means clustering

- We have to define the number of clusters (*k*)

- *K*-means clustering converges quickly (it is one of the fastest clustering techniques)

- *K*-means clustering may not yield the best solution because we assigned the initial centroids randomly

- Potential solution: Run it a few times and choose the best outcome

Hard clustering vs. soft clustering

- Each instance has a label in *k*-means clustering (hard clustering)
- The label is NOT the class label (remember that clustering is not classification)
- Shortcoming: *K*-means clustering does not behave well when clusters have different diameters because it analyzes the distance of an instance from the centroid
- Potential solution: We may assign each instance a score per cluster (this is soft clustering)

Extensions of *k*-means clustering

- Regular *k*-means clustering initializes the clusters randomly and iterates using all instances to determine the clusters

Extended versions:

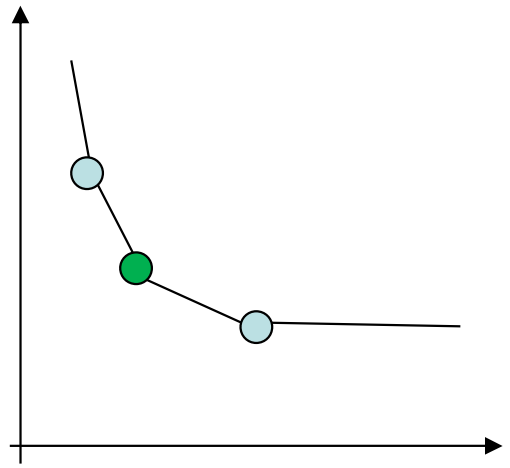
1. Smarter calculations of distances: Avoid unnecessary calculations of distances (2003) based on the triangle inequality
2. Different initialization: Use a smarter initialization, which is the basis of *k*-means++ (2006), rather than random
3. Mini-batch *k*-means: Use mini-batches of data (2010)

Evaluating the performance of clustering

Elbow method for choosing the best number of clusters

1. Calculate Within-Cluster Sum of Squares (WCSS)

$$\text{WCSS} = \sum_i \sum_j \text{distance}\left(x_j, c_i\right)^2$$



x_j: instance j

c_i: centroid i

2. Find the cluster number where it is similar to an elbow (minus 45-degree tangent)

Q: What is WCSS when we assign every instance its own cluster?

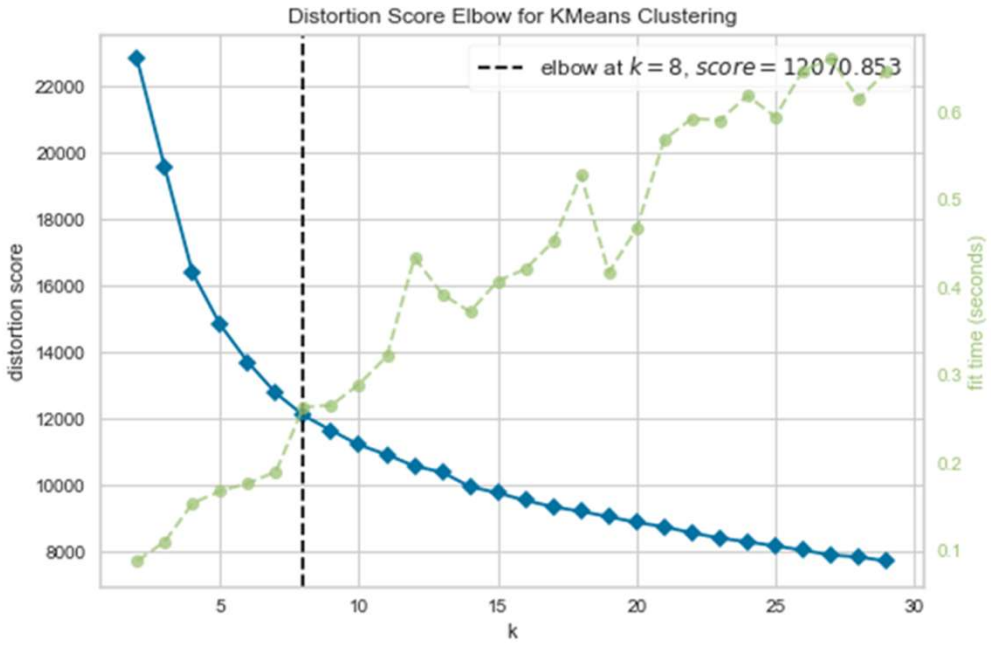
Silhouette coefficient

$$s(i)=[b(i)-a(i)]/\max(a(i),b(i))$$

- a(i) is the distance between point i and other points in its cluster
- b(j) is the smallest mean distance of point i to all points in any other cluster
- $-1 \leq s(i) \leq +1$
- Higher s(i) means better clustering
- Silhouette score is the average of the Silhouette coefficient for all points (average of s(i))

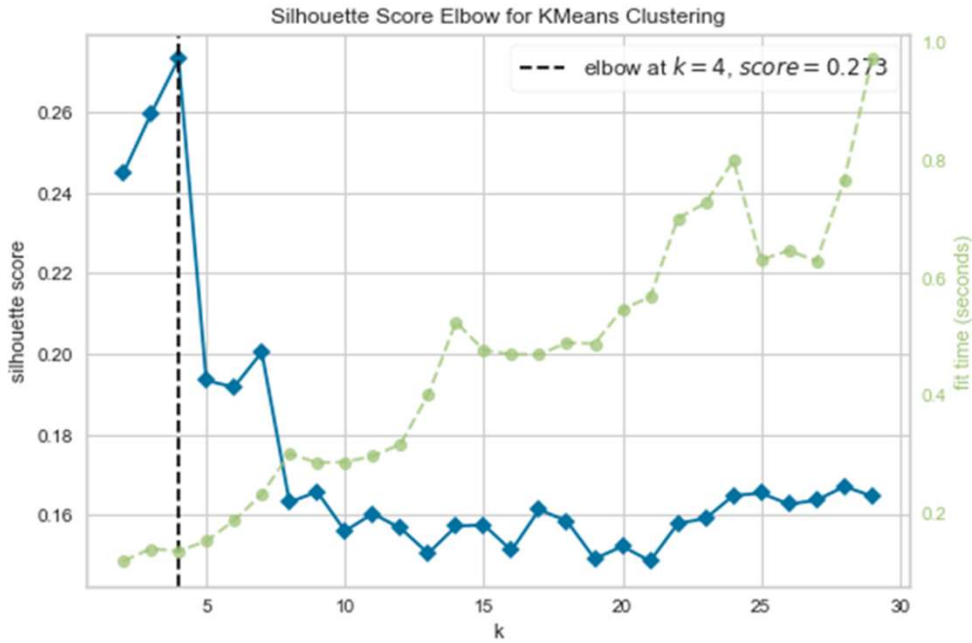
Select the optimal number of clusters

Elbow Method



8 Clusters

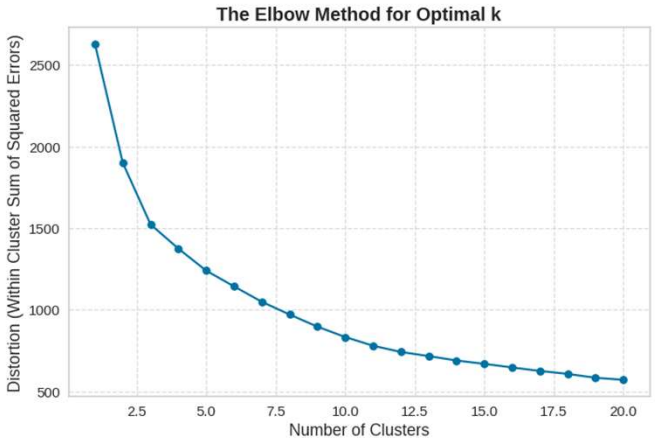
Silhouette Method

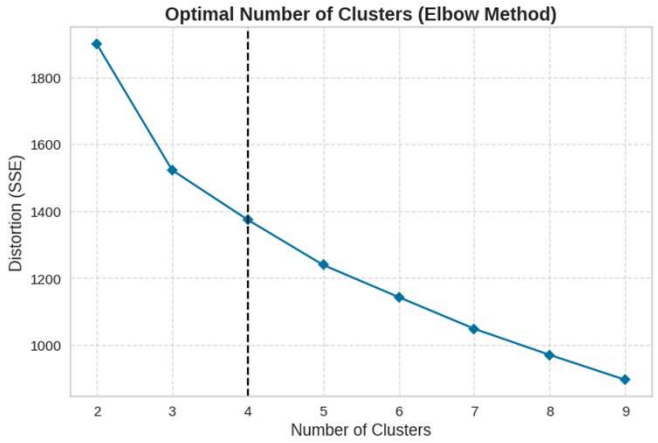


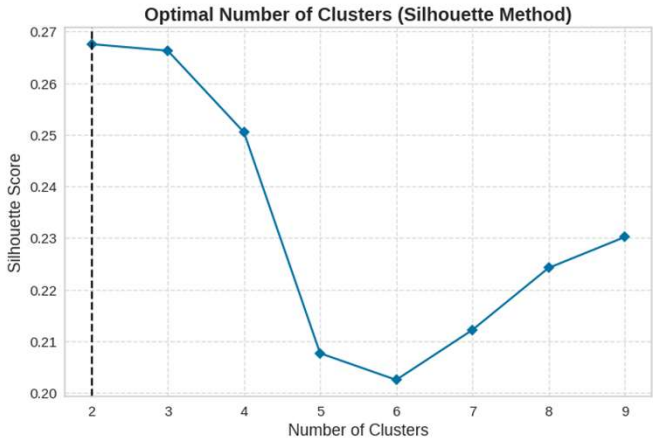
4 Clusters

Code: Let us try *k*-means clustering

- What is the optimal number of clusters? Why?







Limitations of *K*-means clustering

- *k*-means clustering performs poorly when clusters

  – have different sizes

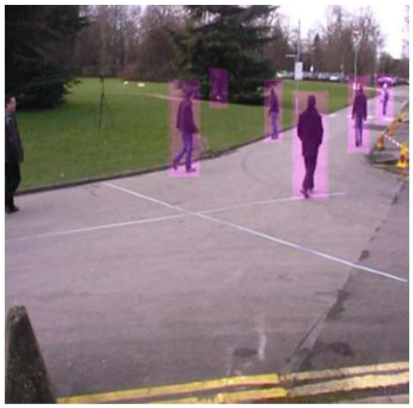
  – have different densities

  – are non-spherical shapes

- Scaling improves the performance (but does not necessarily yield acceptable results)

Image segmentation is a common application of *k*-means clustering

- Color segmentation groups pixels with a similar color (example: determine which fraction of Canada's forest remained intact after wildfire)



Ullah et al. 2018

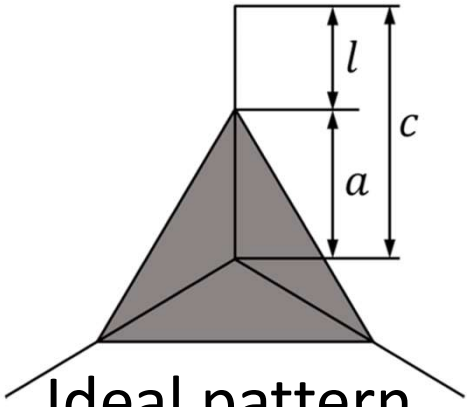
- Semantic segmentation clusters pixels as part of an object (example: identify all pedestrians as a single group)

- Instance segmentation identifies the boundaries of individual objects (example: identify people separately)

Bolya et al. 2019

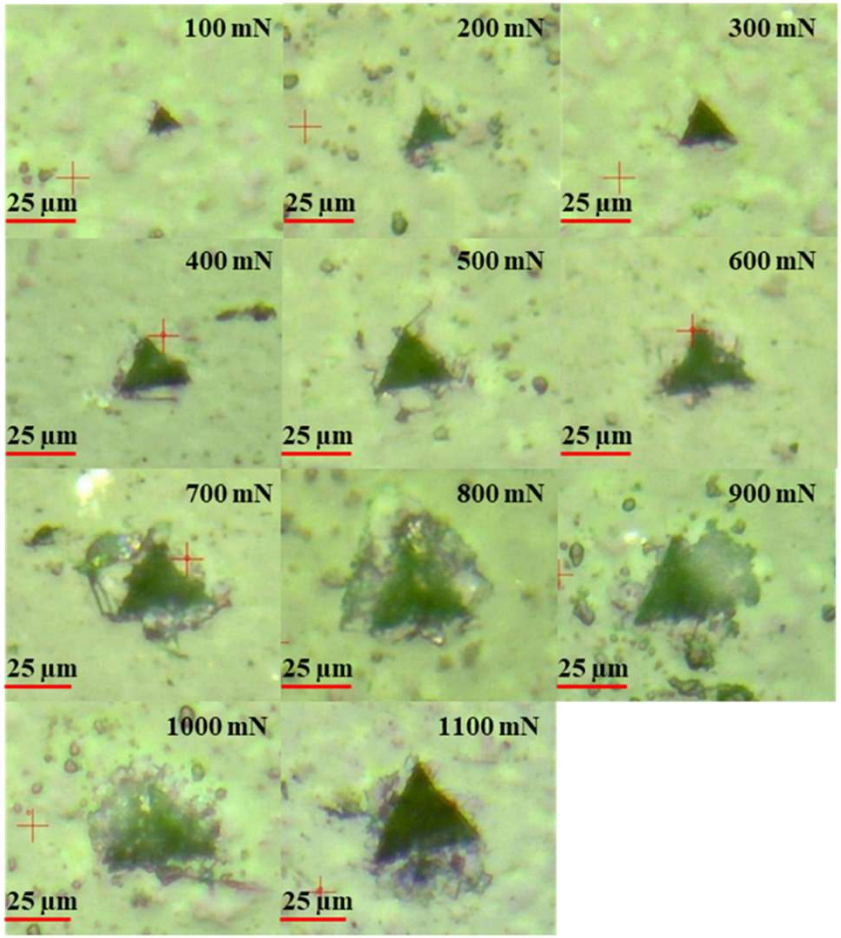
Another example: Fracture toughness characterization of shale

- Fracture toughness shows rock resistance against creating new fractures

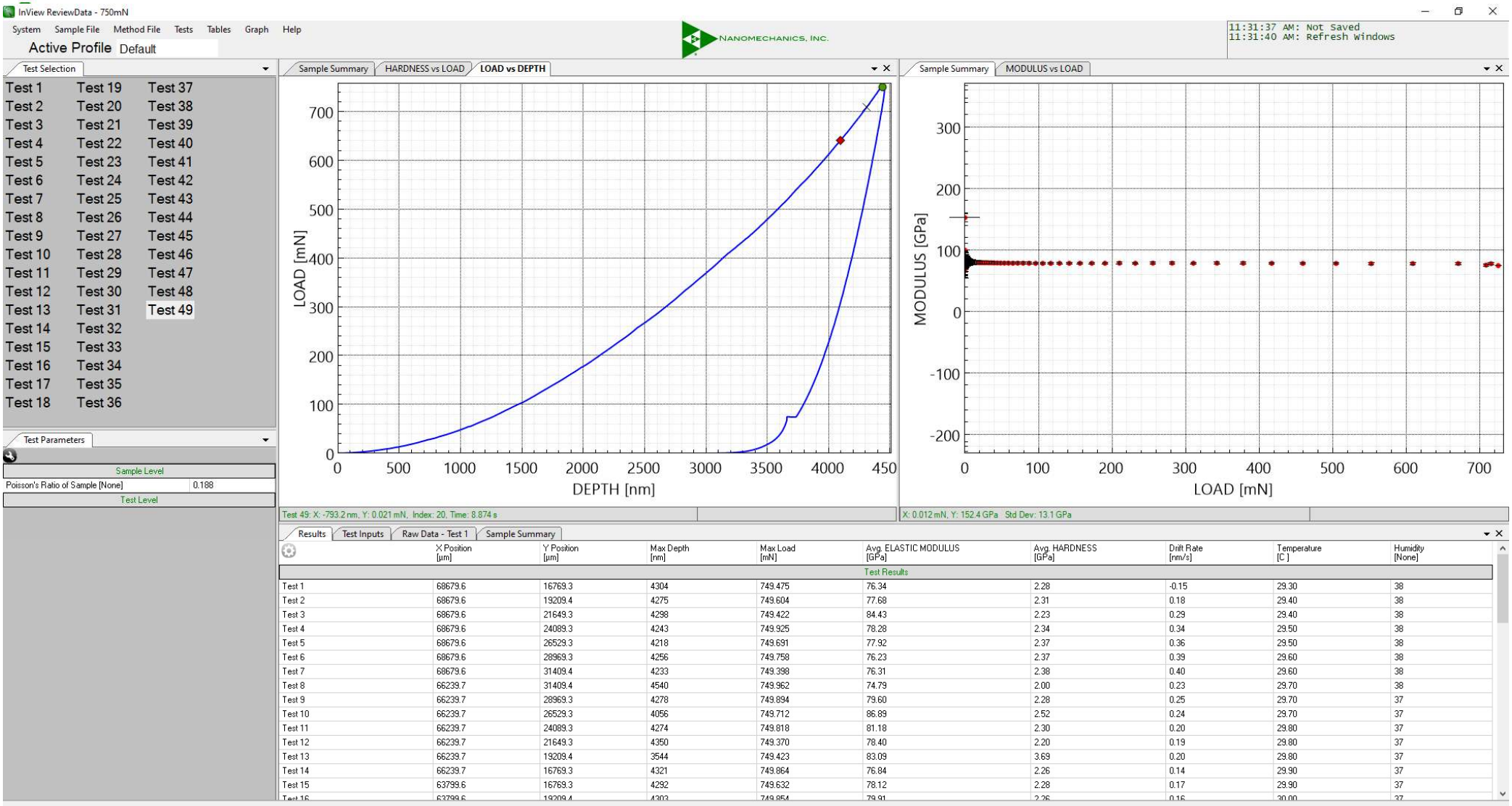


Ideal pattern
(solid mechanics)

$$K_c = \alpha \left(\frac{E_s}{H}\right)^{\frac{1}{2}} \frac{F_{\max}}{c^{\frac{3}{2}}}$$

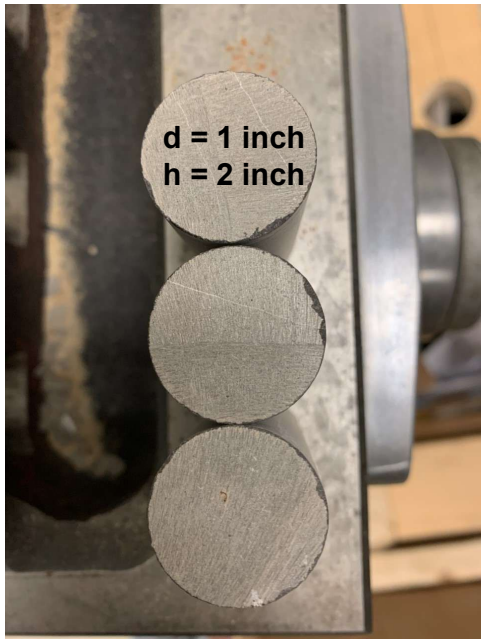


Actual pattern
(shale)


InView ReviewData - 750mN
System Sample File Method File Tests Tables Graph Help
NANOMECHANICS, INC.
11:31:37 AM: Not Saved
11:31:40 AM: Refresh Windows
Active Profile Default
Test Selection
Test 1 Test 19 Test 37
Test 2 Test 20 Test 38
Test 3 Test 21 Test 39
Test 4 Test 22 Test 40
Test 5 Test 23 Test 41
Test 6 Test 24 Test 42
Test 7 Test 25 Test 43
Test 8 Test 26 Test 44
Test 9 Test 27 Test 45
Test 10 Test 28 Test 46
Test 11 Test 29 Test 47
Test 12 Test 30 Test 48
Test 13 Test 31 Test 49
Test 14 Test 32
Test 15 Test 33
Test 16 Test 34
Test 17 Test 35
Test 18 Test 36
Test Parameters
Sample Level
Poisson's Ratio of Sample [None] 0.188
Test Level
Sample Summary HARDNESS vs LOAD LOAD vs DEPTH
LOAD [mN]
DEPTH [nm]
Test 49: X: -793.2 nm, Y: 0.021 mN, Index: 20, Time: 8.874 s
Sample Summary MODULUS vs LOAD
MODULUS [GPa]
LOAD [mN]
X: 0.012 mN, Y: 152.4 GPa Std Dev: 13.1 GPa
Results Test Inputs Raw Data - Test 1 Sample Summary
X Position [µm] Y Position [µm] Max Depth [nm] Max Load [mN] Avg. ELASTIC MODULUS [GPa] Avg. HARDNESS [GPa] Drift Rate [nm/s] Temperature [C] Humidity [None]
Test Results
Test 1 68679.6 16769.3 4304 749.475 76.34 2.28 -0.15 29.30 38
Test 2 68679.6 19209.4 4275 749.604 77.68 2.31 0.18 29.40 38
Test 3 68679.6 21649.3 4298 749.422 84.43 2.23 0.29 29.40 38
Test 4 68679.6 24089.3 4243 749.925 78.28 2.34 0.34 29.50 38
Test 5 68679.6 26529.3 4218 749.691 77.92 2.37 0.36 29.50 38
Test 6 68679.6 28969.3 4256 749.758 76.23 2.37 0.39 29.60 38
Test 7 68679.6 31409.4 4233 749.398 76.31 2.38 0.40 29.60 38
Test 8 66239.7 31409.4 4540 749.962 74.79 2.00 0.23 29.70 38
Test 9 66239.7 28969.3 4278 749.894 79.60 2.28 0.25 29.70 37
Test 10 66239.7 26529.3 4056 749.712 86.89 2.52 0.24 29.70 37
Test 11 66239.7 24089.3 4274 749.818 81.18 2.30 0.20 29.80 37
Test 12 66239.7 21649.3 4350 749.370 78.40 2.20 0.19 29.80 37
Test 13 66239.7 19209.4 3544 749.423 83.09 3.69 0.20 29.80 37
Test 14 66239.7 16769.3 4321 749.864 76.84 2.26 0.14 29.90 37
Test 15 63799.6 16769.3 4292 749.632 78.12 2.28 0.17 29.90 37

tray

Sample preparation for standard triaxial test

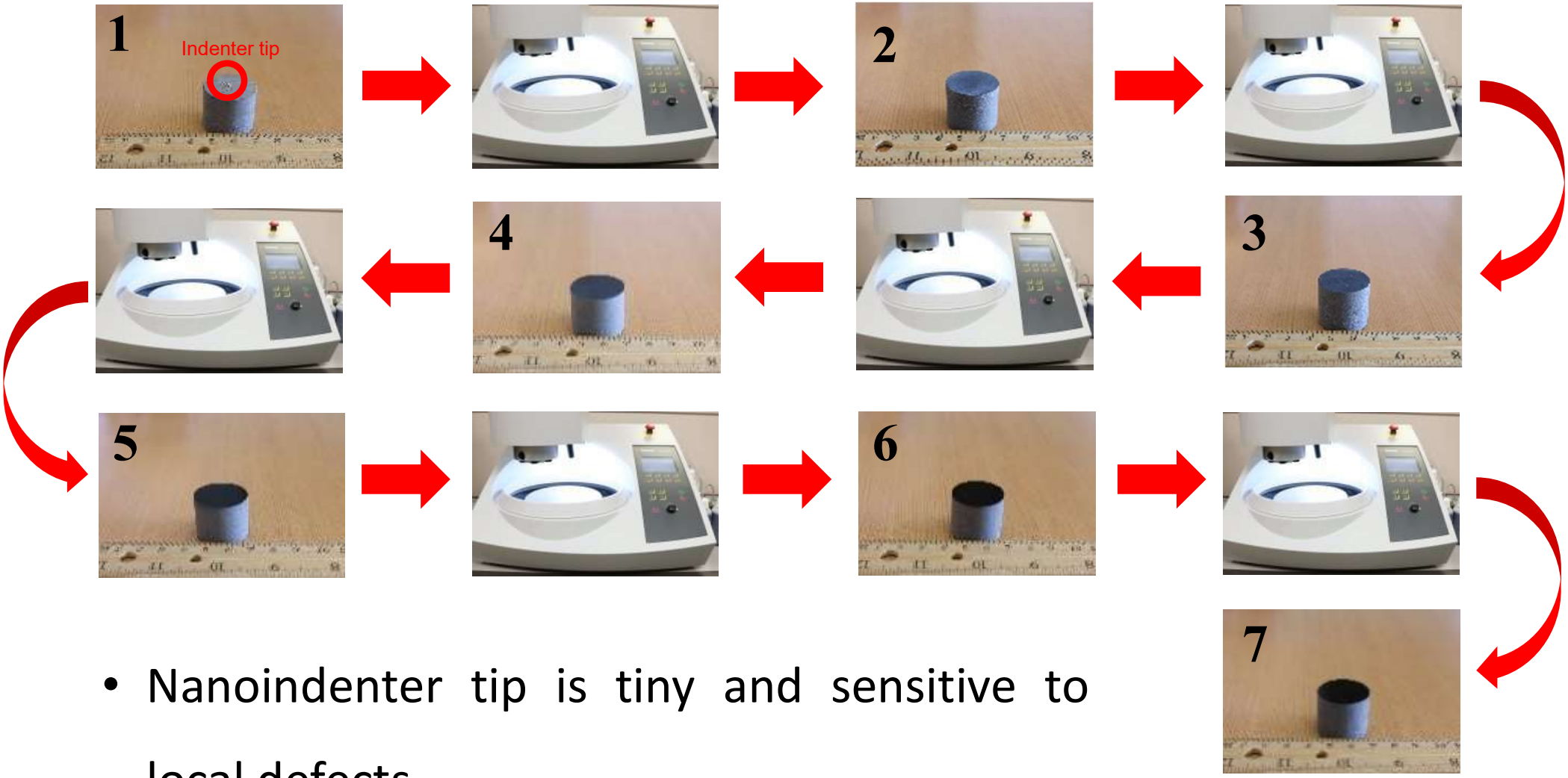


Original sample

Preparation: ~ 10 minutes

Prepared sample

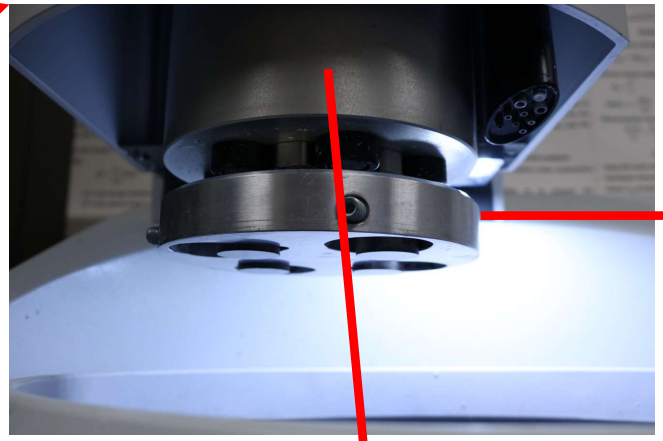
Sample preparation for nanoindentation

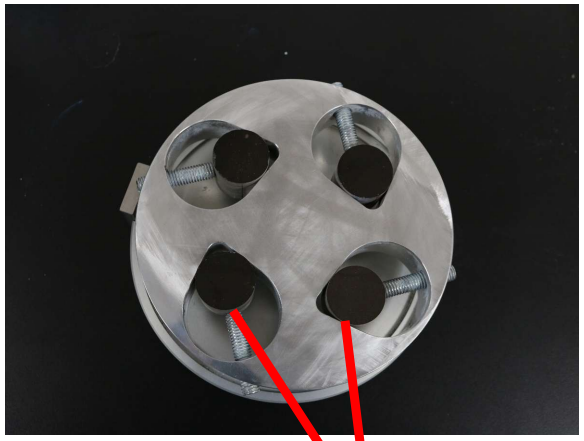


- Nanoindenter tip is tiny and sensitive to local defects

Nanogeo Lab

Surface polisher





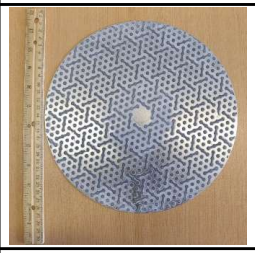
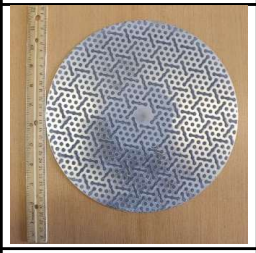
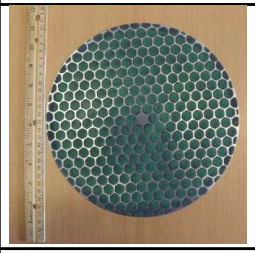
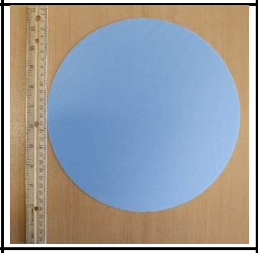
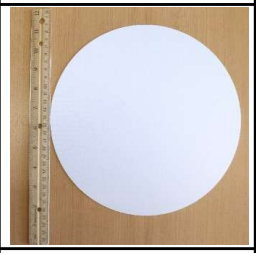
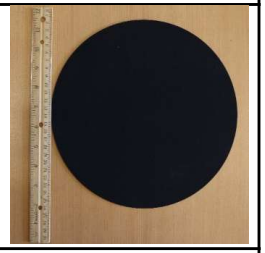
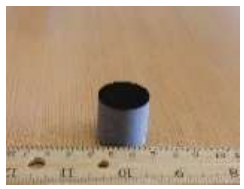
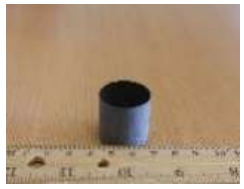
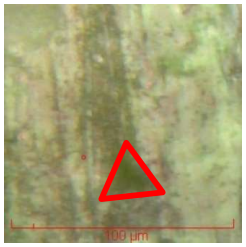
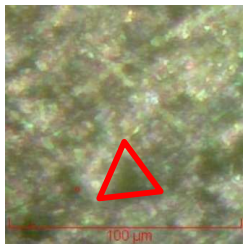
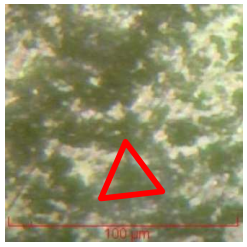
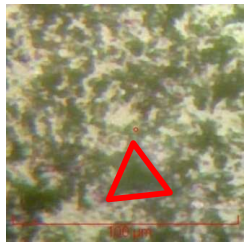
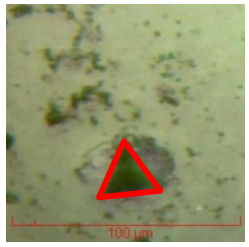
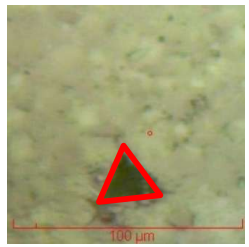
Mover plate

Suspensions

Mover head

Core plugs

Preparation discs for nanoindentation

Stages	Original (1)	MD-Piano 500 (2)	MD-Piano 1200 (3)	MD-Largo (4)	MD-Dac (5)	MD-Dur (6)	MD-Chem (7)
Disc Shape	-						
Time (mins)	0:00	1:00	1:00	5:00	4:00	4:00	2:00
Surface of Core Plug							
Results							

100 µm

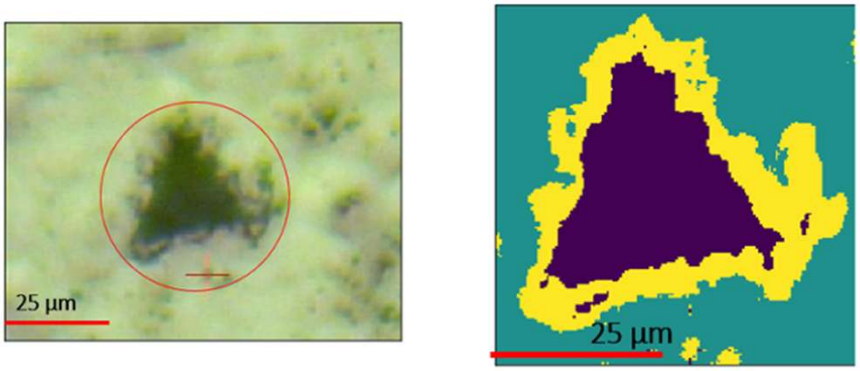
Grinding

Polishing

Image segmentation allows us to analyze complex images of nanoindentation

- Each image has a bunch of pixels (example: MNIST gray image has 28×28 pixels)

- Color image has three channels: Red, Green, and Blue (RGB)



(a) (b)

Alipour et al. 2021

To-do: read the reference to answer the listed questions

- What is active learning?
- What is uncertainty sampling?
- What is novelty detection?

Gaussian mixture model (GMM)

- It is based on the notion that data belong to a mixture of Gaussian distributions

- It is used for clustering

- It is also used for anomaly detection

- We must specify the number of Gaussian distributions (this is a generalization of *k*-means clustering)

GMM workflow

- It randomly initializes the parameters (mean and covariance) of the distributions

- Soft clustering: It assigns samples to the clusters randomly with some weights (this is different from *k*-means clustering)

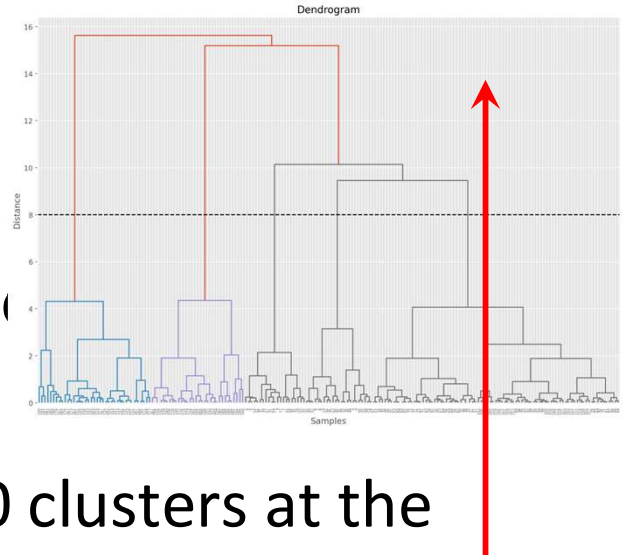
- It uses expectation maximization (instead of distance minimization used in k-means clustering)

Hierarchical clustering

- Hierarchical clustering uses a distance-based algorithm to measure the distance between clusters (similar to *k*-means clustering)

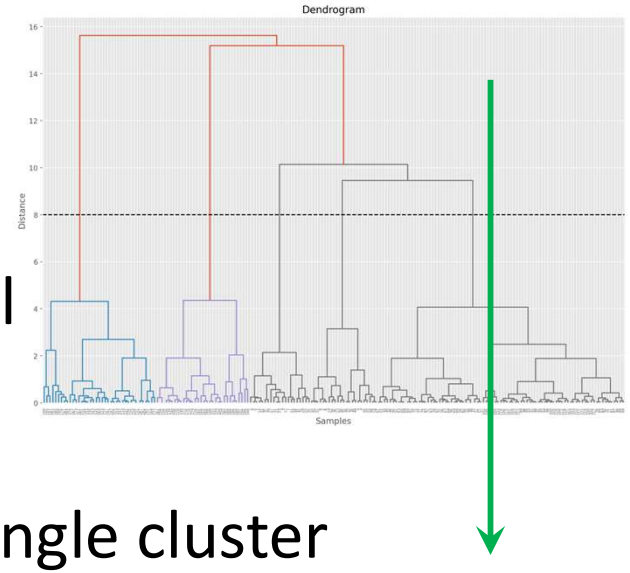
- Two types of hierarchical clustering are discussed
  - Additive hierarchical clustering (agglomerative hierarchical clustering)
  - Divisive hierarchical clustering

Additive hierarchical clustering (agglomerative hierarchical clustering)



- This is a bottom-up approach
- First, every point will be assigned its uniqu
- If there are 10 data points, there will be 10 clusters at the beginning
- Then, the closest pair of clusters are merged based on their distance
- This process is iterated to create a single cluster

Divisive hierarchical clustering (top-down)



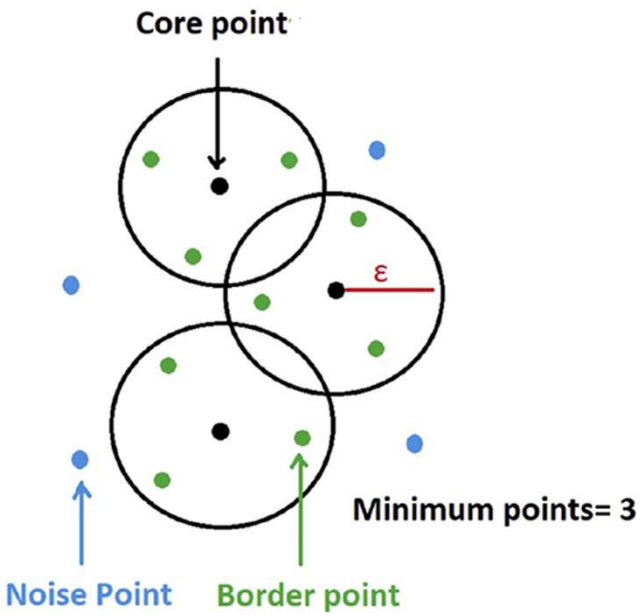
- This is a top-down approach
- This is the opposite of additive hierarchical clustering
- First, all data points will be assigned to a single cluster
- The farthest point is then split in the cluster
- This process continues so each cluster has a single point

Density-based spatial clustering of applications with noise (DBSCAN)

- This approach divides the data into high-density and low-density regions
- It does not predict after training (we cannot use it to assign a new instance to a cluster)
- It has two tuning parameters (next slide)

Tuning parameters of DBSCAN

- ε: The radius of the circle that shows how close instances must be from one another to create a cluster
- MinPts: The minimum number of instances within the radius of a data point to be considered as a core point
- Core instance: An instance with at least MinPts in its ε neighborhood
- Noise (anomaly): An instance that is not a core instance and has no core instance in its ε neighborhood



Advantages of DBSCAN

- It is robust to outliers (use this method instead of *k*-means clustering when you are unsure)

- It detects anomaly (noise)

- It can detect clusters with arbitrary shapes

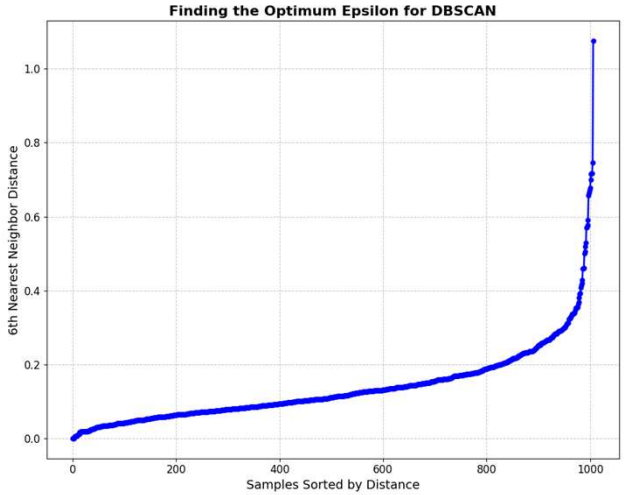
Disadvantages of DBSCAN

- It performs poorly if clusters have very different densities

- It does not scale well to large datasets because of its computational cost

Code: Let us try DBSCAN

- Q1: What are the tuning parameters in DBSCAN?
- Q2: How does this code find epsilon?



- Q3: How many clusters are created?
- Q4: How can we tune the parameters more realistically?

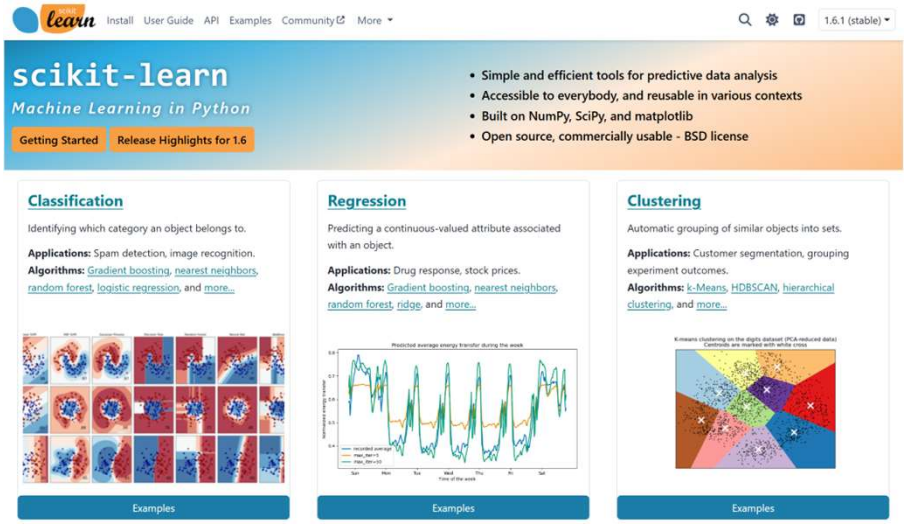
Two common clustering techniques

- *K*-means clustering is quite basic but works in most scenarios in practice

- DBSCAN helps you discard some samples as noise (this is useful in dealing with actual subsurface data with uncertainty)

Other clustering techniques

- BIRCH
- Mean-shift
- Affinity propagation
- Spectral clustering



Do not hesitate to try other clustering methods IF the common methods do not work for your data. You can compare the performance to pick the best model.